Introduction to Testing

Dr. Durga Prasad Mohapatra
Professor
Department Of CSE
NIT, Rourkela.

Defect Reduction Techniques

- Review
- Testing
- Formal verification
- Development process
- Systematic methodologies

Why Test?





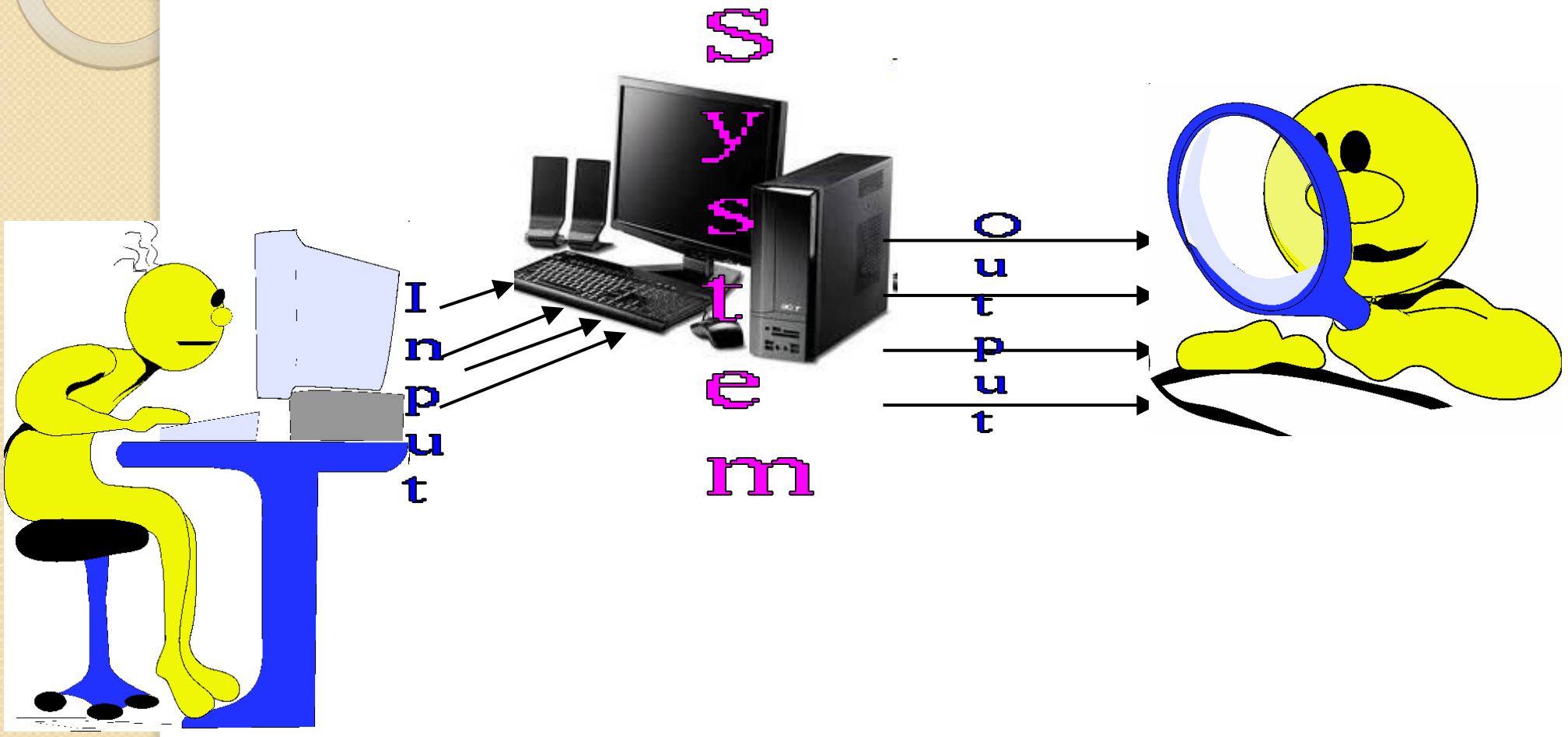


- Ariane 5 rocket self-destructed 37 seconds after launch
- Reason: A control software bug that went undetected
  - Conversion from 64-bit floating point to 16-bit signed integer value had caused an exception
    - The floating point number was larger than 32767
    - Efficiency considerations had led to the disabling of the exception handler.
- Total Cost: over $1 billion

How Do You Test a Program?

- Input test data to the program.
- Observe the output:
  - Check if the program behaved as expected.

How Do You Test a Program?



How Do You Test a Program?

- If the program does not behave as expected:
  - Note the conditions under which it failed.
  - Later debug and correct.

What's So Hard About Testing ?

- Consider **int proc1(int x, int y)**
- Assuming a 64 bit computer
  - Input space = 2^{128}
- Assuming it takes 10secs to key-in an integer pair
  - It would take about a billion years to enter all possible values!
  - Automatic testing has its own problems!

Testing Facts

- Consumes largest effort among all phases
  - Largest manpower among all other development roles
  - Implies more job opportunities
- About 50% development effort
  - But 10% of development time?
  - How?

Testing Facts

- Testing is getting more complex and sophisticated every year.
  - Larger and more complex programs
  - Newer programming paradigms

Overview of Testing Activities

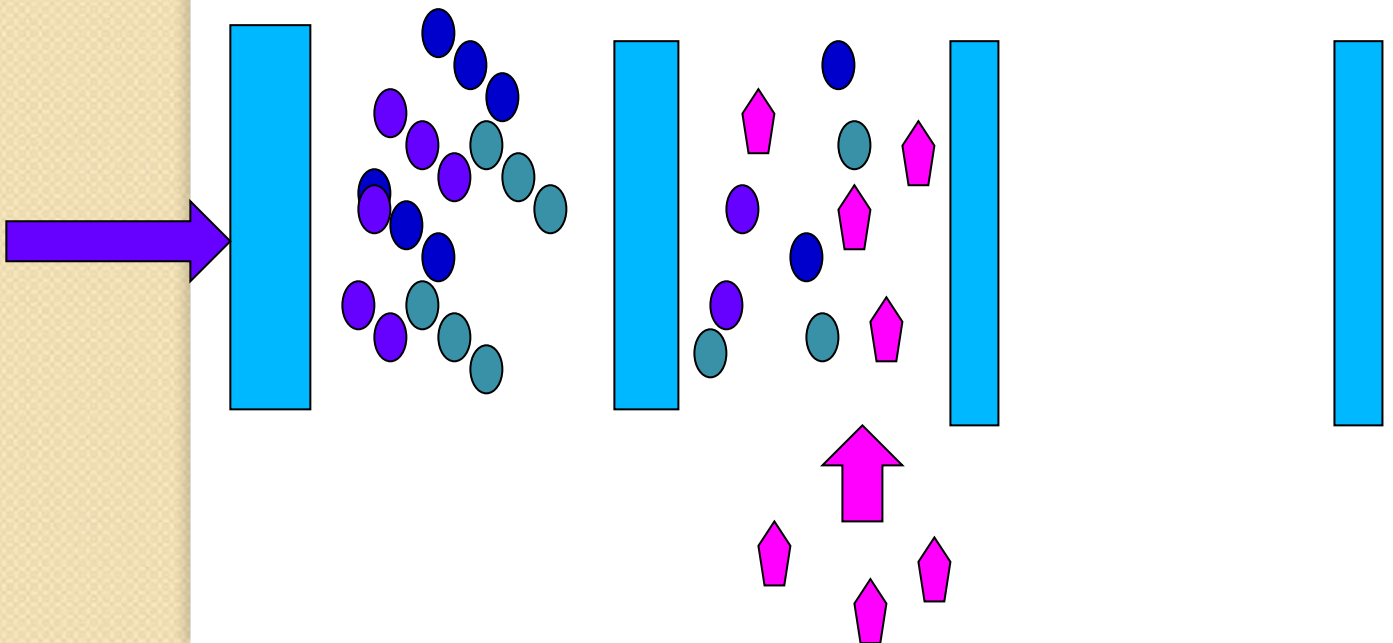
- Test Suite Design
- Run test cases and observe results to detect failures.
- Debug to locate errors
- Correct errors.

Error, Faults, and Failures

- A failure is a manifestation of an error (also defect or bug).
  - Mere presence of an error may not lead to a failure.

Pesticide Effect

- Errors that escape a fault detection technique:
  - Can not be detected by further applications of that technique.



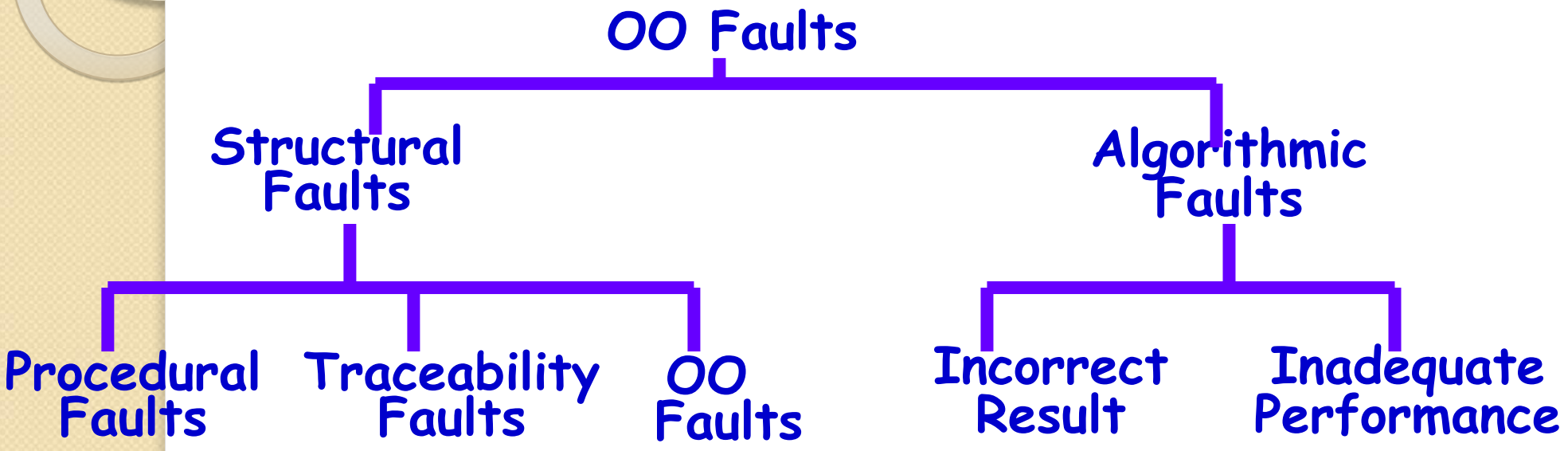
Pesticide Effect

- Assume we use 4 fault detection techniques and 1000 bugs:
  - Each detects only 70% bugs
  - How many bugs would remain
  - $1000*(0.3)^4=81$ bugs

Fault Model

- Types of faults possible in a program.
- Some types can be ruled out
  - Concurrency related-problems in a sequential program

Fault Model of an OO Program



Hardware Fault-Model

- Simple:
  - Stuck-at 0
  - Stuck-at 1
  - Open circuit
  - Short circuit
- Simple ways to test the presence of each
- Hardware testing is fault-based testing

Software Testing

- Each test case typically tries to establish correct working of some functionality
  - Executes (covers) some program elements
  - For restricted types of faults, fault-based testing exists.

Test Cases and Test Suites

- Test a software using a set of carefully designed test cases:
  - The set of all test cases is called the test suite

Test Cases and Test Suites

- A test case is a triplet [I,S,O]
  - I is the data to be input to the system,
  - S is the state of the system at which the data will be input,
  - O is the expected output of the system.

Aim of Testing

- The aim of testing is to identify all defects in a software product.
- However, in practice even after thorough testing:
  - one cannot guarantee that the software is error-free.

Aim of Testing

- The input data domain of most software products is very large:
  - it is not practical to test the software exhaustively with each input data value.

Aim of Testing

- Testing does however expose many errors:
  - testing provides a practical way of reducing defects in a system
  - increases the users' confidence in a developed system.

Aim of Testing

- Testing is an important development phase:
  - requires the maximum effort among all development phases.
- In a typical development organization:
  - maximum number of software engineers can be found to be engaged in testing activities.

Aim of Testing

- Many engineers have the wrong impression:
  - testing is a secondary activity
  - it is intellectually not as stimulating as the other development activities, etc.

Aim of Testing

- Testing a software product is in fact:
  - as much challenging as initial development activities such as specification, design, and coding.
- Also, testing involves a lot of creative thinking.

Levels of Testing

- Software products are tested at three levels:
  - Unit testing
  - Integration testing
  - System testing

Unit testing

- During unit testing, modules are tested in isolation:
  - If all modules were to be tested together:
    - it may not be easy to determine which module has the error.

Unit testing

- Unit testing reduces debugging effort several folds.
  - Programmers carry out unit testing immediately after they complete the coding of a module.

Integration testing

- After different modules of a system have been coded and unit tested:
  - modules are integrated in steps according to an integration plan
  - partially integrated system is tested at each integration step.

System Testing

- System testing involves:
  - validating a fully developed system against its requirements.

Verification versus Validation

- Verification is the process of determining:
  - Whether output of one phase of development conforms to its previous phase.
- Validation is the process of determining:
  - Whether a fully developed system conforms to its SRS document.

Verification versus Validation

- Verification is concerned with phase containment of errors,
  - Whereas the aim of validation is that the final product be error free.

Design of Test Cases

- Exhaustive testing of any non-trivial system is impractical:
  - Input data domain is extremely large.
- Design an optimal test suite:
  - Of reasonable size and
  - Uncovers as many errors as possible.

Design of Test Cases

- If test cases are selected randomly:
  - Many test cases would not contribute to the significance of the test suite,
  - Would not detect errors not already being detected by other test cases in the suite.
- Number of test cases in a randomly selected test suite:
  - Not an indication of effectiveness of testing.

Design of Test Cases

- Testing a system using a large number of randomly selected test cases:
  - Does not mean that many errors in the system will be uncovered.
- Consider following example:
  - Find the maximum of two integers x and y.

Design of Test Cases

- The code has a simple programming error:
- If (x>y) max = x;
  else max = y;
- Test suite {(x=3,y=2);(x=2,y=3)} can detect the error,
- A larger test suite {(x=3,y=2);(x=4,y=3); (x=5,y=1)} does not detect the error.



Design of Test Cases

- Systematic approaches are required to design an optimal test suite:
  - Each test case in the suite should detect different errors.

Design of Test Cases

- There are essentially three main approaches to design test cases:
  - Black-box approach
  - White-box (or glass-box) approach
  - Grey-box (or model based) approach

Black-Box Testing

- Test cases are designed using only functional specification of the software:
  - Without any knowledge of the internal structure of the software.
- For this reason, black-box testing is also known as functional testing.

Black-box Testing Techniques

- There are many approaches to design black box test cases:
  - Equivalence class partitioning
  - Boundary value analysis
  - State table based testing
  - Decision table based testing
  - Cause-effect graph based testing
  - Orthogonal array testing
  - Positive-negative testing

White-box Testing

- Designing white-box test cases:
  - Requires knowledge about the internal structure of software.
  - White-box testing is also called structural testing.

White-Box Testing Techniques

- There exist several popular white-box testing methodologies:
  - Statement coverage
  - Branch coverage
  - Path coverage
  - Condition coverage
  - MC/DC coverage
  - Mutation testing
  - Data flow-based testing

Coverage-Based Testing Versus Fault-Based Testing

- Idea behind coverage-based testing:
  - Design test cases so that certain program elements are executed (or covered).
  - Example: statement coverage, path coverage, etc.
- Idea behind fault-based testing:
  - Design test cases that focus on discovering certain types of faults.
  - Example: Mutation testing.

Why Both BB and WB Testing?

Black-box

- Impossible to write a test case for every possible set of inputs and outputs
- Some code parts may not be reachable
- Does not tell if extra functionality has been implemented.

White-box

- Does not address the question of whether or not a program matches the specification
- Does not tell you if all of the functionality has been implemented
- Does not discover missing program logic

Grey Box / Model Based Testing

- In grey box testing, test cases are designed from design documents / models, such as UML diagrams.
- Grey-box testing is also called model based testing.
- Mainly used for testing of O-O systems.

Summary

- Discussed importance of testing and the basic concepts of testing.
- Presented the levels of testing.
  - Unit testing
  - Integration testing
  - System testing
- Discussed the fundamentals of black box testing, white box testing and grey box testing.

References

1. R. Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.

Thank You

Black-box testing techniques

Black-box Testing

- Test cases are designed using only functional specification of the software:
  - without any knowledge of the internal structure of the software.
- For this reason, black-box testing is also known as functional testing.

Errors detected by black box testing

- Black-box testing attempts to find errors in the following categories:

1. To test the modules independently .
2. To test the functional validity of the software so that incorrect or missing functions can be recognized.
3. To look for interface errors. □

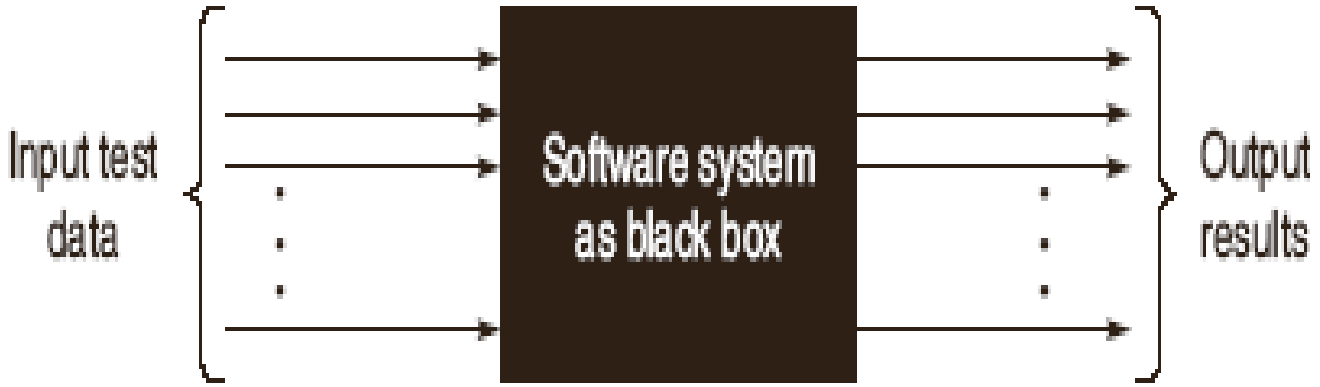
Errors detected by black box testing

4. To test the system behavior and check its performance □
5. To test the maximum load or stress on the system.
6. To test the software such that the user/customer accepts the system within defined acceptable limits.

Black-box Testing Techniques

- There are many approaches to design black box test cases:
  - Equivalence class partitioning
  - Boundary value analysis
  - State table based testing
  - Decision table based testing
  - Cause-effect graph based testing
  - Orthogonal array testing
  - Positive-negative testing

Black-box Testing



Equivalence Class Partitioning

- Input values to a program are partitioned into equivalence classes.
- Partitioning is done such that:
  - program behaves in similar ways to every input value belonging to an equivalence class.

Why define equivalence classes?

- Test the code with just one representative value from each equivalence class:
  - as good as testing using any other values from the equivalence classes.

Equivalence Class Partitioning

- How do you determine the equivalence classes?
  - examine the input data.
  - few general guidelines for determining the equivalence classes can be given

Equivalence Class Partitioning

- If the input data to the program is specified by a range of values:
  - e.g. numbers between 1 to 5000.
  - one valid and two invalid equivalence classes are defined.

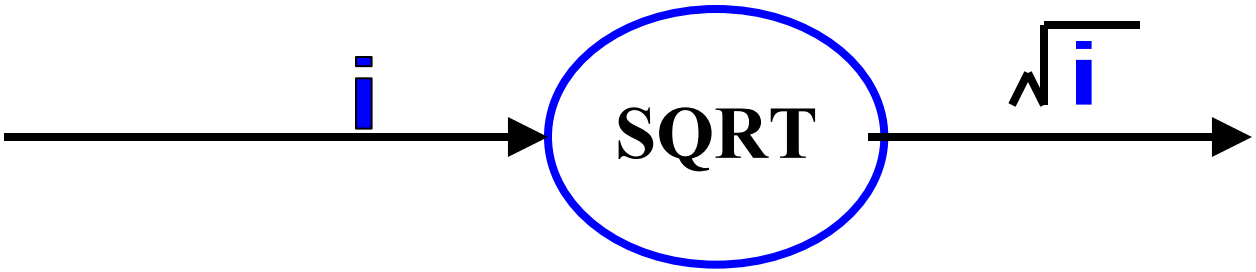


Equivalence Class Partitioning

- If input is an enumerated set of values:
  - e.g. {a,b,c}
  - one equivalence class for valid input values
  - another equivalence class for invalid input values should be defined.

Example - 1

- A program reads an input value in the range of 1 and 5000:
  - • Computes the square root of the input number



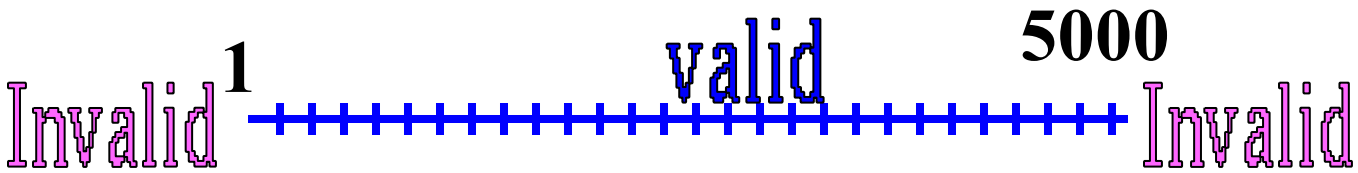
Example - 1 (cont.)

- There are three equivalence classes:
  - the set of negative integers,
  - set of integers in the range of 1 and 5000,
  - integers larger than 5000.



Example - 1 (cont.)

- The test suite must include:
  - representatives from each of the three equivalence classes:
  - a possible test suite can be: {-5,500,6000}.



Example - 2

- A program reads three numbers, A, B, and C, with a range [1, 50] and prints the largest number. Design test cases for this program using equivalence class testing technique.

Solution

1. First we partition the domain of input as valid input values and invalid values, getting the following classes:

- I1 = {<A, B, C> : 1 ≤ A ≤ 50}
- I2 = {<A, B, C> : 1 ≤ B ≤ 50}
- I3 = {<A, B, C> : 1 ≤ C ≤ 50}
- I4 = {<A, B, C> : A < 1}

Solution

- I5 = {<A, B, C> : A > 50}
- I6 = {<A, B, C> : B < 1}
- I7 = {<A, B, C> : B > 50}
- I8 = {<A, B, C> : C < 1}
- I9 = {<A, B, C> : C > 50}

Solution

- Now the test cases can be designed from the above derived classes, taking
  - one test case from each class such that the test case covers maximum valid input classes, and
  - separate test cases for each invalid class.

Solution

- The test cases are shown below:

Test case ID	A	B	C	Expected result	Classes covered by the test case
1	13	25	36	C is greatest	I_1, I_2, I_3
2	0	13	45	Invalid input	I_4
3	51	34	17	Invalid input	I_5
4	29	0	18	Invalid input	I_6
5	36	53	32	Invalid input	I_7
6	27	42	0	Invalid input	I_8
7	33	21	51	Invalid input	I_9

Solution

2. We can derive another set of equivalence classes based on some possibilities for three integers, A, B, and C. These are given below:

- I1 = {<A, B, C> : A > B, A > C}
- I2 = {<A, B, C> : B > A, B > C}
- I3 = {<A, B, C> : C > A, C > B}

Solution

- I4 = {<A, B, C> : A = B, A ≠ C}
- I5 = {<A, B, C> : B = C, A ≠ B}
- I6 = {<A, B, C> : A = C, C ≠ B}
- I7 = {<A, B, C> : A = B = C}

Solution

Test case ID	A	B	C	Expected Result	Classes Covered by the test case
1	25	13	13	A is greatest	I_1, I_5
2	25	40	25	B is greatest	I_2, I_6
3	24	24	37	C is greatest	I_3, I_4
4	25	25	25	All three are equal	I_7

Example - 3

A program determines the next date in the calendar. Its input is entered in the form of with the following range:

- $1 \leq mm \leq 12$
- $1 \leq dd \leq 31$
- $1900 \leq yyyy \leq 2025$

Example

- Its output would be the next date or an error message ‘invalid date.’ Design test cases using equivalence class partitioning method.

Solution

First, we partition the domain of input in terms of valid input values and invalid values, getting the following classes:

- I1 = { <m, d, y> : 1 ≤ m ≤ 12}
- I2 = {<m, d, y> : 1 ≤ d ≤ 31}
- I3 = {<m, d, y> : 1900 ≤ y ≤ 2025}
- I4 = {<m, d, y> : m < 1}

Solution

- I5 = {<m, d, y> : m > 12}
- I6 = {<m, d, y> : d < 1}
- I7 = {<m, d, y> : d > 31}
- I8 = {<m, d, y> : y < 1900}
- I9 = {<m, d, y> : y > 2025}

Solution

- The test cases can be designed from the above derived classes,
  - taking one test case from each class such that the test case covers maximum valid input classes, and
  - separate test cases for each invalid class.
- The test cases are shown in next slide.

Solution

Test case ID	mm	dd	yyyy	Expected result	Classes covered by the test case
1	5	20	1996	21-5-1996	I_1, I_2, I_3
2	0	13	2000	Invalid input	I_4
3	13	13	1950	Invalid input	I_5
4	12	0	2007	Invalid input	I_6
5	6	32	1956	Invalid input	I_7
6	11	15	1899	Invalid input	I_8
7	10	19	2026	Invalid input	I_9

Example - 4

- A program takes an angle as input within the range [0, 360] and determines in which quadrant the angle lies. Design test cases using equivalence class partitioning method.

Solution

1. First, we partition the domain of input as valid and invalid classes as follows:

- I1 = {<Angle> : 0 ≤ Angle ≤ 360}
- I2 = {<Angle> : Angle < 0}
- I3 = {<Angle> : Angle > 360}

Solution

- The test cases designed from these classes are shown below:

Test Case ID	Angle	Expected results	Classes covered by the test case
1	50	I Quadrant	I_1
2	–1	Invalid input	I_2
3	361	Invalid input	I_3

Solution

2. The classes can also be prepared based on the output criteria as shown below:

- O1 = {<Angle>: First Quadrant, if 0 ≤ Angle ≤ 90}
- O2 = {<Angle>: Second Quadrant, if 91 ≤ Angle ≤ 180}
- O3 = {<Angle>: Third Quadrant, if 181 ≤ Angle ≤ 270}

Solution

- O4 = {<Angle>: Fourth Quadrant, if 271 ≤ Angle ≤ 360}
- O5 = {<Angle>: Invalid Angle};
- However, O5 is not sufficient to cover all invalid conditions this way. Therefore, it must be further divided into equivalence classes as shown in next slide:

Solution

- $O5_1$={<Angle>: Invalid Angle, if Angle > 360}
- $O5_2$={<Angle>: Invalid Angle, if Angle < 0}

Solution

- Now, the test cases can be designed from the above derived classes as shown below:

Test Case ID	Angle	Expected results	Classes covered by the test case
1	50	I Quadrant	O_1
2	135	II Quadrant	O_2
3	250	III Quadrant	O_3
4	320	IV Quadrant	O_4
5	370	Invalid angle	O_{51}
6	–1	Invalid angle	O_{52}

Few more examples

- Student Marks Validation
- User login
- ATM withdrawal
- Password length
- Voting system
- Online shopping coupon code
- Mobile number verification
- File upload
- Email validation

Why use ECP?

- Reduces the number of test cases
- Saves testing time
- Provides good test coverage
- Detects input validation defects efficiently

Summary

- Discussed the errors detected by black-box test testing techniques.
- Explained equivalence partitioning technique with some examples.

Practice Exercise

- An airline booking system allows booking 1–9 passengers in one reservation. Design Equivalence Class Partitioning test cases.
- An e-commerce application has the following validation rules:
  - Quantity: 1–20
  - Coupon code: Exactly 8 characters
  - Mobile number: Exactly 10 digits
  - Order value: $50–$5,000

  Design complete Equivalence Class Partitioning test cases.

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 4), Second Edition, Oxford University Press, 2016.

Thank You

Boundary Value Analysis

Boundary Value Analysis

- Some typical programming errors occur:
  - at boundaries of equivalence classes
  - might be purely due to psychological factors.
- Programmers often fail to see:
  - special processing required at the boundaries of equivalence classes.

Boundary Value Analysis

- Programmers may improperly use < instead of <=
- Boundary value analysis:
  - select test cases at the boundaries of different equivalence classes.

Example

- For a function that computes the square root of an integer in the range of 1 and 5000:
  - test cases must include the values {0,1,5000,5001} along with the values obtained from Equivalence partitioning.



BOUNDARY VALUE ANALYSIS (BVA)

- BVA offers several methods to design test cases. Following are the few methods used:
- **1.** BOUNDARY VALUE CHECKING (BVC)
- **2.** ROBUSTNESS TESTING METHOD
- **3.** WORST-CASE TESTING METHOD
- **4.** ROBUST WORST-CASE TESTING METHOD

BOUNDARY VALUE CHECKING (BVC)

- In this method, the test cases are designed by holding one variable at its extreme value and other variables at their nominal values in the input domain.

- The variable at its extreme value can be selected at:

BOUNDARY VALUE CHECKING (BVC)

- (a) Minimum value (Min)
- (b) Value just above the minimum value (Min+)
- (c) Maximum value (Max)
- (d) Value just below the maximum value (Max−)

BOUNDARY VALUE CHECKING (BVC)

- Let us take the example of two variables, A and B.
- If we consider all the above combinations with nominal values, then following test cases

  (see Fig. 1) can be designed:

- 1. Anom, Bmin
- 2. Anom, Bmin+
- 3. Anom, Bmax
- 4. Anom, Bmax–
- 5. Amin, Bnom
- 6. Amin+, Bnom
- 7. Amax, Bnom
- 8. Amax–, Bnom
- 9. Anom, Bnom

BOUNDARY VALUE CHECKING (BVC)

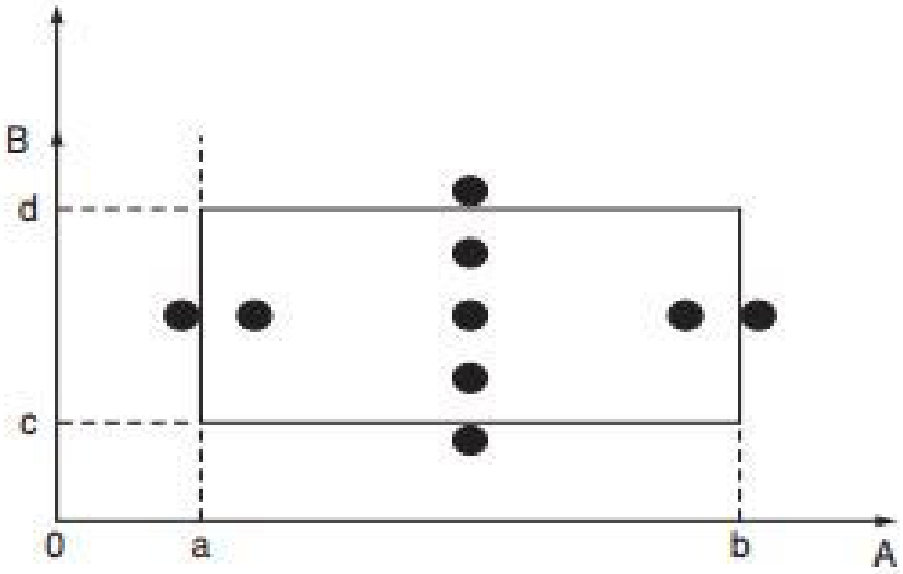


Fig 1: Boundary Value Checking

BOUNDARY VALUE CHECKING (BVC)

- It can be generalized that for n variables in a module, 4n + 1 test cases can be designed with boundary value checking method.

ROBUSTNESS TESTING METHOD

- The idea of BVC can be extended such that boundary values are exceeded as: □

1. A value just greater than the Maximum value (Max+)

2. □A value just less than Minimum value (Min−)

ROBUSTNESS TESTING METHOD

- When test cases are designed considering the above points in addition to BVC, it is called robustness testing.

- Let us take the previous example again. Add the following test cases to the list of 9 test cases designed in BVC:

- 10. Amax+, Bnom
- 11. Amin–, Bnom
- 12. Anom, Bmax+
- 13. Anom, Bmin–

ROBUSTNESS TESTING METHOD

- It can be generalized that for n input variables in a module, 6n + 1 test cases can be designed with robustness testing.

WORST-CASE TESTING METHOD

- We can again extend the concept of BVC by assuming more than one variable on the boundary.
- It is called worst-case testing method.
- Again, take the previous example of two variables, A and B. We can add the following test cases to the list of 9 test cases designed in BVC as:

WORST-CASE TESTING METHOD

10. Amin, Bmin
11. Amin+, Bmin
12. Amin, Bmin+
13. Amin+, Bmin+
14. Amax, Bmin
15. Amax–, Bmin
16. Amax, Bmin+
17. Amax–, Bmin+
18. Amin, Bmax
19. Amin+, Bmax
20. Amin, Bmax–
21. Amin+,Bmax–
22. Amax, Bmax
23. Amax–, Bmax
24. Amax, Bmax–
25. Amax–,Bmax–

WORST-CASE TESTING METHOD

- It can be generalized that for n input variables in a module, 5^n test cases can be designed with worst-case testing.

ROBUST WORST-CASE TESTING METHOD

- In the previous method, the extreme values of a variable considered are of BVC only.
- The worst case can be further extended if we consider robustness also, that is,
- in worst case testing if we consider the extreme values of the variables as in robustness testing method covered in Robustness Testing

ROBUST WORST-CASE TESTING METHOD

- Again take the example of two variables, A and B. We can add the following test cases to the list of 25 test cases designed in previous section.
- 26. Amin-, Bmin-
- 27. Amin-, Bmin
- 28. Amin, Bmin-
- 29. Amin-, Bmin+
- 30. Amin+, Bmin-
- 31. Amin-, Bmax

- 32. Amax, Bmin-
- 33. Amin-, Bmax-
- 34. Amax-, Bmin-
- 35. Amax+,Bmax+
- 36. Amax+, Bmin
- 37. Amin, Bmin+
- 38. Amax+, Bmin+
- 39. Amax+,Bmax+
- 40. Amax+,Bmax
- 41. Amax, Bmax+
- 42. Amax+,Bmax-
- 43. Amax-, Bmax+
- 44. Amax+,Bnom
- 45. Anom, Bmax+
- 46. Amin-,Bnom
- 47. Anom, Bmin-
- 48. Amax+,Bmin-
- 49. Amin-, Bmax+

Example

A program reads an integer number within the range [1,100] and determines whether it is a prime number or not. Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Test cases using BVC

- Since there is one variable, the total number of
- test cases will be 4n + 1 = 5.
- In our example, the set of minimum and maximum values is shown below:

- Min value = 1
- Min+ value = 2
- Max value = 100
- Max– value = 99
- Nominal value = 50–55

- Using these values, test cases can be designed as shown below:

Test Case ID	Integer Variable	Expected Output
1	1	Not a prime number
2	2	Prime number
3	100	Not a prime number
4	99	Not a prime number
5	53	Prime number

Test cases using robust testing

- Since there is one variable, the total number of test cases will be 6n + 1 = 7. The set of boundary values is shown below:

- Min value = 1
- Min– value = 0
- Min+ value = 2
- Max value = 100
- Max− value = 99
- Max+ value = 101
- Nominal value = 50–55

- Using these values, test cases can be designed as shown below:

Test Case ID	Integer Variable	Expected Output
1	0	Invalid input
2	1	Not a prime number
3	2	Prime number
4	100	Not a prime number
5	99	Not a prime number
6	101	Invalid input
7	53	Prime number

Test cases using worst-case testing

- Since there is one variable, the total number of test cases will be $5^n = 5$.

- Therefore, the number of test cases will be same as BVC.

Example

- A program computes a^b where a lies in the range [1,10] and b within [1,5].

- Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Test cases using BVC

- Since there are two variables, a and b, the total number of test cases will be 4n + 1 = 9. The set of boundary values is shown below:

	a	b
Min value	1	1
Min+ value	2	2
Max value	10	5
Max– value	9	4
Nominal value	5	3

Using these values, test cases can be designed as shown below:

Test Case ID	a	b	Expected Output
1	1	3	1
2	2	3	8
3	10	3	1000
4	9	3	729
5	5	1	5
6	5	2	25
7	5	4	625
8	5	5	3125
9	5	3	125

Test cases using robust testing

- Since there are two variables, a and b, the total number of test cases will be 6n + 1 = 13.
- The set of boundary values is shown below:

	a	b
Min value	1	1
Min– value	0	0
Min+ value	2	2
Max value	10	5
Max+ value	11	6
Max– value	9	4
Nominal value	5	3

Using these values, test cases can be designed as shown below:

Test Case ID	a	b	Expected output
1	0	3	Invalid input
2	1	3	1
3	2	3	8
4	10	3	1000
5	11	3	Invalid input
6	9	3	729
7	5	0	Invalid input
8	5	1	5
9	5	2	25
10	5	4	625
11	5	5	3125
12	5	6	Invalid input
13	5	3	125

Test cases using worst-case testing

- Since there are two variables, a and b, the total number of test cases will be $5^n = 25$.
- The set of boundary values is shown below:

	a	b
Min value	1	1
Min+ value	2	2
Max value	10	5
Max– value	9	4
Nominal value	5	3

There may be more than one variable at extreme values in this case. Therefore, test cases can be designed as shown below :

Test Case ID	a	b	Expected Output
1	1	1	1
2	1	2	1
3	1	3	3
4	1	4	1
5	1	5	1
6	2	1	2
7	2	2	4
8	2	3	8
9	2	4	16
10	2	5	32
11	5	1	5
12	5	2	25
13	5	3	125
14	5	4	625
15	5	5	3125
16	9	1	9
17	9	2	81
18	9	3	729
19	9	4	6561
20	9	5	59049
21	10	1	10
22	10	2	100
23	10	3	1000
24	10	4	10000
25	10	5	100000

Summary

- We discussed black-box test case design using:
  - boundary value analysis
- Explained BVA with some examples.

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 4), Second Edition, Oxford University Press, 2016.

Thank You

Examples on Equivalence Class Partition-Based Testing

Durga Prasad Mohapatra
NIT Rourkela

Example-1: Intersection of two straight lines

- Q. Design B-B test cases for the following program:

The program checks if 2 straight lines intersect and print their point of intersection.

Ans: The straight line may be represented as: $y = mx + c$

Straight lines are given in the form of (m_1, c_1) and (m_2, c_2).

Equivalent classes are as follows:

- ✓ Case 1: The lines are parallel i.e. $m_1 = m_2$

  So points are (1, 2) and (1, 5)

- ✓ Case 2: Coincident lines i.e. $m_1 = m_2$ and $c_1 = c_2$

  And points are (2, 3) and (2, 3)

Example-1 Contd.

- ✓ Case 3: Lines intersecting at one point i.e. $m_1 \neq m_2$

The points may be (2, 5) and (3, 6).

So there are 3 valid equivalent classes. You may include one more valid equivalent classes

There are no boundary values here.

So, the resultant test suite is:

{{(1, 2),(1, 5)}, {(2, 3),(2, 3)} {(2, 5),(3, 6)}.

Example-2: Solving quadratic equations

- Q. Design B-B test cases for the following program:
  The program solves quadratic equations of the form

$$ax^2 + bx + c$$

  It accepts 3 floating point values as input and gives the roots, e.g. The input may be (7.7, 3.3 and 4.5).

Ans: Equivalent Classes are as follows:

I. $b^2 = 4ac$ inputs are: a= 2.0, b=4.0 and c=2.0
II. $b^2 > 4ac$ inputs are a=2.0, b= 5.0 and c=2.0
III. $b^2 < 4ac$ inputs are a=2.0, b= 3.0 and c=2.0
IV. Invalid Equation inputs are a=0, b= 0 and c=10.0

Write down the test cases accordingly.

Example-3: Solving linear equations

- Q. Design B-B test cases for the following program:

The program solves linear equations in upto 10 independent variables

e.g. 5x+6y+z=5

10x+2y+5z=20

……

……

Example-3 Contd.

- Ans: Equivalent Classes are as follows:

I. Valid Equivalent Classes

   a. Many Solution (# var < #eqns)
   b. No Solution (# var > #eqns)
   c. Unique Solution (# var = #eqns)

II. Invalid Equivalent Classes

   a. Too many Variables (# var > 10)
   b. Invalid Equation (# var = 0)

Write down the test cases accordingly.

Example-4: Intersection of two circles

- Q. Design B-B test cases for the following program:

The program finds the points of intersection of 2 circles

- Ans: Equivalent Classes are as follows:

I. $r_1+r_2<$ distance between(x_1, y_1) and (x_2, y_2) i.e. not intersecting

II. $r_1+r_2=$ distance i.e. touching at 1 point

III. $r_1+r_2>$ distance i.e. intersecting at 2 points

IV. Distance=0 and $r_1= r_2$ i.e. overlapping

V. Distance=0 and $r_1\neq r_2$

VI. Invalid circles

Write down the test cases accordingly.

Example-5: Query Book Option in LIS

Q. Design B-B test cases for the option Query Book using a Keyword (e.g. Author name or title).

Ans: The equivalent classes and the corresponding test cases are as follows:

- Not present in catalogue (SE, not present)
- Present in catalogue (SE, present, 15 issued, not available)
- Present in catalogue (SE, present, 10 issued, 5 available)

References

1. Fundamentals of Software Engineering, Rajib Mall, Fifth Edition, PHI, 2018.

Thank You

STATE TABLE AND DECISION TABLE BASED TESTING

Durga Prasad Mohapatra
Professor
Dept. of CSE
N.I.T. Rourkela

STATE TABLE – BASED TESTING

- Tables are useful tools for representing and documenting many types of information relating to test case design.

- These are beneficial for applications which can be described using state transition diagrams and state tables.

Basic terms related to State Table

1. Finite State Machine (FSM)

- An FSM is a behavioral model whose outcome depends upon both the previous and current inputs.
- This model can be prepared for software structure or software behavior.
- It can be used as a tool for functional testing.

2. State Transition Diagrams or State Graph

- A system or its components may have a number of states depending on its *input and time*.
- States are represented by *nodes.*
- *With the help of nodes and transition links between nodes, a STD or SG can be prepared.*

State Graph

- A state graph is the pictorial representation of an FSM.
- Its purpose is to depict the states that a system or its components can assume.
- It shows the events or circumstances that cause or result from a change from one state to another.

State Graph cont …

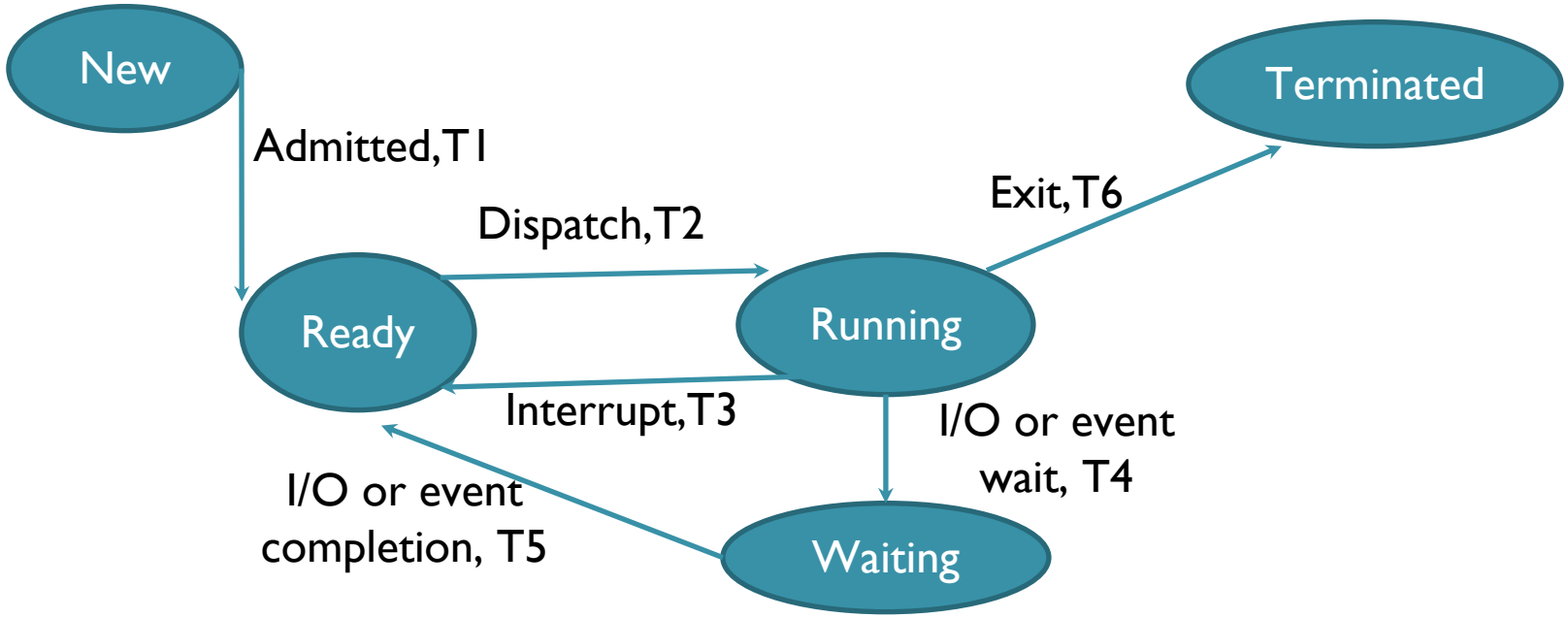
- Whatever is being modeled is subjected to inputs.
- As a result of these inputs, when one state is changed to another is called a *transition.*
- Transitions are represented by links that join the nodes.

Example

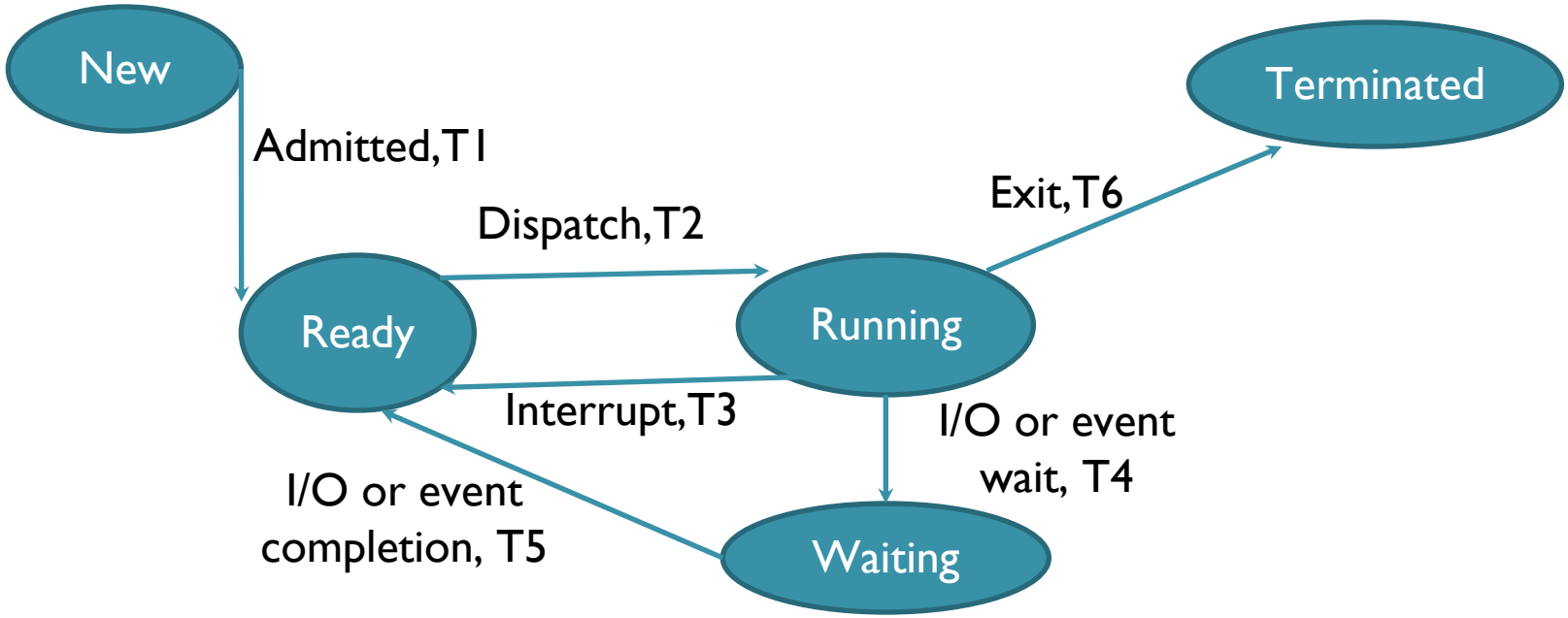
For example, a task in an O. S. can have its following states:

i. **New State :** When a task is newly created
ii. **Ready :** When the task is waiting in the ready queue for its turn.
iii. **Running :** When Instructions of the task are being executed by CPU.
iv. **Waiting :** When the task is waiting for an I/O event or reception of a signal
v. **Terminated :** The task has finished execution.

State Graph



- New State : When a task is newly created
- Ready : When the task is waiting in the ready queue for its turn.
- Running : When Instructions of the task are being executed by CPU.
- Waiting : When the task is waiting for an I/O event or reception of a signal
- Terminated : The task has finished execution.



Each arrow link provides two types of Information :

1. **Transition events like admitted, dispatch, interrupt, etc.**
2. **The resulting output from a state like T1, T2, T3 etc.**

T0=Task is in new state and waiting for admission to ready queue.
T1= A new task admitted to ready queue
T2= A ready task has started running
T3= Running task has been interrupted
T4= Running task is waiting for I/O or event
T5= Wait period of waiting task is over
T6= Task has completed execution

3. State Table

State/input Event	Admit	Dispatch	Interrupt	I/O or event Wait	I/O or event Wait Over	Exit
New	**Ready/T1**	New / T0	New / T0	New / T0	New / T0	New / T0
Ready	Ready / T1	**Running /T2**	Ready / T1	Ready / T1	Ready / T1	Ready / T1
Running	Running /T2	Running /T2	**Ready / T3**	**Waiting/T4**	Running/ T2	**Terminated/ T6**
Waiting	Waiting/T4	Waiting/T4	Waiting/ T4	Waiting/T4	**Ready /T5**	Waiting/T4

- Each rows of the table corresponds to a state.
- Each column corresponds to an input condition
- The box at the intersection of a row and a column specifies the next state (transition) and the outputs, if any.

4. State Table-Based Testing

- After reviewing the basics, we can start functional testing with state tables.

Steps:

1. Identify the states

The number of states in a state graph is the number of states we choose to recognize or model.

Finding the number of states

- First, identify all the component factors of the state.
- Identify all the allowable values for each factor
- The number of states is the product of the number of allowable values of all the factors.

Steps cont …

2. Prepare state transition diagram after understanding transitions between states

- After having all the states, identify the inputs on each state and transitions between states and prepare the state graph.
- Every input state combination must have a specified transition.

Steps cont …

3. **Convert the state graph into the state table as discussed earlier**

4. **Analyze the state table for its completeness.**

Steps cont …

5. Develop the test cases from the state table, in a tabular form, with the following entries.

- **Test cases ID :** a unique identifier for each test case
- **Test Source :** a trace back to the corresponding cell in the state table.
- **Current state :** the initial condition to run the test
- **Event :** the input triggered by the user
- **Output :** the current value returned
- **Next State :** the new state achieved

Test Cases for the O.S. Example

Test case ID	Test Source	Input		Expected results	
		Current State	Event	Output	Next state
TC1	Cell 1	New	Admit	T1	Ready
TC2	Cell 2	New	Dispatch	T0	New
TC3	Cell 3	New	Interrupt	T0	New
TC4	Cell 4	New	I/O wait	T0	New
TC5	Cell 5	New	I/O wait over	T0	New
TC6	Cell 6	New	Exit	T0	New
TC7	Cell 7	Ready	Admit	T1	Ready
TC8	Cell 8	Ready	Dispatch	T2	Running
TC9	Cell 9	Ready	Interrupt	T1	Ready
TC10	Cell 10	Ready	I/O wait	T1	Ready
TC11	Cell 11	Ready	I/O wait over	T1	Ready
TC12	Cell 12	Ready	Exit	T1	Ready
TC13	Cell 13	Running	Admit	T2	Running
TC14	Cell 14	Running	Dispatch	T2	Running
TC15	Cell 15	Running	Interrupt	T3	Ready
TC16	Cell 16	Running	I/O wait	T4	Waiting
TC17	Cell 17	Running	I/O wait over	T2	Running
TC18	Cell 18	Running	Exit	T6	Terminated
TC19	Cell 19	Waiting	Admit	T4	Waiting
TC20	Cell 20	Waiting	Dispatch	T4	Waiting
TC21	Cell 21	Waiting	Interrupt	T4	Waiting
TC22	Cell 22	Waiting	I/O wait	T4	Waiting
TC23	Cell 23	Waiting	I/O wait over	T5	Ready
TC24	Cell 24	Waiting	Exit	T4	Waiting

Decision Table – Based Testing

- Boundary value analysis and equivalence class partitioning methods do not consider combinations of input conditions.
- There may be some critical behavior to be tested when some combinations of input conditions are considered.
- Decision table is another useful method to represent the information in a tabular method.
- Decision table has the specialty to consider complex combinations of input conditions and resulting actions.
- Decision tables obtain their power from logical expressions.
- Each operand or variable in a logical expression takes on the value, TRUE or FALSE

Formation of Decision Table

		Rule 1	Rule 2	Rule 3	Rule 4	...
Condition Stub	C1	True	True	False	I	
	C2	False	True	False	True	
	C3	True	True	True	I	
Action Stub	A1	X	X	X	X	
	A2					
	A3					

- **Condition stub** It is a list a list of input conditions for which the complex combination is made.
- **Action stub** It is a list of resulting action which will be performed if a combination of input condition is satisfied.

Condition entry

- It is a specific entry in the table corresponding to input conditions mentioned in the condition stub.
- When the condition entry takes only two values – TRUE or FALSE then it is called ***Limited Entry Decision Table.***
- When the condition entry takes several values , then it is called ***Extended Entry Decision Table.***

Action entry It is the entry in the table for the resulting action to be performed.

- List all actions that can be associated with a specific procedure.
- List all conditions during execution of the procedure.
- Associate specific sets of conditions with specific actions, eliminating impossible combinations and conditions; alternatively, develop every possible permutation of conditions.
- Define rules by indicating what action occurs for a set of conditions.

Test Case Design using Decision Table

- Interpret condition stubs as the inputs for the test case.
- Interpret action stubs as the expected output for the test case.
- Rule, which is the combination of input conditions, becomes the test case itself.
- If there are *k* rules over *n* binary conditions, there are at least *k* test cases and at the most 2^n test cases.

Example - 1

A program calculates the total salary of an employee with the conditions that if the working hours are less than or equal to 48, then give normal salary. The hours over 48 normal working days are calculated at the rate of 1.25 of the salary. However, on holidays or Sundays, the hours are calculated at the rate of 2.00 times of the salary. Design test cases using decision table testing.

Solution

		Entry		
		Rule 1	**Rule 2**	**Rule 3**
Condition Stub	C1: Working hours > 48	I	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal Salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

Decision Table

Test case ID	Working hour	Day	Expected Result
1	48	Monday	Normal Salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary

Expanding the Immaterial Cases in Decision Table

- These conditions means that the value of a particular condition in the specific rule does not make a difference whether it is TRUE or FALSE.
- Sometimes expanding the decision table to spell out don't-care conditions can reveal hidden problems.

Example - 1

Entry (Decision table)

		Rule 1	Rule 2	Rule 3
Condition Stub	C1: Working hours > 48	I	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal Salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

- The immaterial test case in rule 1 of the above table can be expanded by taking both T and F values of C1.

Entry (Expanded decision table)

		Rule 1-1	Rule 1-2	Rule 2	Rule 3
Condition Stub	C1: Working hours > 48 C2: Holidays or Sundays	F T	T T	F F	T F
Action Stub	A1: Normal Salary A2: 1.25 of salary A3: 2.00 of salary	 X	 X	X 	 X

Entry (Expanded decision table)

Test case ID	Working hour	Day	Expected Result
1	48	Monday	Normal Salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary
4	30	Sunday	2.00 of salary

Example - 2

A wholesaler has three commodities to sell and has three types of customers. Discount is given as per the following procedure:

(a) For DGS & D orders, 10% discount is given irrespective of the value of the order.

(b) For orders of more than Rs 50,000, agents get a discount of 15% and the retailer gets a discount of 10%.

(c) For orders of Rs 20,000 or more and up to Rs 50,000, agents get 12% and the retailer gets 8% discount.

(d) For orders of less than Rs 20,000, agents get 8% and the retailer gets 5% discount.

The aforementioned rules do not apply to the furniture items wherein a flat rate of 10% discount s admissible to all customers irrespective of the value of the order.

Design the test cases for this system using decision table testing.

Decision Table for Example 2

		ENTRY							
		R1	R2	R3	R4	R5	R6	R7	R8
Condition Stub	C1: DGS & D	T	F	F	F	F	F	F	F
	C2: Agent	F	T	F	T	F	T	F	I
	C3: Retailer	F	F	T	F	T	F	T	I
	C4: Order > 50,000	I	T	T	F	F	F	F	I
	C5: Order ≥ 20000 to < 50,000	I	F	F	T	T	F	F	I
	C6: Order < 20,000	I	F	F	F	F	T	T	I
	C7: Furniture	F	F	F	F	F	F	F	T
Action Stub	A1: Discount of 5%							X	
	A2: Discount of 8%					X	X		
	A3: Discount of 10%	X		X					X
	A4: Discount of 12%				X				
	A5: Discount of 15%		X						

Test cases for Example - 2

Test Case Id	Type of Customer	Product Furniture?	Order value (Rs)	Expected Result
1	DGS & D	No	51,000	10% Discount
2	Agent	No	52,000	15% Discount
3	Retailer	No	53,000	10% Discount
4	Agent	No	23,000	12% Discount
5	Retailer	No	27,000	8% Discount
6	Agent	No	15,000	8% Discount
7	Retailer	No	18,000	5% Discount
8	Agent	Yes	34,000	10% Discount

References

1. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 4), Second Edition, Oxford University Press, 2016.

Thank You

Cause-Effect Graphing

Dr. Durga Prasad Mohapatra
Professor
Department of CSE
NIT Rourkela

Cause-Effect Graphing

- Cause-effect graphing, also known as *dependency modeling*,
  - focuses on modelling dependency relationships amongst
    - program input conditions, known as *causes*, and
    - output conditions, known as *effects*.

- The relationship is expressed visually in terms of a cause-effect graph.

- The graph is a visual representation of a logical relationship amongst inputs and outputs that can be expressed as a Boolean expression.

Cause-Effect Graphing (Contd..)

- The graph allows selection of various combinations of input values as tests.

- The combinatorial explosion in the number of tests is avoided by using certain heuristics during test generation.

Cause-Effect Graphing (Contd..)

- A cause is any condition in the requirements that may effect the program output.

- An effect is the response of the program to some combination of input conditions.
  - For example, it may be
    - An error message displayed on the screen
    - A new window displayed
    - A database updated.

Cause-Effect Graphing (Contd..)

- An effect need not be an "output" visible to the user of the program.

- Instead, it could also be an internal *test point* in the program that can be probed during testing to check if some intermediate result is as expected.
  - For example, the intermediate test point could be at the entrance into a method to indicate that indeed the method has been invoked.

Cause-Effect Graphing (Contd..)

Cause-effect graph testing helps by:

- Finding missing test cases
- Eliminating redundant tests
- Covering all logical combinations
- Detecting requirement inconsistencies

Example

- Consider the requirement "Dispense food only when the DF switch is ON"
  - Cause is "DF switch is ON".
  - Effect is "Dispense food".

- This requirement implies a relationship between the "DF switch is ON" and the effect "Dispense food".

- Other requirements might require additional causes for the occurrence of the "Dispense food" effect.

Cause and Effect Graphs

- Testing would be a lot easier:
  - if we could automatically generate test cases from requirements.
- Work done at IBM:
  - Can requirements specifications be systematically used to design functional test cases?

Cause and Effect Graphs

- Examine the requirements:
  - restate them as logical relation between inputs and outputs.
  - The result is a Boolean graph representing the relationships
    - called a cause-effect graph.

Cause and Effect Graphs

- Convert the graph to a decision table:
  - each column of the decision table corresponds to a test case for functional testing.

Steps to create cause-effect graph

- Study the functional requirements.
- Mark and number all causes and effects.
- Numbered causes and effects:
  - become nodes of the graph.

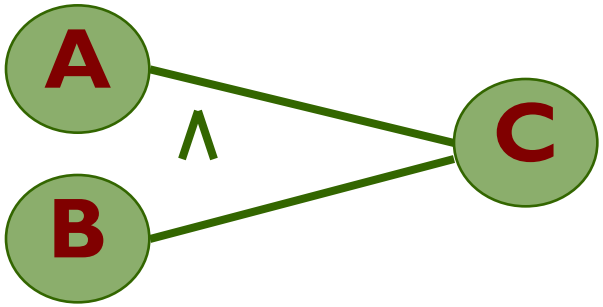
Steps to create cause-effect graph

- Draw causes on the LHS
- Draw effects on the RHS
- Draw logical relationship between causes and effects
  - as edges in the graph.
- Extra nodes can be added
  - to simplify the graph

Drawing Cause-Effect Graphs

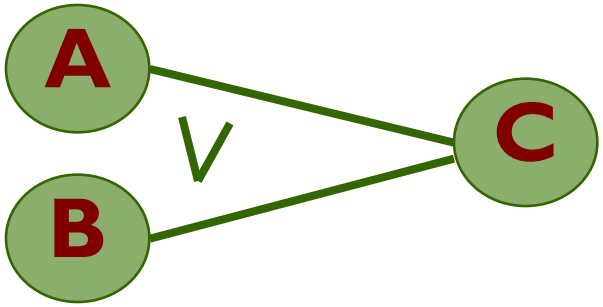


If A then B

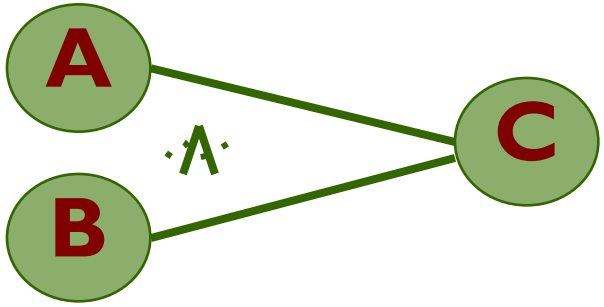


If (A and B)then C

Drawing Cause-Effect Graphs

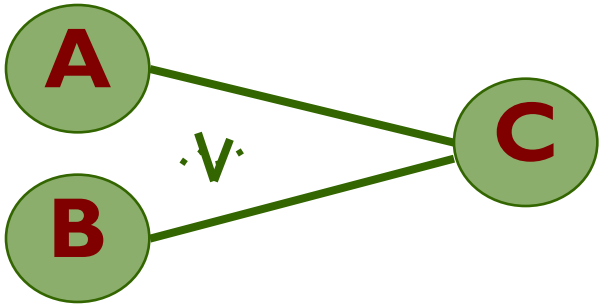


If (A or B)then C



If (not(A and B))then C

Drawing Cause-Effect Graphs



If (not (A or B))then C



If (not A) then B

Cause effect graph- Example

- A water level monitoring system
  - used by an agency involved in flood control.

  - Input: level(a,b)
    - a is the height of water in dam in meters
    - b is the rainfall in the last 24 hours in cms

Cause effect graph- Example

- Processing
  - The function calculates whether the level is safe, too high, or too low.
- Output
  - message on screen
    - level=safe
    - level=high
    - invalid syntax

Cause effect graph- Example

- We can separate the requirements into 5 causes:

1. first five letters of the command is "level"
2. command contains exactly two parameters
   - separated by comma and enclosed in parentheses

Cause effect graph- Example

- Parameters a and b are real numbers:

3

  - such that the water level is calculated to be low

4

  - or safe.

- The parameters a and b are real numbers:

5

  - such that the water level is calculated to be high.

Cause effect graph- Example

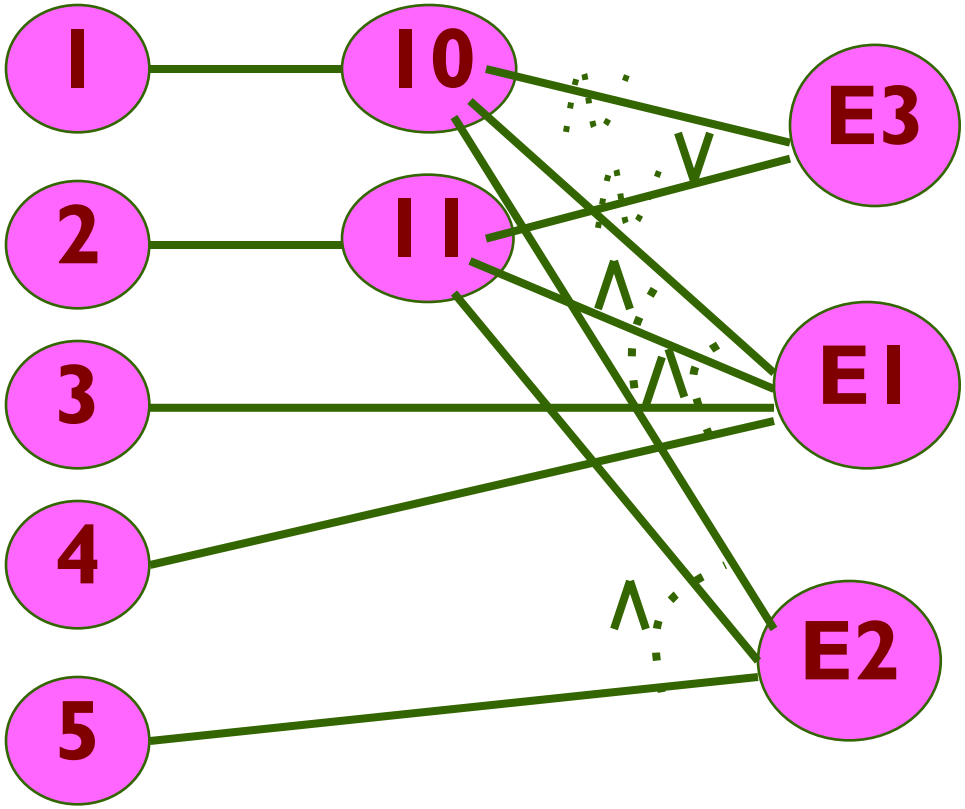
10 ◦ Command is syntactically valid

11 ◦ Operands are syntactically valid.

Cause effect graph- Example

- Three effects
  - level = safe E1
  - level = high E2
  - invalid syntax E3

Cause effect graph- Example



Cause effect graph- Decision table

	Test 1	Test 2	Test 3	Test 4	Test 5
Cause 1	I	I	I	S	I
Cause 2	I	I	I	X	S
Cause 3	I	S	S	X	X
Cause 4	S	I	S	X	X
Cause 5	S	S	I	X	X
Effect 1	P	P	A	A	A
Effect 2	A	A	P	A	A
Effect 3	A	A	A	P	P

I = Invoked

x = don't care

s = supressed

P = present

A = absent

Cause effect graph- Example

- Put a row in the decision table for each cause or effect:
  - in the example, there are five rows for causes and three for effects.

Cause effect graph- Example

- The columns of the decision table correspond to test cases.
- Define the columns by examining each effect:
  - list each combination of causes that can lead to that effect.

Cause effect graph- Example

- We can determine the number of columns of the decision table
  - by examining the lines flowing into the effect nodes of the graph.

Cause effect graph- Example

- Theoretically we could have generated $2^5=32$ test cases.
  - Using cause effect graphing technique reduces that number to 5.

Cause effect graph

- Not practical for systems which:
  - include timing aspects
  - feedback from processes is used for some other processes.

Procedure used for the generation of tests

- Identify causes and effects by reading the requirements. Each cause and effect is assigned a unique identifier. Note that an effect can also be a cause for some other effect.

- Express the relationship between causes and effects using a cause-effect graph.

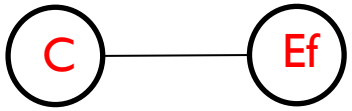
- Transform the cause-effect graph into a limited entry decision table, hereafter referred to as decision table.

- Generate tests from the decision table.

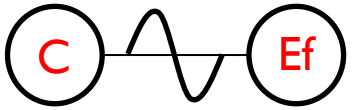
Basic elements of a cause-effect graph

- implication
- not (~)
- and (^)
- or (v)

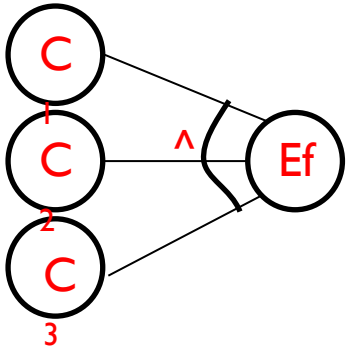
C implies Ef



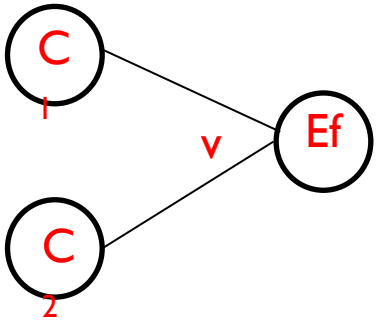
not C implies Ef



Ef when C_1 and C_2 and C_3



Ef when C_1 or C_2



- C, C_1, C_2, C_3 denote causes.
- Ef denotes an effect.

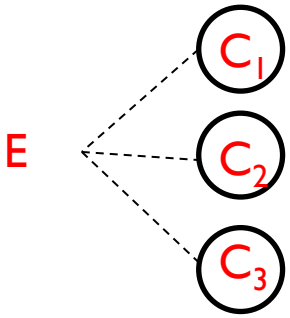
Semantics of basic elements

- *C* implies *Ef* : if(*C*) then *Ef*;
- not *C* implies *Ef* : if(¬*C*) then *Ef*;
- *Ef* when C_1 and C_2 and C_3: then *Ef*; if(C_1&&C_2&&C_3)
- *Ef* when C_1 or C_2 : if(C_1||C_2) then *Ef*;

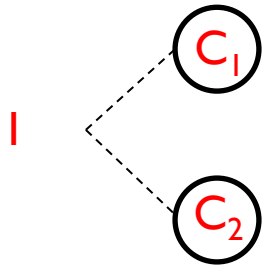
Constraints amongst causes (E,I,O,R)

- Constraints show the relationship between the causes.
- Exclusive (E)
- Inclusive (I)
- Requires (R)
- One and only one (O)

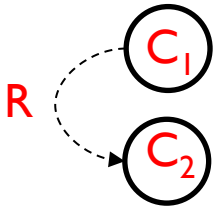
Exclusive: either C_1 or C_2 or C_3



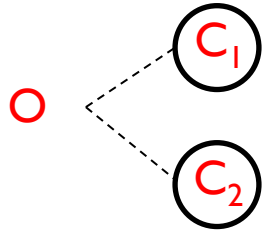
Inclusive: at least C_1 or C_2



C_1 requires C_2



One and only one, of C_1 and C_2



Constraints amongst causes (E,I,O,R)

- Exclusive (E) constraint between three causes C_1, C_2 and C_3 implies that exactly one of C_1, C_2, C_3 can be true.
- Inclusive (I) constraint between two causes C_1 and C_2 implies that at least one of the two must be present.
- Requires (R) constraint between C_1 and C_2 implies that C_1 requires C_2.
- One and only one (O) constraint models the condition that one, and only one, of C_1 and C_2 must hold.

Possible values of causes constrained by E, I, R,O

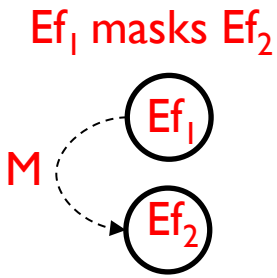
- A 0 or 1 under a cause implies that the corresponding condition is, respectively, false and true.
- The arity of all constraints, except R, is greater than or equal to 2, i.e., all except the R constraint can be applied to two or more causes; the R constraint is applied to two causes.
- A condition that is false (true) is said to be in the "0-state" (1 state).
- Similarly, an effect can be "present" (1 state) or "absent" (0 state).

Possible values of causes constrained by E, I, R,O

Constraint	Arity	Possible values		
		C1	C2	C3
$E(C_1,C_2,C_3)$	$n \geq 2$	0	0	0
		I	0	0
		0	I	0
		0	0	I
$I(C_1,C_2)$	$n \geq 2$	I	0	-
		0	I	-
		I	I	-
$R(C_1,C_2)$	$n=2$	I	I	-
		0	0	-
		0	I	-
$O(C_1,C_2,C_3)$	$n \geq 2$	I	0	0
		0	I	0
		0	0	I

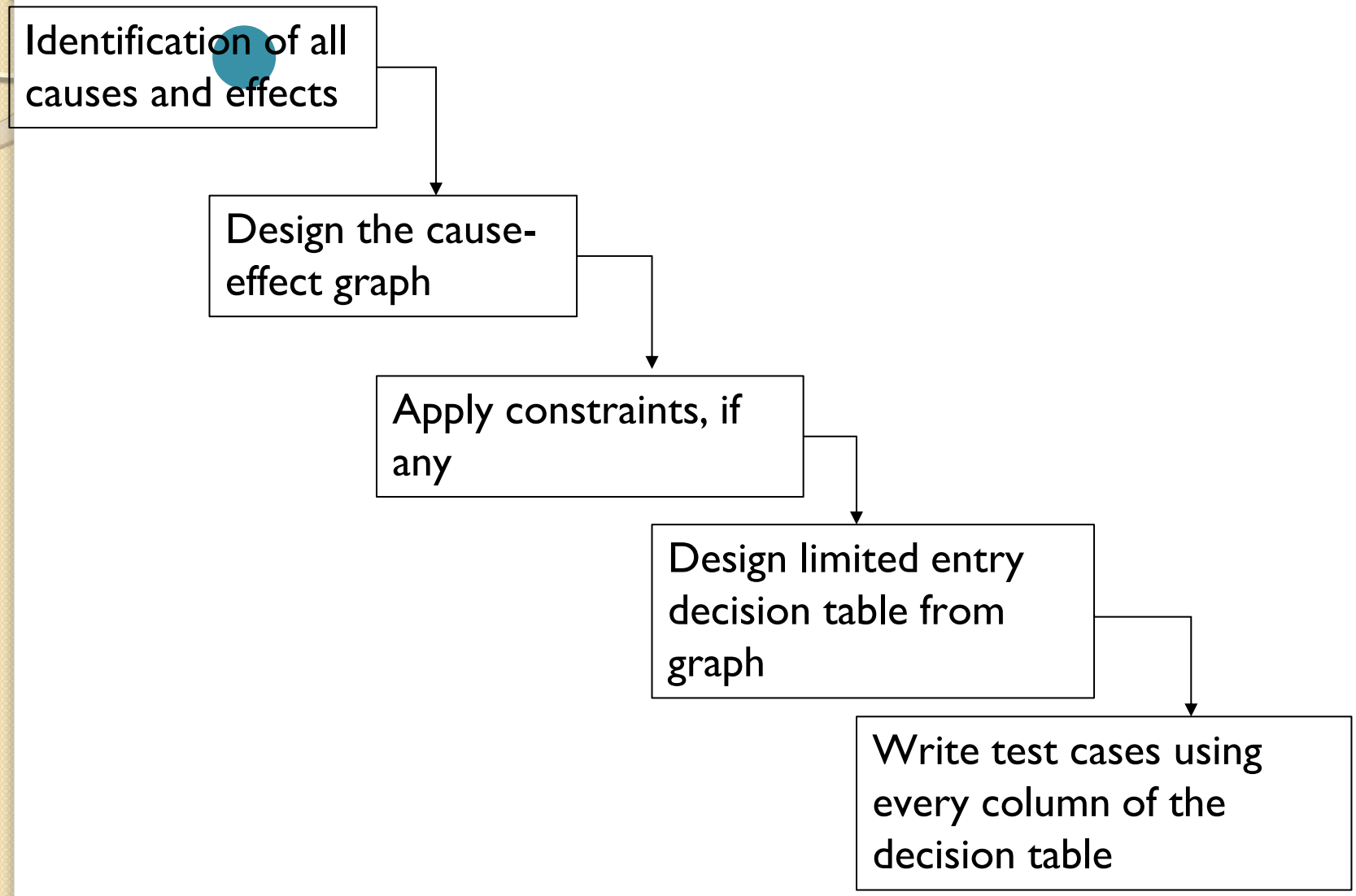
Constraint amongst effects

- Masking (M)



- Masking (M) constraint between two effects Ef_1 and Ef_2 implies that if Ef_1 is present, then Ef_2 is forced to be absent.

Steps for generating test cases using Cause-Effect Graph



Creating Cause-Effect Graph

- The process of creating a cause-effect graph consists of two major steps.

- The causes and effects are identified by a careful examination of the requirements.
  - This process also exposes the relationships amongst various causes and effects as well as constraints amongst the causes and effects.
  - Each cause and effect is assigned a unique identifier for ease of reference in the cause-effect graph.

Creating Cause-Effect Graph

- The cause-effect graph is constructed to
  - express the relationships extracted from the requirements.

- When the number of causes and effects is large, say over 100 causes and 45 effects,
  - it is appropriate to use an incremental approach.

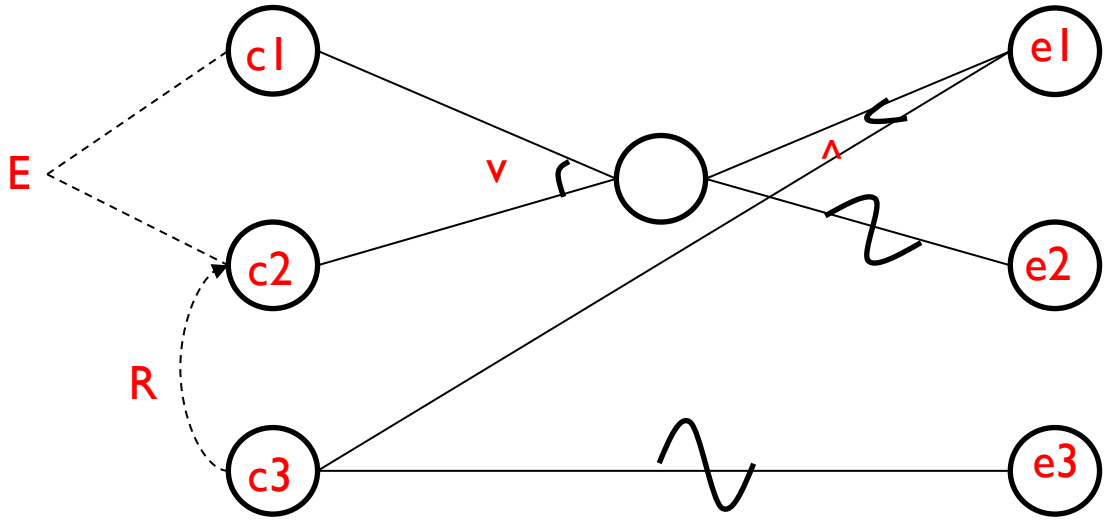
Another example

- Consider the example of keeping the record of marital status and number of children of a citizen.
- The value of marital status must be `U' or `M'.
- The value of the number of children must be digit or null in case a citizen is unmarried.
- If the information entered by the user is correct then an update is made.
- If the value of marital status of the citizen is incorrect, then the error message 1 is issued.
- Similarly, if the value of the number of children is incorrect, then the error message 2 is issued.

Answer

- Causes are
  - c1: marital status is U
  - c2: marital status is M
  - c3: number of children is a digit
- Effects are
  - e1: updation made
  - e2: error message 1 is issued
  - e3: error message 2 is issued

Answer



- There are two constraints
  - Exclusive (between c1 and c2) and
  - Requires (between c3 and c2)
- Causes c1 and c2 cannot occur simultaneously.
- For cause c3 to be true, cause c2 has to be true.

Decision Table from cause-effect graph

- Each column of the decision table represents a combination of input values, and hence a test.
- There is one row for each condition and effect.
- Thus the table decision table can be viewed as an N X M matrix with
  - N being the sum of the number of conditions and effects and
  - M the number of tests.
- Each entry in the decision table is a 0 or 1
  - depending on whether or not the corresponding condition is false or true, respectively.
- For a row corresponding to an effect, an entry is 0 or 1
  - if the effect is not present or present, respectively.

Test generation from a decision table

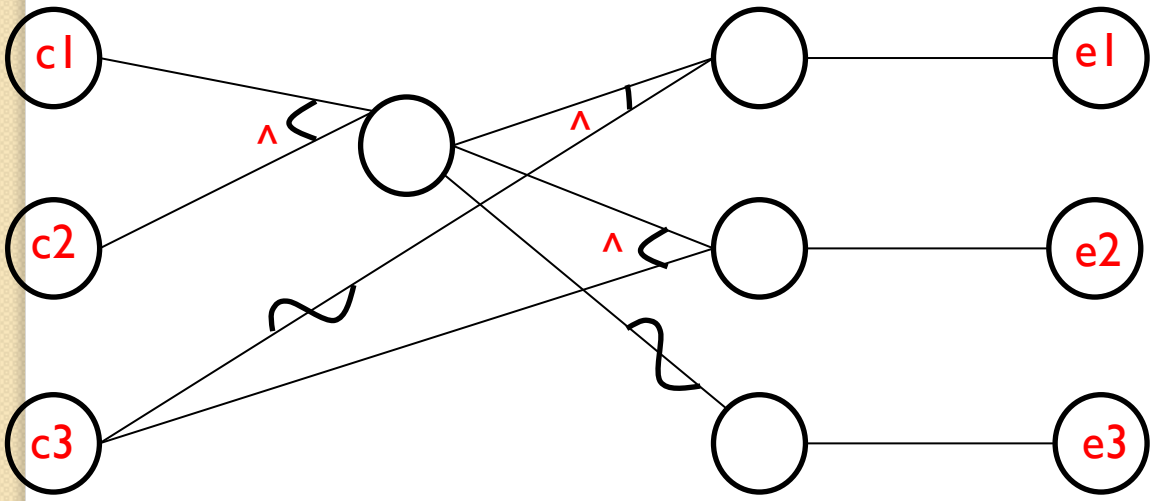
- Test generation from a decision table is relatively forward.
- Each column in the decision table generates at least one test input.
- Note that each combination might be able to generate more than one test when a condition in the cause-effect graph can be satisfied in more than one way.
- For example, consider the following cause:
- C: $x<99$
- The condition above can be satisfied by many values such as $x=1$ and $x=49$.
- Also, C can be made false by many values of x such as $x=100$ and $x=999$.
- Thus, one might have a choice of values of input variables while generating tests using columns from a decision table

Example

- A tourist of age greater than 21 years and having a clean driving record is supplied a rental car.
- A premium amount is also charged if the tourist is on business,
- Otherwise, it is not charged.
- If the tourist is less than 21 year old, or does not have a clean driving record,
  - The system will display the following message: “Car cannot be supplied”.

Answer

- Causes are
  - c1: Age is over 21
  - c2: Driving record is clean
  - c3: Tourist is on business
- Effects are
  - e1: Supply a rental car without premium charge
  - e2: Supply a rental car with premium charge
  - e3: Car cannot be supplied



Decision Table and Test Cases

	1	2	3	4
c1: Over 21?	F	T	T	T
c2: Driving record clean?	-	F	T	T
c3: On business?	-	-	F	T
e1: Supply a rental car without premium charge			X	
e2: Supply a rental car with premium charge				X
e3: Car cannot be supplied	X	X		

Test Case	Age	Driving_record_clean	On_business	Expected Output
1	20	Yes	Yes	Car cannot be supplied
2	26	No	Yes	Car cannot be supplied
3	62	Yes	No	Supply a rental car without premium charge
4	62	Yes	Yes	Supply a rental car with premium charge

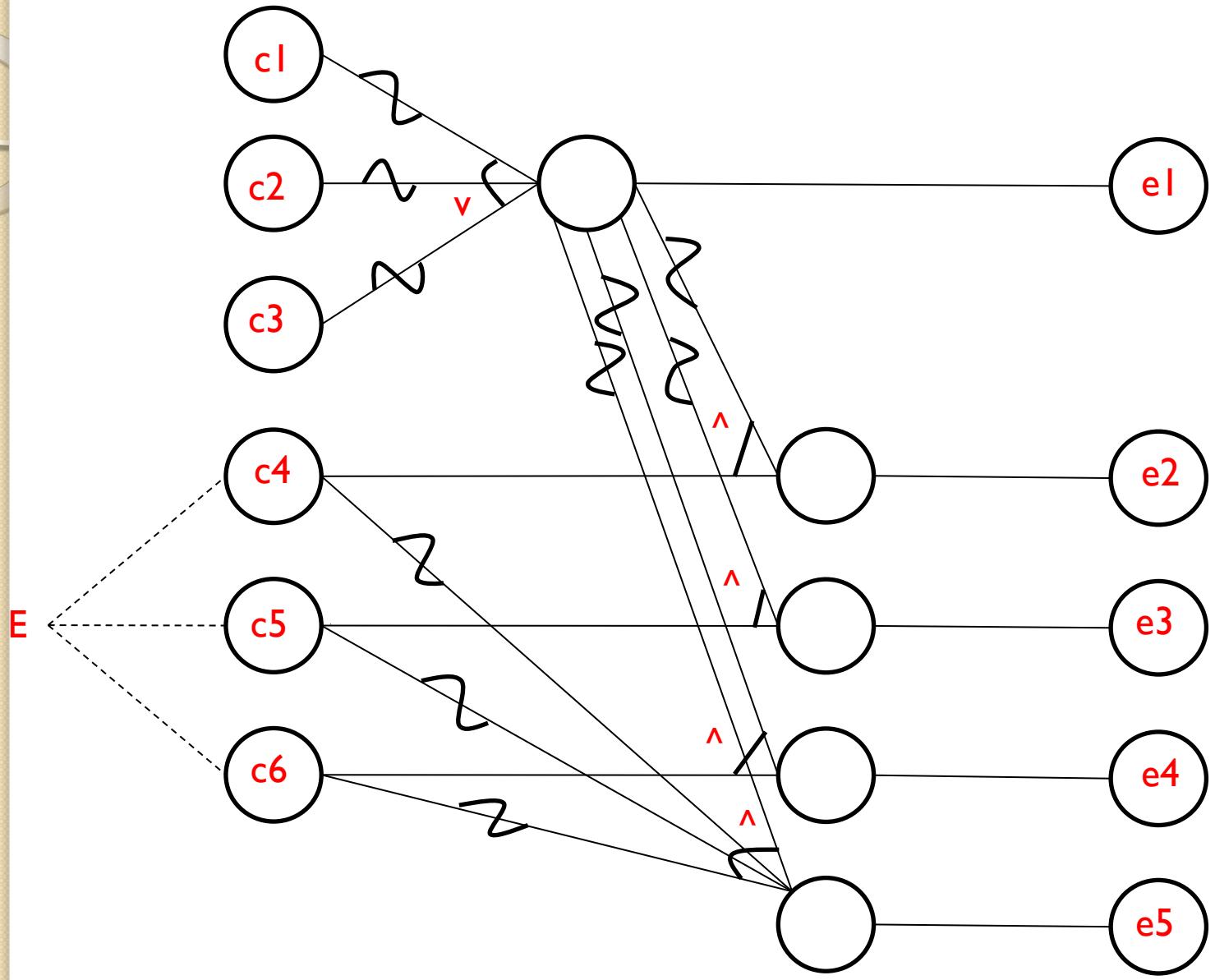
Example 2: Triangle Classification Problem

- Consider a program for classification of a triangle.
- Its input is a triple of positive integers (say a, b and c) and the input values are greater than zero and less than or equal to 100.
- The triangle is classified according to the following rules:
  - Right angled triangle: $c^2=a^2+b^2$ or $a^2=b^2+c^2$ or $b^2=c^2+ a^2$
  - Obtuse angled triangle: $c^2>a^2+b^2$ or $a^2>b^2+c^2$ or $b^2>c^2+ a^2$
  - Acute angled triangle: $c^2<a^2+b^2$ or $a^2<b^2+c^2$ or $b^2<c^2+a^2$
  - The program output may have one of the following words: [Acute angled triangle, Obtuse angled triangle, Right angled triangle, Invalid triangle, Input values are out of range]

Answer

- Causes are:
  - c1: side “a” is less than the sum of sides “b” and “c”.
  - c2: side “b” is less than the sum of sides “a” and “c”.
  - c3: side “c” is less than the sum of sides “a” and “b”.
  - c4: square of side “a” is equal to the sum of squares of sides “b” and “c”.
  - c5: square of side “a” is greater than the sum of squares of sides “b” and “c”.
  - c6: square of side “a” is less than the the sum of squares of sides “b” and “c”.
- Effects are:
  - e1: Invalid triangle
  - e2: Right angle triangle
  - e3: Obtuse angled triangle
  - e4: Acute angled triangle
  - e5: Impossible stage

Cause-Effect Graph



Decision Table

	1	2	3	4	5	6	7	8	9	10	11
c1: $a<b+c$	F	T	T	T	T	T	T	T	T	T	T
c2: $b<a+c$	-	F	T	T	T	T	T	T	T	T	T
c3: $c<a+b$	-	-	F	T	T	T	T	T	T	T	T
c4: $a^2=b^2+c^2$	-	-	-	T	T	T	T	F	F	F	F
c5: $a^2>b^2+c^2$	-	-	-	T	T	F	F	T	T	F	F
c6: $a^2<b^2+c^2$	-	-	-	T	F	T	F	T	F	T	F
e1: Invalid triangle	X	X	X								
e2: Right angle triangle							X				
e3: Obtuse angled triangle									X		
e4: Acute angled triangle										X	
e5: Impossible				X	X	X		X			X

Thank You

Orthogonal Array Testing

Dr. Durga Prasad Mohapatra
Professor
CSE Department
NIT Rourkela

Orthogonal Array Testing

- Orthogonal Arrays are two dimensional arrays of numbers that have the attractive feature that
  - by selecting any two columns in the array, an even distribution of all pairwise combinations of values in the array can be achieved.

Example

- In the following table, after selecting first two columns, we get four ordered pairs namely (0,0), (1,1), (0,1) and (1,0).
- These pairs form all the possible ordered pairs of two-element set and each ordered pair appears exactly once.

EXAMPLE

0	0	0
1	1	0
0	1	1
1	0	1

We obtain same values when selecting second and third or first and third column combination. An array exhibiting this feature is known as orthogonal array.

Orthogonal Array Testing

- Orthogonal arrays can be used in software testing for pairwise interactions.
- It provides uniformly distributed coverage for all variable pairwise combinations.
- It is commonly used for integration testing like testing of object-oriented systems, where multiple subclasses can be substituted as the server for a client.

Orthogonal Array Testing

- It is a black-box testing technique.
- OATS is used when the input to the system to be tested are low,
  - but if exhaustive testing is used then it is not possible to test completely every input of the system.
- 100% OATS implies 100% pairwise testing.
- OATS can be used for testing combinations of configurable options like a webpage that allows the other user to select:

Configurable options - Example

- Font style;
- Font color;
- Back ground Color;
- Page layout;
- Etc.

Steps to use OATS

- Step 1: Identify the independent variables that are to be used for interaction. These will be mapped as "factor" (f) of array.
- Step 2: Identify the maximum number of values, which each variable will take. These will be mapped as "levels" (p) of the array.
- Step 3: Search for an orthogonal array that has all factors from Step 1 and all levels from Step 2.

Steps to use OATS

- Step 4: Map all the factors and levels with your requirements.
- Translate them into suitable test cases.
- Look out for any special test cases.
- If we have 3 variables (parameters) that have 3 values, then the possible number of test cases using conventional technique is 3*3*3=27. But, if OATS is used, then the number of test cases will be 9.

Example 1

- Consider a scenario in which we need to derive test cases for a web page of a research paper that has four different sections:

(a) Abstract

(b) Related work

(c) Proposed work

(d) Conclusion

Example 1

- The four sections can be individually ***shown*** or ***hidden*** to the user or show ***error message***. Thus it is required to design the test condition to test interaction between different sections.

Solution

- Step 1: Number of independent variables or factors = 4.
- Step 2: Value that each independent variable can take = 3 values (shown, hidden or error message).
- Step 3: Orthogonal array would be 3^2 .
- Step 4: The appropriate orthogonal array for 4 factors and 3 levels is shown in the table.

Solution

Experiment No.	Factor A	Factor B	Factor C	Factor D
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

Solution

- Step 5: Now, map the table with our requirements.
- 1 will represent “Shown” value.
- 2 will represent “Hidden” value.
- 3 will represent “Error Message” value.
- Factor A will represent “Abstract” section.
- Factor B will represent “Related Work” section.
- Factor C will represent “Proposed Work” section.

Solution

- Factor D will represent “Conclusion” section.
- Experiment will represent “Test Case #”.
- Step 6: After mapping, the table will look like:

Solution

Test Case	Abstract	Related Work	Proposed work	Conclusion
Test Case 1	Shown	Shown	Shown	Shown
Test Case 2	Shown	Hidden	Hidden	Hidden
Test Case 3	Shown	Error Message	Error Message	Error Message
Test Case 4	Hidden	Shown	Hidden	Error Message
Test Case 5	Hidden	Hidden	Error Message	Shown
Test Case 6	Hidden	Error Message	Shown	Hidden
Test Case 7	Error Message	Shown	Error Message	Hidden
Test Case 8	Error Message	Hidden	Shown	Error Message
Test Case 9	Error Message	Error Message	Hidden	Shown

Positive Testing

- Positive testing tries to prove that a given product does what it is supposed to do.
- Positive Testing is testing process where the system is validated against the valid input data.
- In this testing tester always check for only valid set of values and check if an application behaves as expected with its expected inputs.

Positive Testing cont …

- The main intention of this testing is to check whether software application not showing error when not supposed to and showing error when supposed to.
- Such testing is to be carried out keeping positive point of view and only execute the positive scenario.
- When a test case verifies the requirements of the product with a set of expected output, it is called *positive test case*.

Positive Testing cont …

- The purpose of positive testing is to prove that the product works as per specification and expectations.
- A product delivering an error when it is expected to give an error, is also a part of positive testing.
- Positive testing can thus be said to check the product's behaviour for positive and negative conditions as stated in the requirement.

Example of Positive testing

- Consider a scenario where you want to test an application which contains a simple textbox to enter age and requirements say that it should take only integer values.
- So here provide only positive integer values to check whether it is working as expected or not is the Positive Testing.
- Most of the applications developers implement Positive scenarios where testers get less defects count around positive testing.



Another example

Let us take a lock and key. We do not know how the levers in the lock work, but we only know the set of inputs (the number of keys, specific sequence of using the keys and the direction of turn of each key) and the expected outcome (locking and unlocking). For example, if a key is turned clockwise it should unlock and if turned anticlockwise it should lock.

To use the lock one need not understand how they work. However, it is essential to know the external functionality of the lock and key system. Some of the functionality that you need to know to use the lock are given as follows.

Some functionalities to use the lock

Functionality	What you need to know to use
Features of a lock	It is made of metal, has a hole provision to lock, has a facility to insert the key and the keyhole ability to turn clockwise or anticlockwise.
Features of key	It is made of metal and created to fit into a particular lock's keyhole.
Actions performed	Key inserted and turned clockwise to unlock. Key inserted and turned anticlockwise to lock.
States Inputs Expected outcome	Locked. Unlocked. Key turned clockwise or anticlockwise. Locking. Unlocking.

Sample requirements specification for lock and key system

Sl. No.	Requirements identifier	Description	Priority (High, med, low)
1	BR-01	Inserting the key numbered 123-456 and turning it clockwise should facilitate locking	H
2	BR-02	Inserting the key numbered 123-456 and turning it anticlockwise should facilitate unlocking	H
3	BR-03	Only key number 123-456 can be used to lock and unlock	H
4	BR-04	No other object can be used to lock	M
5	BR-05	No other object can be used to unlock	M
6	BR-06	The lock must not open even when it is hit with a heavy object	M
7	BR-07	The lock and key must be made of metal and must weigh approximately 150 grams	L
8	BR-08	Lock and unlock directions should be changeable for usability of left-handers	L

Example of positive test cases

Test Case id	Req. No.	Input 1	Input 2	Current state	Expected output
TC 1	BR-01	Key 123-456	Turn clockwise	Unlocked	Locked
TC 2	BR-01	Key 123-456	Turn clockwise	Locked	No change
TC 3	BR-02	Key 123-456	Turn anticlockwise	Unlocked	No change
TC 4	BR-02	Key 123-456	Turn anticlockwise	Locked	Unlock
TC 5	BR-04	Hairpin	Turn clockwise	Locked	No change

Example of positive test condition for positive testing

- Take the first row in the table.
- When the lock is in an unlocked state and you use key 123-456 and turn it clockwise, the expected outcome is to get it locked.
- During test execution, if the test results in locking, then the test is passed.

Example of negative test condition for positive testing

- In the fifth row of the table, the lock is in locked state.
- Using a hairpin and turning it clockwise should not cause a change in state or cause any damage to the lock.
- On test execution, if there are no changes, then this positive test case is passed.

Negative Testing

- Negative testing is done to show that the product does not fail when an unexpected input is given.

- The purpose of negative testing is to try and break the system.

- Negative testing covers scenarios for which the product is not designed and coded.

Negative Testing cont …

- In other words, the input values may not have been represented in the specification of the product.
- These test conditions can be termed as unknown conditions for the product as far as the specifications are concerned.
- But, at the end-user level, there are multiple scenarios that are encountered and that need to be taken care of by the product.

Negative Testing cont …

- It becomes even more important for the tester to know the negative situations that may occur at the end-user level so that the application can be tested and made fool proof.
- A negative test would be a product *not delivering an error when it should* or *delivering an error when it should not*.

Negative Testing cont …

- Negative Testing is testing process where the system is validated against the invalid input data.
- A negative test checks if an application behaves as expected with its negative inputs.
- The main intention of this testing is to check whether software application not showing error when supposed to and showing error when not supposed to.

Negative Testing cont …

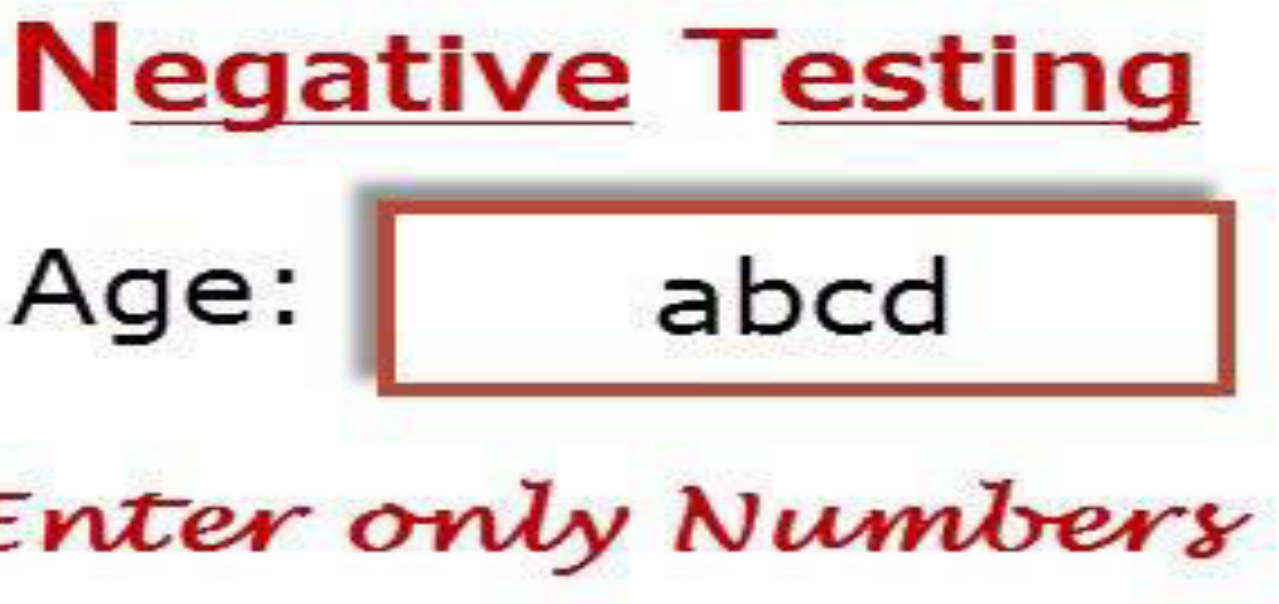
- Such testing is to be carried out keeping negative point of view and only execute the test cases for only invalid set of input data.
- Negative testing is a testing process to identify the inputs where system is not designed or un-handled inputs by providing different invalid inputs.

Negative Testing cont …

- The main reason behind Negative testing is to check the stability of the software application against the influences of different variety of incorrect validation data set.
- The Negative testing helps to improve the testing coverage of your software application under test.
- Both positive and negative testing approaches are equally important for making your application more reliable and stable.

Example of Negative Testing

- Consider a textbox example which should accept only integer values.
- So here provide the characters like “abcd” in the age textbox and check the behavior of application, either it should show a validation error message for all invalid inputs (for all other than integer values) or system should not allow to enter a non integer values.



Example of negative test cases

Test Case id	Input 1	Input 2	Current state	Expected output
1	Some other lock's key	Turn clockwise	Lock	Lock
2	Some other lock's key	Turn anticlockwise	Unlock	Unlock
3	Thin piece of wire	Turn anticlockwise	Unlock	Unlock
4	Hit with a stone		Lock	Lock

- In the above table, there are no requirement numbers, this is because negative testing focuses on test conditions that lie outside the specification.
- Since all the test conditions are outside the specification, they cannot be categorized as positive and negative test conditions.
- Some people consider all of them as negative test

Positive and negative testing scenarios

- If the requirement is saying that password text field should accepts 6 – 20 characters and only alphanumeric characters.

- **Positive Test Scenarios**
  - Password textbox should accept 6 characters
  - Password textbox should up to 20 characters
  - Password textbox should accepts any value in between 6-20 chars length.
  - Password textbox should accepts all numeric and alphabets values.
- **Negative Test scenarios**
  - Password textbox should not accept less than 6 characters
  - Password textbox should not exceeds more than 20 characters
  - Password textbox should not accept special characters

Positive and Negative Test Scenarios with Example

- Suppose, you are doing the testing on a login form which has following fields like Username, Password, and Sign In, Sign Up, Cancel, Login Button etc.

- Now positive scenario of login form is that you enter the valid username and password in the username and password field, and then click on Login Button to check whether the user is able to login or not.

- Negative scenario of login form is that you leave the password field blank and fill the username field, and then click on Login Button to check whether the user is able to login or not.

Consideration for both the testing

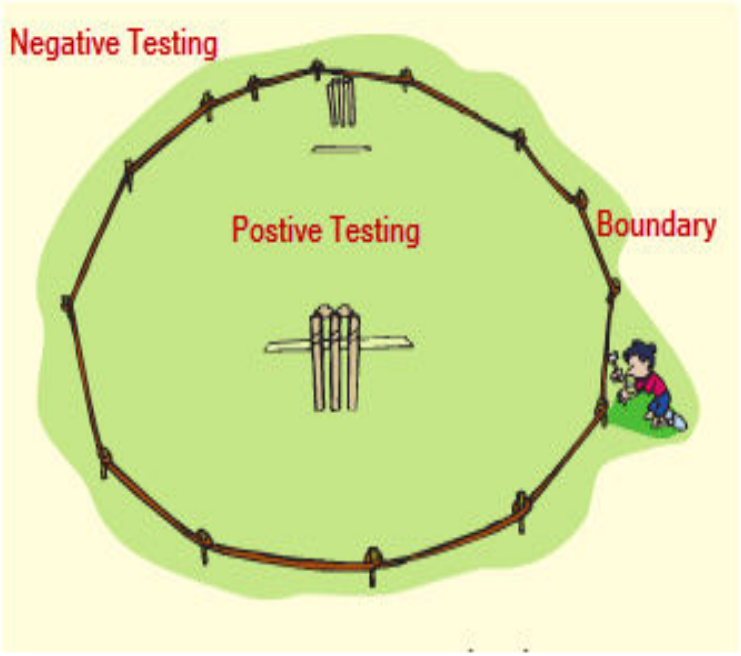
- In both the testing, following needs to be considered:
  - Input data
  - Action which needs to be performed
  - Output Result

Testing Technique used for Positive and Negative Testing

- Following techniques are used for Positive and negative validation of testing:
  - Boundary Value Analysis
  - Equivalence Partitioning

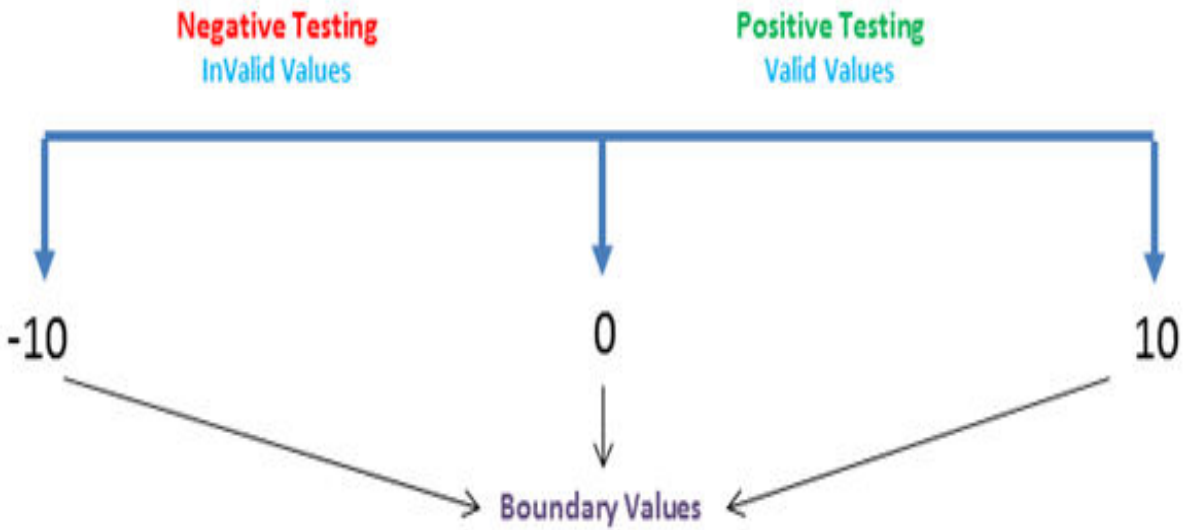
Boundary Value Analysis

- This is one of the software testing technique in which the test cases are designed to include values at the boundary.
- If the input data is used within the boundary value limits, then it is said to be Positive Testing.
- If the input data is picked outside the boundary value limits, then it is said to be Negative Testing.



Example

- A system can accept the numbers from 0 to 10 numeric values.
- All other numbers are invalid values.
- Under this technique , boundary values 0 , 10 and -10 will be tested.

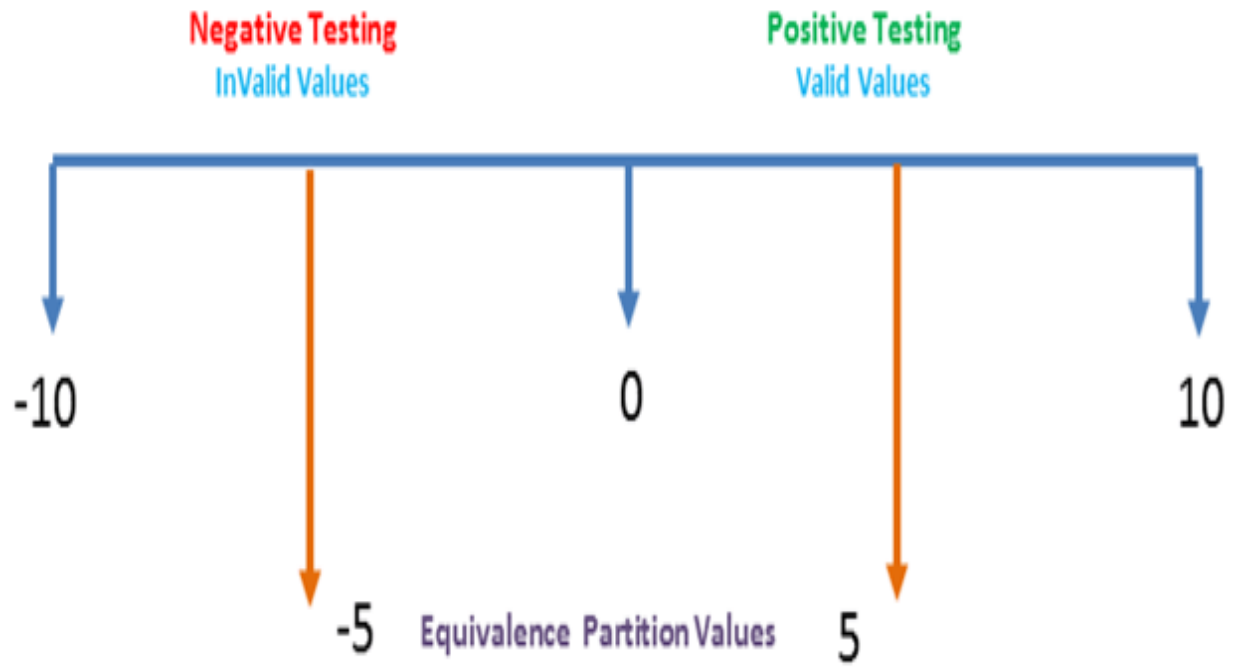


Equivalence Partitioning

- This is a software testing technique which divides the input data into many partitions.
- Values from each partition must be tested at least once.
- Partitions with valid values are used for Positive Testing.
- While partitions with invalid values are used for negative testing.

Example

- Numeric values Zero to ten can be divided to two(or three)partition.
- In our case we have two partitions -10 to -1 and 0 to 10.
- Sample values (5 and -5) can be taken from each part to tes1



Difference between positive and negative testing

- For positive testing if all documented requirements and test conditions are covered, then coverage can be considered to be 100 percent.
- If the specifications are very clear, then coverage can be achieved.
- In contrast, there is no end to negative testing and 100 percent coverage in negative testing is impractical.
- Negative testing requires a high degree of creativity among the testers to cover as many "unknowns" as possible to avoid failure at a customer site.



Top Distinctions between Positive & Negative Testing

Positive Testing (Valid)	Negative Testing (Invalid)
1. Positive Testing means testing the application or system by giving valid data.	1. Negative Testing means testing the application or system by giving invalid data.
2. In this testing tester always check for only valid set of values.	2. In this testing tester always check for only invalid set of values.
3. Positive Testing is done by keeping positive point of view for example checking the mobile number field by giving numbers only like 9999999999.	3. Negative Testing is done by keeping negative point of view for example checking the mobile number field by giving numbers and alphabets like 99999abcde.
4. It is always done to verify the known set of **Test Conditions**.	4. It is always done to break the project and product with unknown set of Test Conditions.
5. This Testing checks how the product and project behave by providing valid set of data.	5. This Testing covers those scenarios for which the product is not designed and coded by providing invalid set of data.
6. Main aim means purpose of this Testing is to prove that the project and product works as per the requirements and specifications.	6. Main aim means purpose of this Testing is try to break the application or system by providing **invalid set of data**.
7. This type of Testing always tries to prove that a given product and project always meets the requirements and specifications of a client and customer.	7. Negative Testing is that in which tester attempts to prove that the given product and project does, which is not said in the client and customer requirements.

ERROR GUESSING

- Error guessing is the preferred method used when all other methods fail.
- Sometimes error guessing is used to test some special cases.
- Error guessing is a very practical case wherein the tester uses his intuition, experience, knowledge of project, bug history and makes a guess about where the bug can be.
- Basic idea is to make a list of possible errors in error prone situations and then develop the test cases.
- No general procedure for this tech exists. It is an adhoc process.

Example - Special cases in the roots of a quadratic equation

- What will happen when a=0 in the quadratic equation?
  - Though, we do consider this case, there are chances that we may overlook it while testing, as it has two cases:
- i) If a=0 then the equation is no longer quadratic.
- ii) For calculation of roots, the division is by zero. So, we may overlook.

- What will happen when all the inputs are negative?
- What will happen when the inputs list is empty?

Summary

- Discussed Orthogonal Array Testing.
- Explained Positive and Negative Testing with suitable examples.
- Presented Error Guessing.

Exercise

Design a black-box test suite for a University Admission System that accepts age 17–25, percentage 50–100, and entrance score 40–100. Admission is confirmed only when the applicant is eligible, documents are complete, and payment is successful.

Apply the following black-box testing techniques:

1. Equivalence Class Partitioning
2. Boundary Value Analysis
3. Decision Table Testing
4. State-Based Testing
5. Cause-Effect Graph Testing
6. Orthogonal Array Testing
7. Positive Testing
8. Negative Testing
9. Error Guessing
10. Pairwise Testing

For each technique, prepare the test conditions, test cases, input data, and expected output.

References

1. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 4), Second Edition, Oxford University Press, 2016.

Thank you

Mutation Testing – A White Box Testing Technique

Dr. Durga Prasad Mohapatra
Dept. of Computer Science & Engineering
NIT Rourkela

Introduction

- Mutation testing is the process of mutating some segment of code (putting some error in the code) and then, testing this mutated code with some data.
- If the test data is able to detect the mutations in the code, then the test data is quite good, otherwise we must focus on the quality of the test data.

Introduction cont...

- Mutation testing helps a user create test data by interacting with user to iteratively strengthen quality of test data.
- During mutation testing, faults are introduced into a program.
- Test data are used to execute these faulty programs with the goal of causing each faulty program to fail.
- Faulty programs are called mutants of the original program.

Introduction cont...

- A mutant is said to be killed when a test case causes it to fail.
- When this happens, the mutant is considered dead and no longer needs to remain in the testing process, since the faults represented by that mutant have been detected.

Introduction cont...

- The software is first tested:
  - using an initial testing method based on white-box strategies.

- After the initial testing is complete,
  - mutation testing is taken up.

Main Idea

- The idea behind mutation testing:
  - make a few arbitrary small changes to a program at a time.

Main Idea cont …

- Insert faults into a program:
  - Check whether the test suite is able to detect these.
  - This either validates or invalidates the test suite.

Main Idea cont …

- Each time the program is changed,
  - it is called a mutated program
  - the change is called a mutant.

Main Idea cont …

- A mutated program:
  - Tested against the full test suite of the program.
- If there exists at least one test case in the test suite for which:
  - A mutant gives an incorrect result,
  - Then the mutant is said to be dead.

Main Idea cont …

- If a mutant remains alive:
  - even after all test cases have been exhausted,
  - the test suite is enhanced to kill the mutant.
- The process of generation and killing of mutants:
  - can be automated by predefining a set of primitive changes that can be applied to the program.

Primitive changes

- The primitive changes can be:
  - Deleting a statement
  - Altering an arithmetic operator,
  - Changing the value of a constant,
  - Changing a data type, etc.

Traditional Mutation Operators

- Deletion of a statement
- Boolean:
  - Replacement of a statement with another

    eg. `==` and `>=`, `<` and `<=`
  - Replacement of boolean expressions with *true* or *false*

    eg. **`a || b`** with ***true***
- Replacement of arithmetic

  eg. `*` and `+`, `/` and `-`
- Replacement of a variable (ensuring same scope/type)

Underlying Hypotheses

- Mutation testing is based on the following two hypotheses:
  - **The Competent Programmer Hypothesis**
  - **The Coupling Effect**

Both of these were proposed by DeMillo *et al.*,1978

The Competent Programmer Hypothesis

- Programmers create programs that are close to being correct:
  - Differ from the correct program by some simple errors.

The Coupling Effect

- **Complex errors are caused due to several simple errors.**
- It therefore suffices to check for the presence of the simple errors

Types of mutants

- Primary Mutant
- Secondary Mutant

Primary Mutant

When the mutants are single modification of the initial program using some operators, they are called primary mutants.

Example

```
if(a>b)
        x=x + y;
else
        x = y;
Printf("%d", x);
……
```

We can consider following mutant for this example;

M1: x=x-y;
M2: x=x/y;
M3: x=x+1;
M4: printf("%d", y);

Example cont …

The results of the initial program and its mutant

Test Data	x	y	Initial program result	Mutant Result
TD1	2	2	4	0(M1)
TD2(x and y# 0)	4	3	7	1.4(M2)
TD3(y #1)	3	2	5	4(M3)
TD4(y #0)	5	2	7	2(M4)

Secondary Mutants

- When multiple levels of mutation are applied on the initial program, then, this class of mutant is called secondary Mutant.
- In this case, it is very difficult to identify the initial program form its mutants.

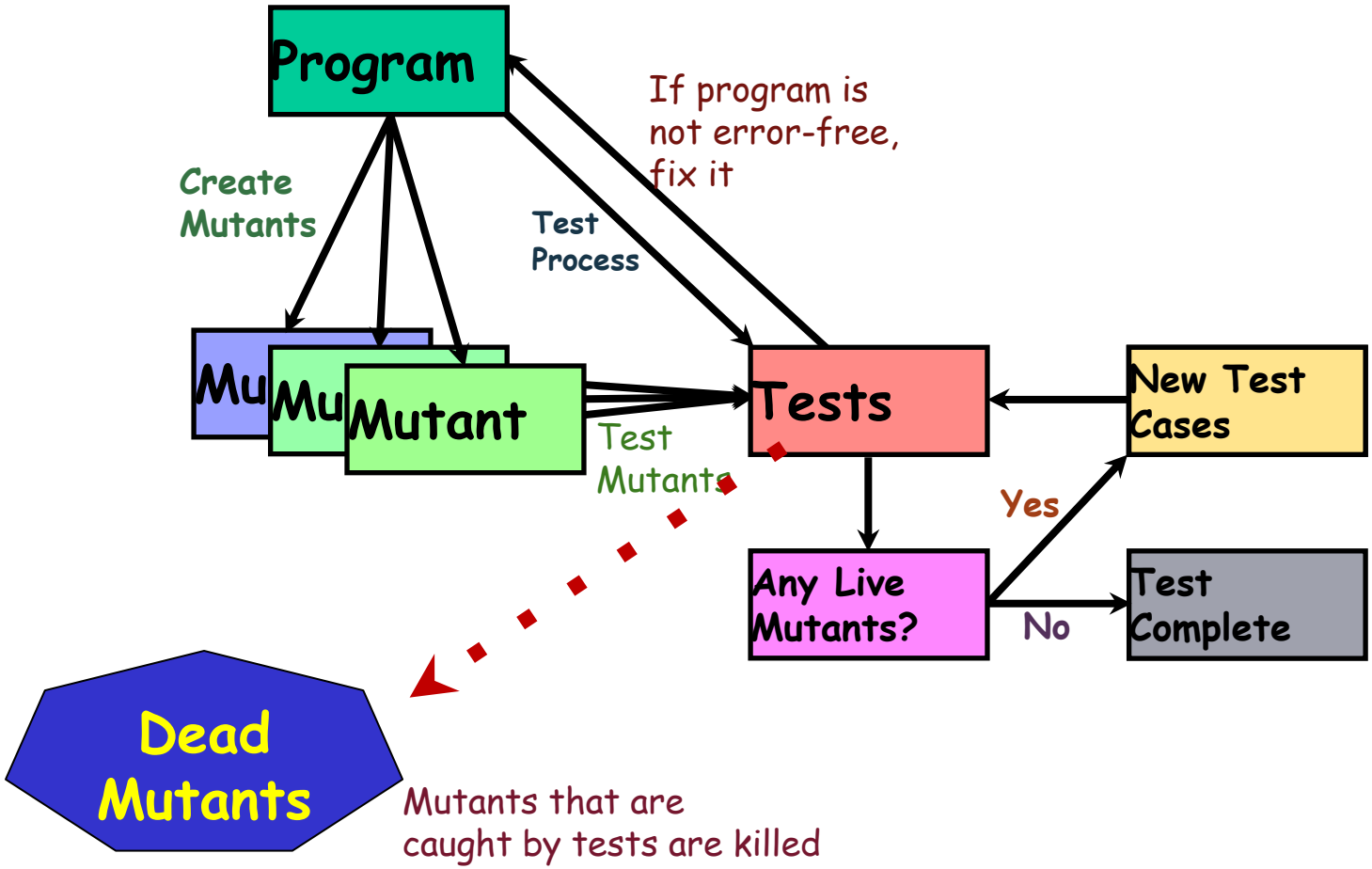
Example

```
If(a<b)
   c = a;
```

Mutants for this code may be as follows:

```
M1 : if(a<= b-1)
                c =a;
M2 : if(a+1<=b)
                c = a;
M3 : if(a == b)
          c =a+1;
```

Mutation Testing Process



Mutation Testing Process cont…

- Construct the mutants of a test program.
- Add test cases to the mutation system and check the output of the program on each test case to see if it is correct.
- If the output is incorrect, a fault has been found and the program must be modified and the process restarted.
- If the output is correct, that test case is executed against each live mutant.
- If the output of a mutant differs from that of the original program on the same test case, the mutant is assumed to be incorrect and is killed.

Mutation Testing Process cont…

- After each test case has been executed against each live mutant, each remaining mutant falls into one of the following two categories:

✓ One, the mutant is functionally equivalent to the original program. An equivalent mutant always produces the same output as the original program, so no test case can kill it.

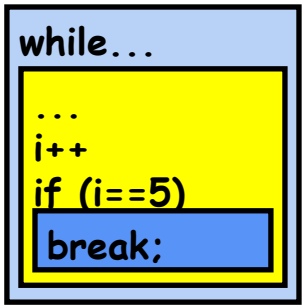
✓ Two, the mutant is killable, but the set of test cases is insufficient to kill it. In this case, new test cases need to be created, and the process iterates until the test set is strong enough to satisfy the tester.

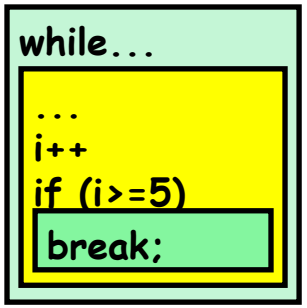
Mutation Testing Process cont…

- The mutation score for a set of test data is the percentage of non-equivalent mutants killed by that data.
- If the mutation score is 100%, then the test data is called mutation adequate.

Equivalent Mutants

- There may be surviving mutants that cannot be killed,
  - These are called Equivalent Mutants
- Although syntactically different:
  - These mutants are indistinguishable through testing.
- Therefore have to be checked 'by hand'





Disadvantages of Mutation Testing

- Equivalent mutants
- Computationally very expensive.
  - A large number of possible mutants can be generated.
- Certain types of faults are very difficult to inject.
  - Only simple syntactic faults introduced

Quiz 1

- Identify one advantage and one disadvantage of the mutation test technique.

Quiz 1: Solution

- Identify two advantages and two disadvantages of the mutation test technique.
- **Adv**:
  - Can be automated
  - Helps effectively strengthen black box and coverage-based test suite
- **Disadv:**
  - Equivalent mutants

Summary

- Presented basic concepts of mutation testing.
- Explained types of mutants.
- Discussed the mutation testing process.
- Presented some limitations of mutation testing.

29

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 5), Second Edition, Oxford University Press, 2016.

Thank You

Example on Mutation Testing

Dr. D. P. Mohapatra

Associate Professor

Department of CSE
National Institute of Technology, Rourkela-769008
Odisha



Example

Consider the following program. Generate five mutants and design test cases taking into account each mutant, using mutation testing. Calculate mutation score of your test suite.

```
 main(){
float x,y,z;
printf("Enter values of three variables x,y,z");
scanf("%f%f%f",&x,&y,&z);
if(x>y){
if(x>z)
printf("%d is greatest",x);
else
printf("%d is greatest",z);
}
else{
if(y>z)
printf("%d is greatest",y);
else
printf("%d is greatest",z);
}
}
```



Solution

The instrumented program is given below.

```
 1    main(){
      float x,y,z;
2     printf("Enter values of three variables x,y,z");
3     scanf("%f%f%f",&x,&y,&z);
4     if(x>y){
5     if(x>z)
6     printf("%d is greatest",x);
       else
7     printf("%d is greatest",z);
      }
      else{
8     if(y>z)
9     printf("%d is greatest",y);
       else
10     printf("%d is greatest",z);
       }
11    }
```



Test suite

Consider the test suite given in Table 1.

Table 1: Test suite for Q **??**

Sl. No.	x	y	z	Expected Output
1	6	10	2	10
2	10	6	2	10
3	6	2	10	10
4	6	10	20	20



Mutated Statements

Table 2: Mutated statements

Mutant No.	Line No.	Original line	Modified line
M1	4	if($x > y$)	if($x < y$)
M2	4	if($x > y$)	if($x > (y + z)$)
M3	5	if($x > z$)	if($x < z$)
M4	8	if($y > z$)	if($y = z$)
M5	7	printf("%d is greatest",z);	printf("%d is greatest",y);



Actual Output of mutant M1

The mutated line numbers and changed lines are given in Table 2. The actual output obtained by executing the mutants M1-M5 is shown in Tables 3–7.

Table 3: Actual output of mutant M1

Test case	x	y	z	Expected output	Actual output
1	6	10	2	10	6
2	10	6	2	10	6
3	6	2	10	10	10
4	6	10	20	20	20



Actual output of mutant M2

Table 4: Actual output of mutant M2

Test case	x	y	z	Expected output	Actual output
1	6	10	2	10	10
2	10	6	2	10	10
3	6	2	10	10	10
4	6	10	20	20	20



Actual output of mutant M3

Table 5: Actual output of mutant M3

Test case	x	y	z	Expected output	Actual output
1	6	10	2	10	10
2	10	6	2	10	2
3	6	2	10	10	6
4	6	10	20	20	20



Actual output of mutant M4

Table 6: Actual output of mutant M4

Test case	x	y	z	Expected output	Actual output
1	6	10	2	10	10
2	10	6	2	10	10
3	6	10	2	10	10
4	6	10	20	20	10



Actual output of mutant M5

Table 7: Actual output of mutant M5

Test case	x	y	z	Expected output	Actual output
1	6	10	2	10	10
2	10	6	2	10	10
3	6	2	10	10	2
4	6	10	20	20	20



Additional test case

Table 8: Additional test case

Test case	x	y	z	Expected Output
5	10	5	6	10



Output of added test case

Table 9: Output of added test case

Test case	x	y	z	Expected output	Actual output
5	10	5	6	10	6



Revised Test suite

Table 10: Revised Test suite

Sl. No.	x	y	z	Expected Output
1	6	10	2	10
2	10	6	2	10
3	6	2	10	10
4	6	10	20	20
5	10	5	6	10



Mutation Score

Mutation score=Number of mutants killed/Total number of mutants=4/5=0.8

Higher the mutant score, better is the effectiveness of the test suite. The mutant M2 is live in the example. We may have to write a specific test case to kill this mutant. The additional test case is given in Table 8.

Now, when we execute test case 5, the actual output will be different from the expected output (Table 9), hence the mutant will be killed. This test case is very important and should be added to the given test suite. Therefore, the revised test suite is given in Table 10.



White-box testing techniques

Dr. Durga Prasad Mohapatra
Professor, Dept. of CSE
N.I.T Rourkela

White-box Testing

- Designing white-box test cases:
  - Requires knowledge about the internal structure of software.
  - White-box testing is also called structural testing.
  - In this unit we will study white-box testing.

White-Box Testing Methodologies

- There exist several popular white-box testing methodologies:
  - Statement coverage
  - Branch coverage
  - Condition coverage
  - MC/DC coverage
  - Path coverage
  - Data flow-based testing
  - Mutation testing

Statement Coverage

- Statement coverage methodology:
  - Design test cases so that every statement in the program is executed at least once.

Statement Coverage

- The principal idea:
  - Unless a statement is executed,
  - We have no way of knowing if an error exists in that statement.

Statement Coverage Criterion

- Observing that a statement behaves properly for one input value:
  - No guarantee that it will behave correctly for all input values.

Statement Testing

- Coverage measurement:

$$\frac{\#\text{ executed statements}}{\#\text{ statements}}$$

- Rationale: a fault in a statement can only be revealed by executing the faulty statement
- statement coverage is a necessary but not a sufficient criterion for logic coverage

Example

```
int f1(int x, int y){
1 while (x != y){
2     if (x>y) then
3           x=x-y;
4     else y=y-x;
5 }
6 return x;          }
```

Euclid's GCD Algorithm

Euclid's GCD Algorithm

- By choosing the test set {(x=3,y=3),(x=4,y=3), (x=3,y=4)}
  - All statements are executed at least once.

Branch Coverage

- Test cases are designed such that:
  - different branch conditions
    - given true and false values in turn.

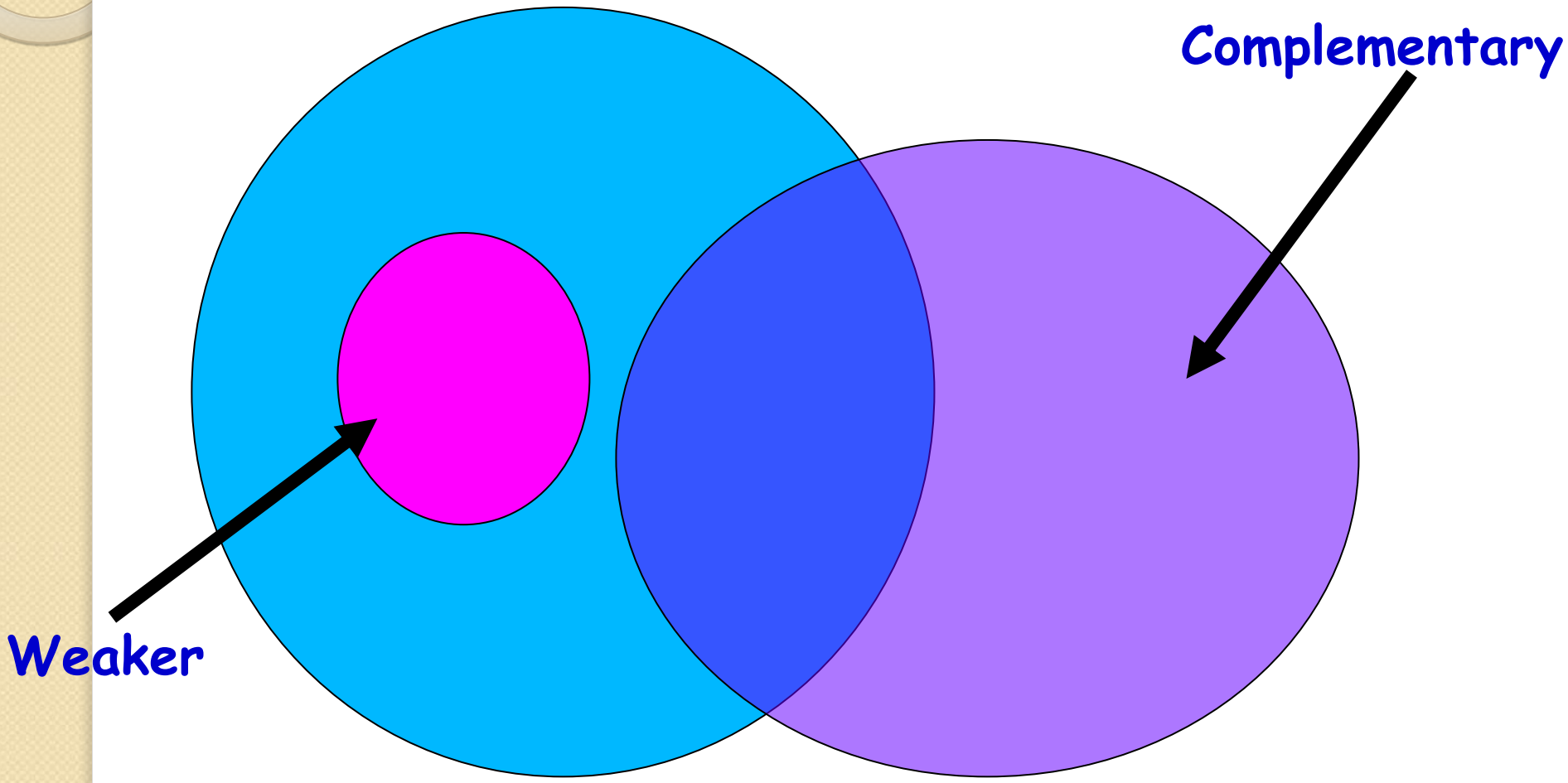
Branch Coverage

- Branch testing guarantees statement coverage:
  - A stronger testing compared to the statement coverage-based testing.

Stronger Testing

- Test cases are a superset of a weaker testing:
  - A stronger testing covers at least all the elements covered by a weaker testing.

Stronger, Weaker, and Complementary Testing



Example

```
int f1(int x,int y){
1 while (x != y){
2     if (x>y) then
3           x=x-y;
4     else y=y-x;
5 }
6 return x;          }
```

Example

- Test cases for branch coverage can be:
- {(x=3,y=3),(x=3,y=2), (x=4,y=3), (x=3,y=4)}

Branch Testing

- Adequacy criterion: Each branch (edge in the CFG) must be executed at least once
- Coverage:

$$\frac{\#\ \text{executed branches}}{\#\ \text{branches}}$$

Statements vs Branch Testing

- Traversing all edges of a graph causes all nodes to be visited
  - So test suites that satisfy the branch adequacy criterion for a program P also satisfy the statement adequacy criterion for the same program
- The converse is not true
  - A statement-adequate (or node-adequate) test suite may not be branch-adequate (edge-adequate)

Condition Coverage

- Test cases are designed such that:
  - Each component of a composite conditional expression
    - Given both true and false values.

Branch vs Condition testing

- Condition testing:
  - Stronger testing than branch testing.
- Branch testing:
  - Stronger than statement coverage testing.

Branch vs Condition testing

- Branch testing is the simplest condition testing strategy:
  - Compound conditions appearing in different branch statements
    - are given true and false values.

Multiple Condition Coverage

- Multiple condition coverage reports whether every possible combination of Boolean sub-expressions occurs.
- The test cases required for full multiple condition coverage of a condition are essentially given by the logical operator truth table for the condition.

Multiple Condition Coverage cont …

- Test cases are designed such that:
  - each component of a composite conditional expression
    - given both true and false values.

Example

- Consider the conditional expression
  - ((c1.and.c2).or.c3):
- Each of c1, c2, and c3 are exercised at least once,
  - i.e. given true and false values.

Multiple Condition Coverage Examples

```
if(A && B)   // condition 1
        F1();
else
        F2();
if(C || D)      // condition 2
        F3()
else
        F4();
```

Test Cases for MCC:

For condition 1	For condition 2
A=T, B=T	C=T, D=T
A=T, B=F	C=T, D=F
A=F, B=T	C=F, D=T
A=F, B=F	C=F, D=F

MCC Testing Adequacy Criterion

- Adequacy criterion: Each basic condition must be executed at least once
- Coverage:

$$\frac{\#\text{ truth values taken by all basic conditions}}{2 * \#\text{ basic conditions}}$$

Branch Testing vs MCC Testing

- Multiple Condition Coverage testing:
  - Stronger testing than branch testing.
- Branch testing:
  - Stronger than statement coverage testing.

Need of a Feasible Testing Technique

- Consider a boolean expression having **n** components:
  - For condition coverage we require 2^n test cases. (example in next slide)
- Condition coverage-based testing technique:
  - Practical only if **n** (the number of component conditions) is small.

Consider: if (a || b && c) then …

Test	a	b	c
(1)	T	T	T
(2)	T	T	F
(3)	T	F	T
(4)	T	F	F
(5)	F	T	T
(6)	T	T	F
(7)	F	F	T
(8)	F	F	F

MCC

Exponential in the number of basic conditions

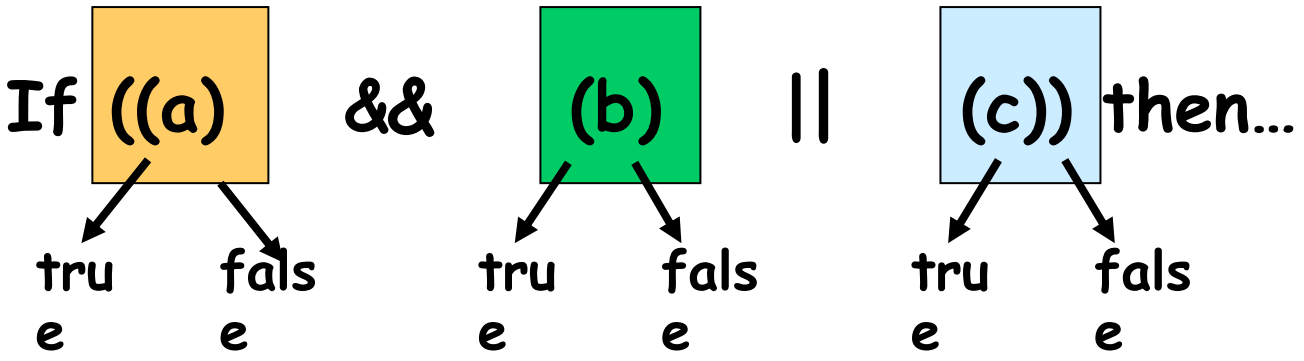
Modified condition/decision (MC/DC)

- Motivation: Effectively test important combinations of conditions, without exponential blowup in test suite size
  - "Important" combinations means: Each basic condition shown to independently affect the outcome of each decision
- Requires:
  - For each basic condition C, two test cases,
  - values of all *evaluated* conditions except C are the same
  - compound condition as a whole evaluates to *true* for one and *false* for the other

What is MC/DC?

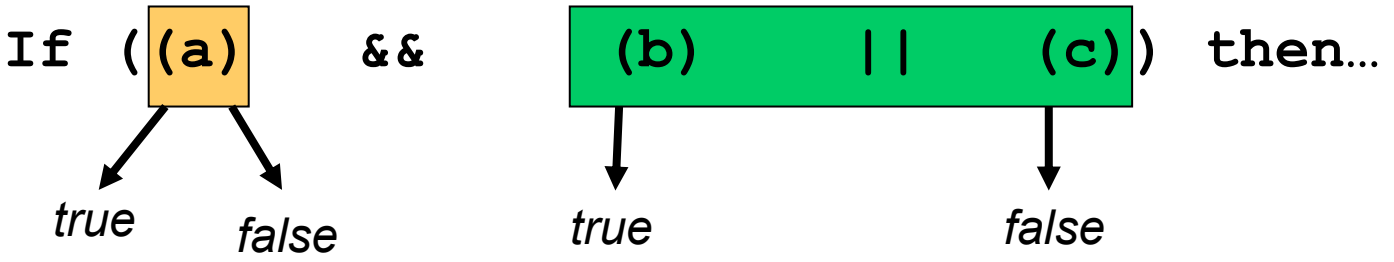
- MC/DC stands for **M**odified **C**ondition / **D**ecision **C**overage
- A kind of Predicate Coverage technique
  - Condition: Leaf level Boolean expression.
  - Decision: Controls the program flow.
- Main idea: Each condition must be shown to independently affect the outcome of a decision, i.e. the outcome of a decision changes as a result of changing a single condition.

Condition Coverage

- Every condition in the decision has taken all possible outcomes at least once.
- 

MC/DC Coverage

- Every condition in the decision independently affects the decision's outcome.



Change the value of each condition individually while keeping all other conditions constant.

Example

If (A && B) then…

(1) create truth table for conditions.

(2) Extend truth table so that it indicated which test cases can be used to show the independence of each condition.

A	B	Result
T	T	T
T	F	F
F	T	F
F	F	F



Number	A	B	Result	A	B
1	T	T	T	3	2
2	T	F	F		1
3	F	T	F	1	
4	F	F	F		

Example cont …

Number	A	B	Result	A	B
1	T	T	T	3	2
2	T	F	F		1
3	F	T	F	1	
4	F	F	F		

- Show independence of **A**:
  - Take 1 + 3
- Show independence of **B**:
  - Take 1 + 2
- Resulting test cases are
  - 1 + 2 + 3
  - (T , T) + (T , F) + (F , T)

More advanced example

```
If (A && (B || C)) then...
```

Number	A B C	Result	A	B	C
1	T T T	T	5		
2	T T F	T	6	4	
3	T F T	T	7		4
4	T F F	F		2	3
5	F T T	F	1		
6	F T F	F	2		
7	F F T	F	3		
8	F F F	F			

More advanced example contd..

Note: We want to determine the MINIMAL set of test cases

Here:

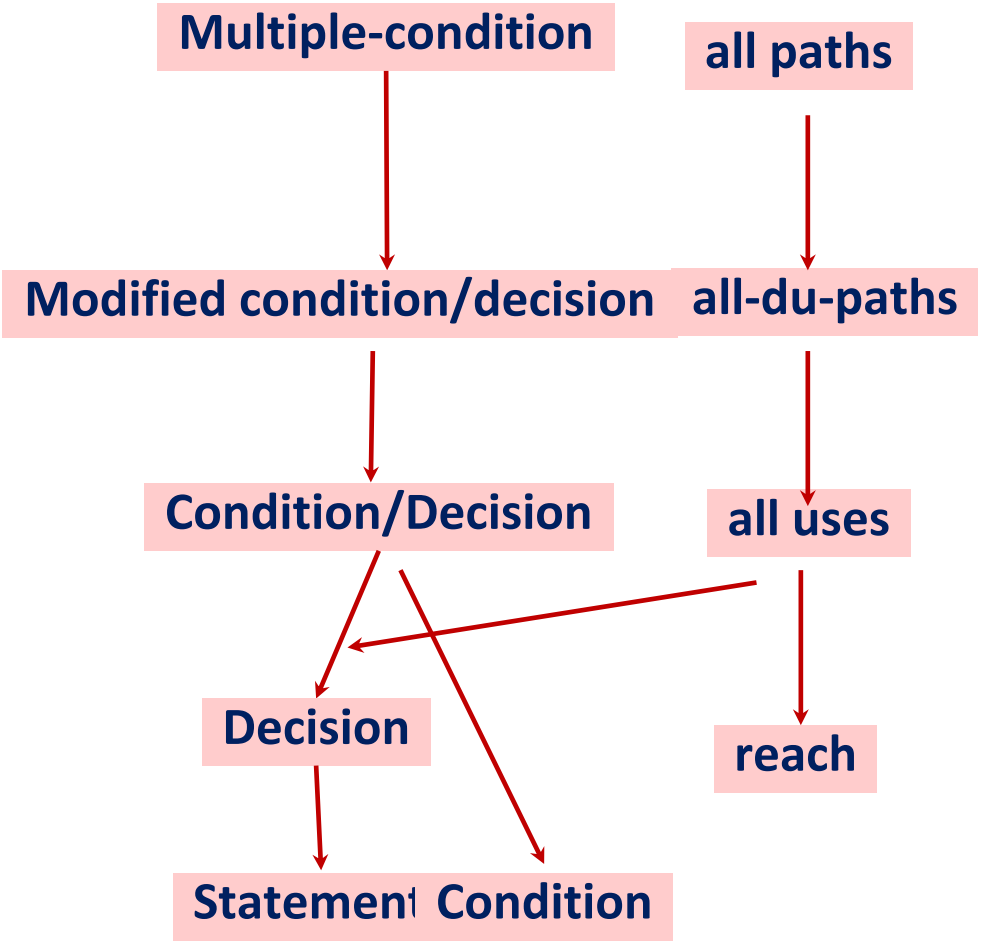
- {2,3,4,6}
- {2,3,4,7}

Non-minimal set is:

- {1,2,3,4,5}

Where does it fit in?

- The MC/DC criterion is much stronger than the condition/decision coverage criterion, but the number of test cases to achieve the MC/DC criterions still varies linearly with the number of conditions **n** in the decisions.
  - Much more complete coverage than condition/decision coverage, but
  - at the same time it is not terribly costly in terms of number of test cases.



Comparison of different coverage based Testing Strategies

Coverage Criteria	Statement Coverage	Decision Coverage	Condition Coverage	Condition/ Decision Coverage	MC/DC	Multiple Condition Coverage
Every point of entry and exit in the program has been invoked at least once		•	•	•	•	•
Every statement in the program has been invoked at least once	•					
Every decision in the program has taken all possible outcomes at least once		•		•	•	•
Every condition in a decision in the program has taken all possible outcomes at least once			•	•	•	•
Every condition in a decision has been shown to independently affect that decision's outcome					•	•[8]
Every combination of condition outcomes within a decision has been invoked at least once						•

weakest → strongest

Summary

- Discussed white-box test case design using:
  - Statement coverage technique
  - Branch coverage technique
  - Condition coverage technique
  - MC/DC technique

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 5), Second Edition, Oxford University Press, 2016.

Thank You

White-box testing techniques contd…

Dr. Durga Prasad Mohapatra
Professor, Dept. of CSE
N.I.T Rourkela

Path Coverage

- Design test cases such that:
  - All linearly independent paths in the program are executed at least once.
- Defined in terms of
  - Control flow graph (CFG) of a program.

Path Coverage-Based Testing

- To understand the path coverage-based testing:
  - we need to learn how to draw control flow graph of a program.
- A control flow graph (CFG) describes:
  - The sequence in which different instructions of a program get executed.
  - The way control flows through the program.

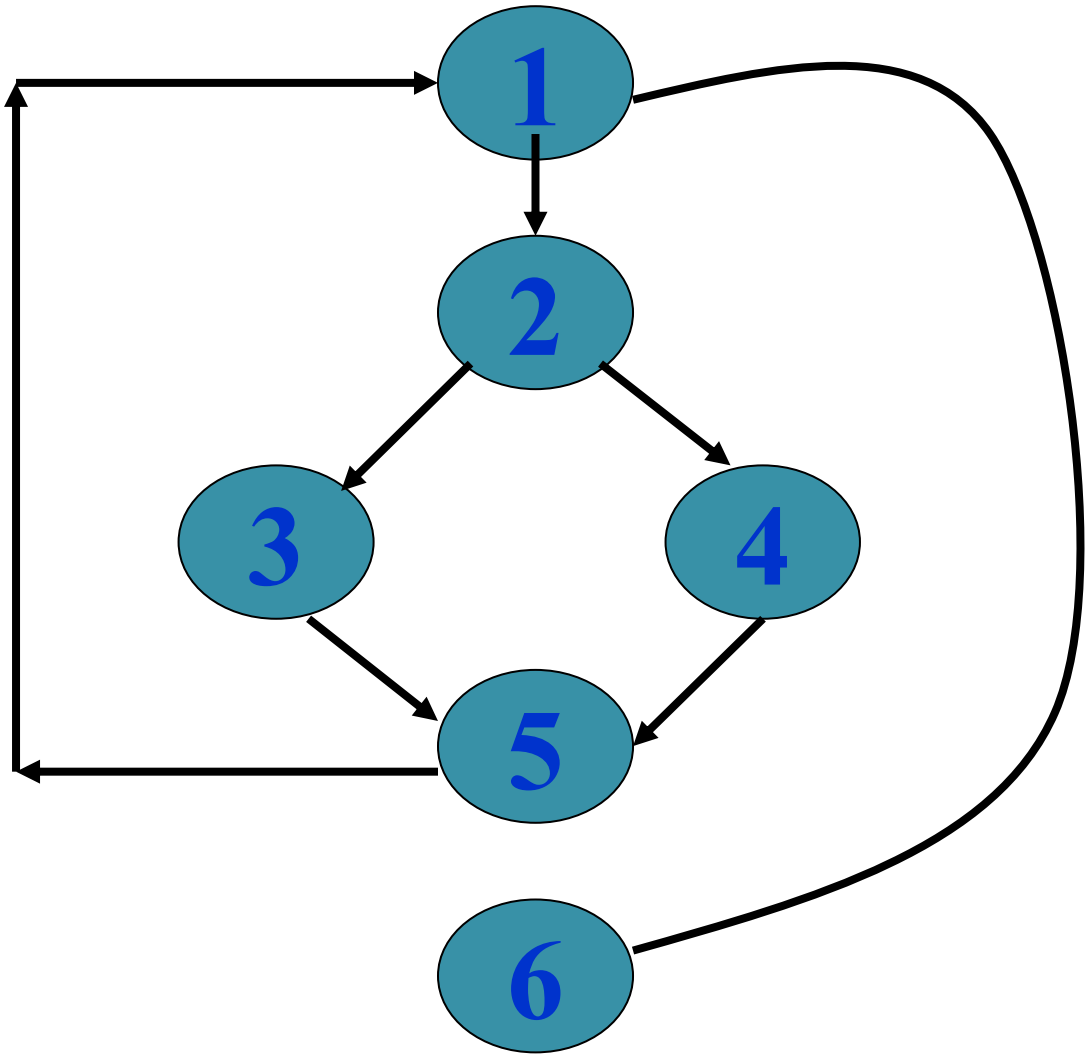
How to Draw Control Flow Graph?

- Number all statements of a program.
- Numbered statements:
  - Represent nodes of control flow graph.
- An edge from one node to another node exists:
  - If execution of the statement representing the first node
    - Can result in transfer of control to the other node.

Example

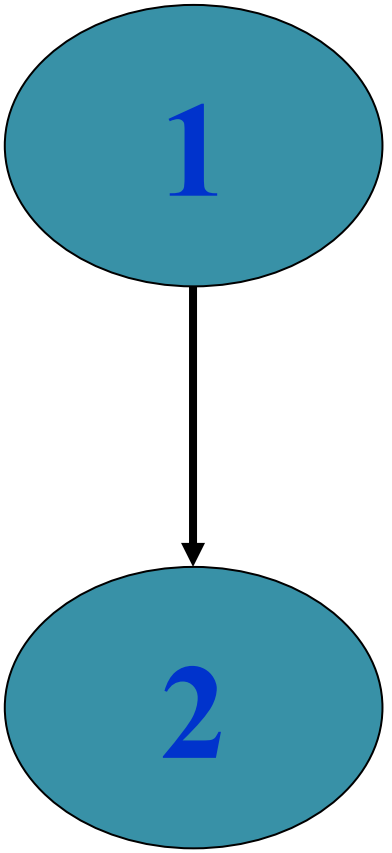
```
int f1(int x,int y){
1 while (x != y){
2    if (x>y) then
3         x=x-y;
4    else y=y-x;
5 }
6 return x;        }
```

Example Control Flow Graph



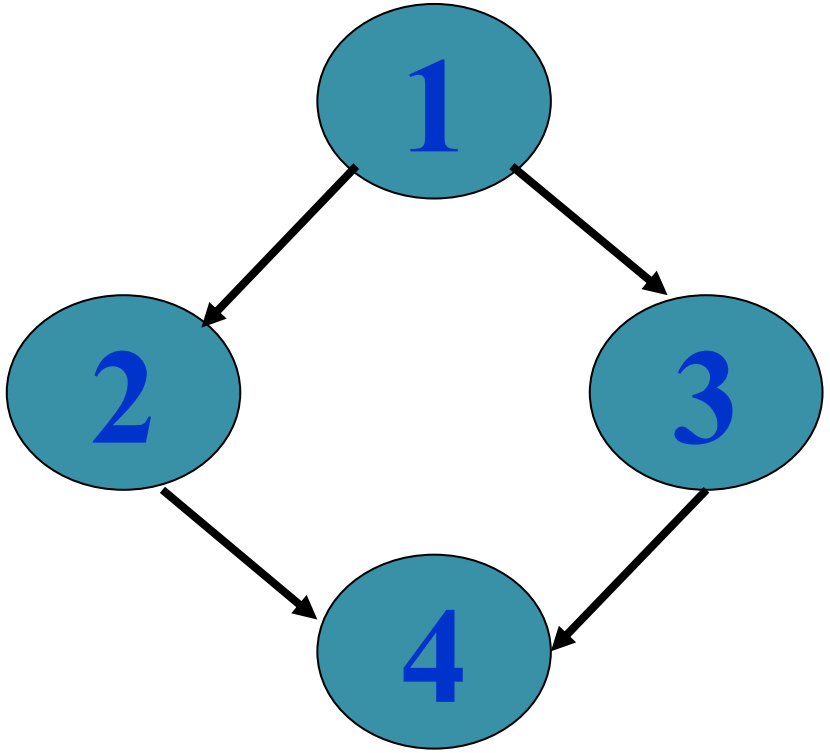
How to Draw Control flow Graph?

- **Sequence:**
  - **1 a=5;**
  - **2 b=a*b-1;**



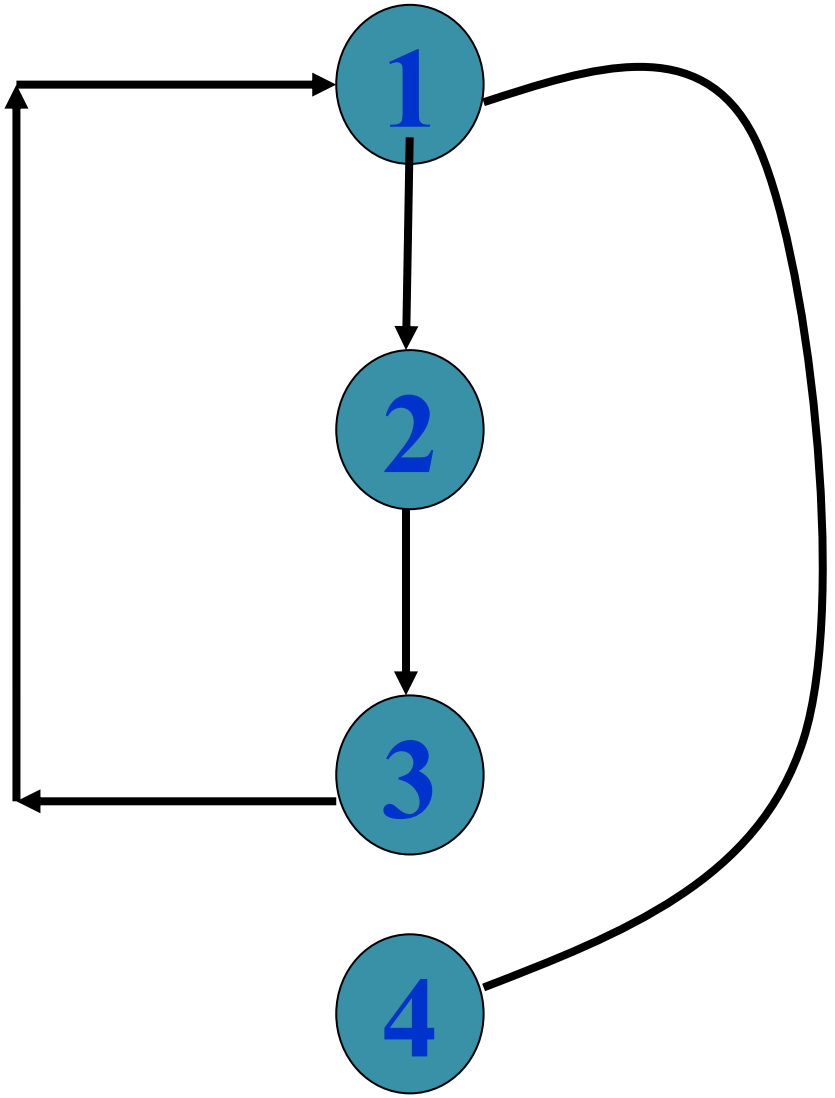
How to Draw Control Flow Graph?

- **Selection:**
  - **1 if(a>b) then**
  - **2 c=3;**
  - **3 else c=5;**
  - **4 c=c*c;**



How to Draw Control Flow Graph?

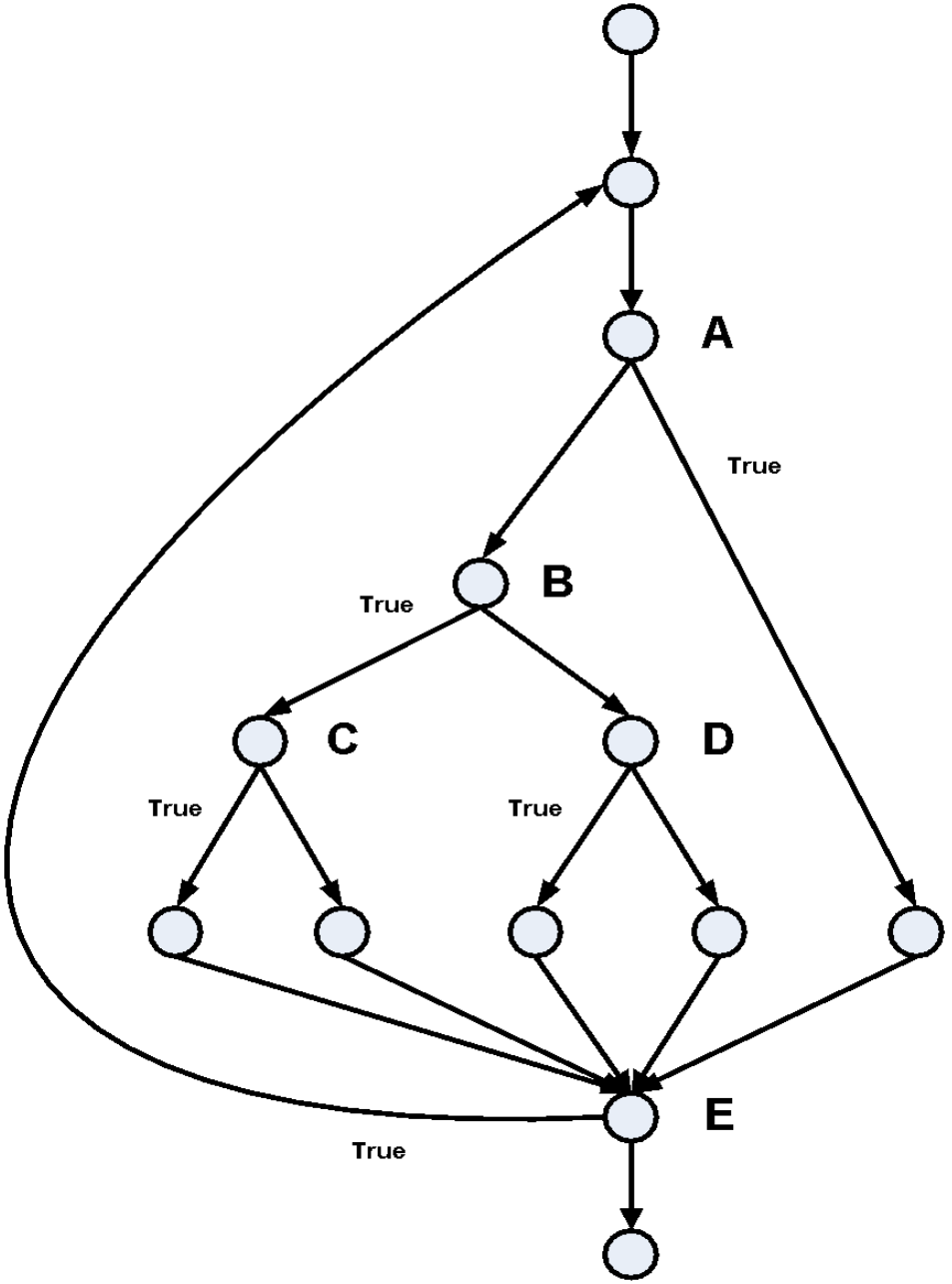
- **Iteration:**
  - **1 while(a>b){**
  - **2 b=b*a;**
  - **3 b=b-1;}**
  - **4 c=b+d;**



Example Code Fragment

```
Do
{
   if (A) then {...};
   else {
      if (B) then {
                  if (C) then {...};
                  else {…}
      }
      else if (D) then {...};
            else {...};
   }
}
While (E);
```

Example Control Flow Graph



Source: The Art of Software Testing – Glenford Myers

Path

- A path through a program:
  - A node and edge sequence from the starting node to a terminal node of the control flow graph.
  - There may be several terminal nodes for program.

Linearly Independent Path

- Any path through the program:
  - Introduces at least one new edge:
    - Not included in any other independent paths.

Independent path

- It is straight forward:
  - To identify linearly independent paths of simple programs.
- For complicated programs:
  - It is not easy to determine the number of independent paths.

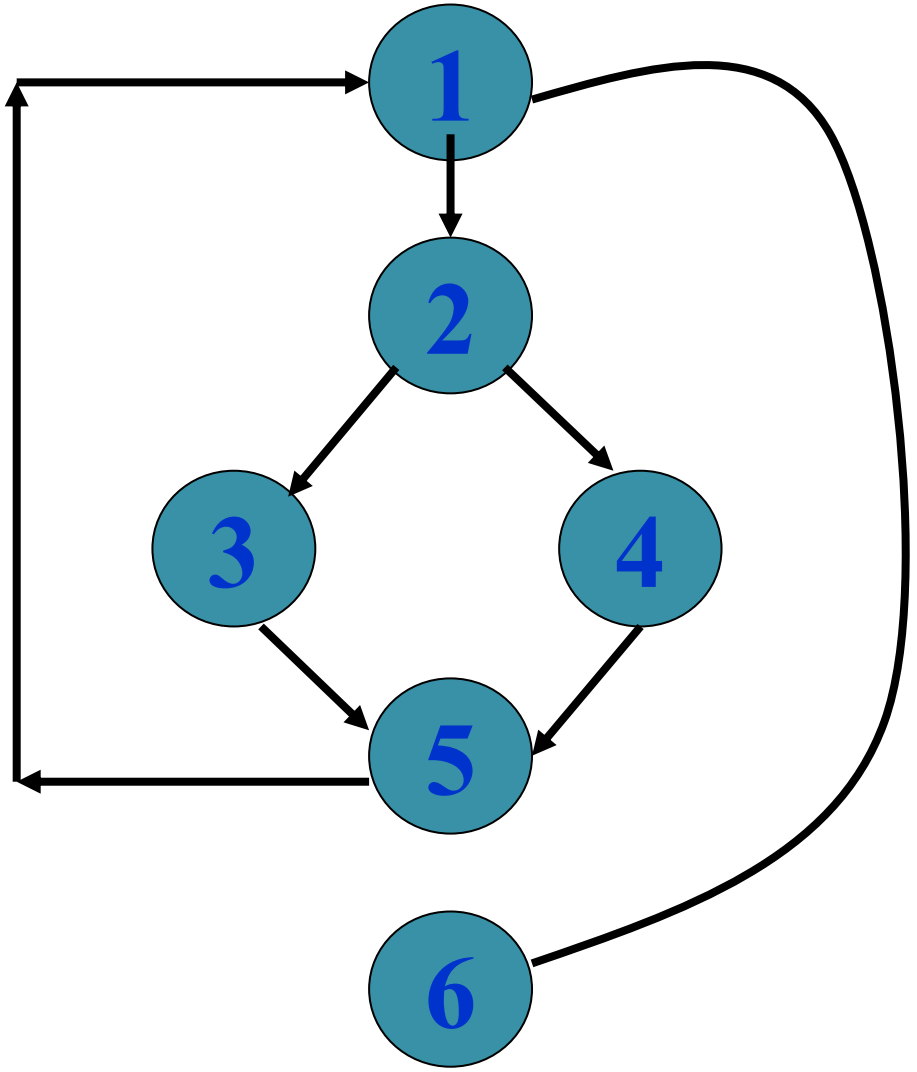
McCabe's Cyclomatic Metric

- An upper bound:
  - For the number of linearly independent paths of a program
- Provides a practical way of determining:
  - The maximum number of linearly independent paths in a program.

McCabe's Cyclomatic Metric

- Given a control flow graph G, cyclomatic complexity V(G):
  - V(G)= E-N+2
    - N is the number of nodes in G
    - E is the number of edges in G

Example Control Flow Graph

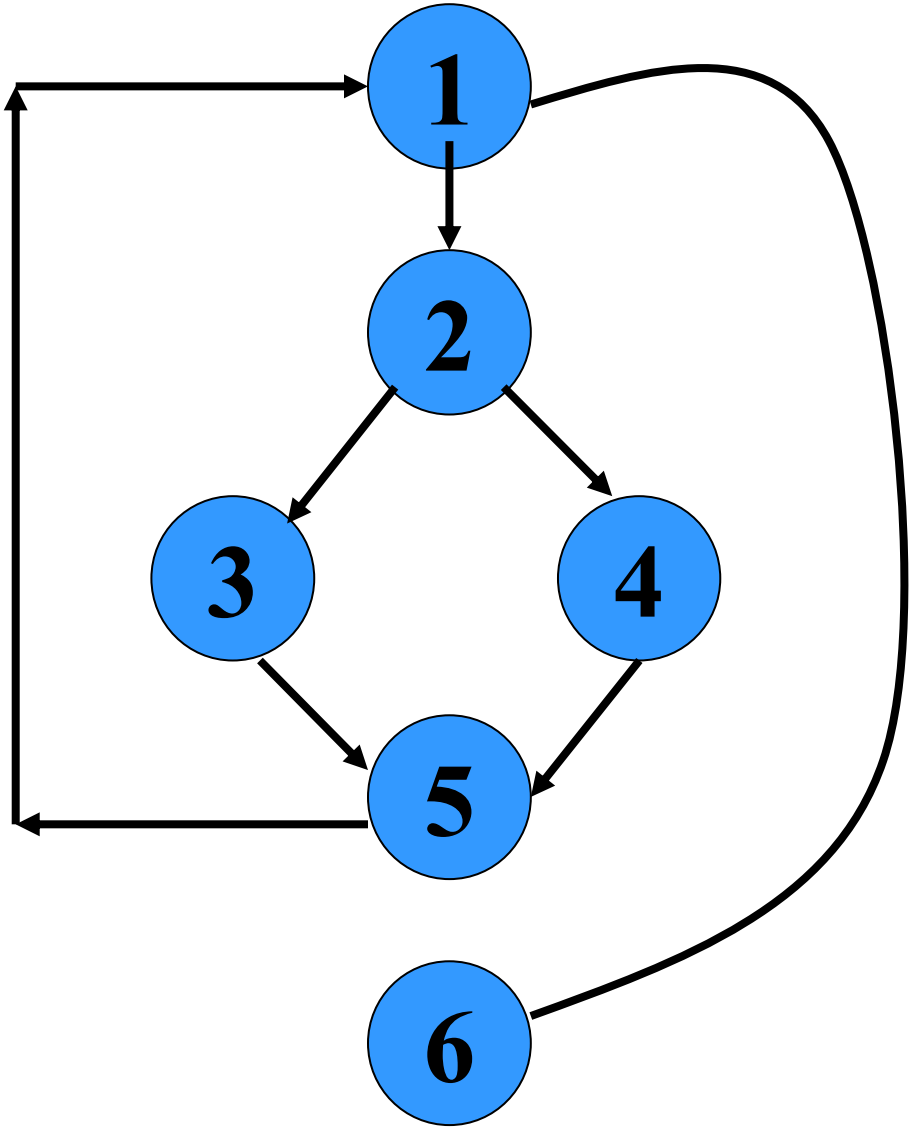


Cyclomatic complexity = 7-6+2 = 3.

Cyclomatic Complexity

- Another way of computing cyclomatic complexity:
  - inspect control flow graph
  - determine number of bounded areas in the graph
- V(G) = Total number of bounded areas + 1
  - Any region enclosed by a nodes and edge sequence.

Example Control Flow Graph



Example

- From a visual examination of the CFG:
  - Number of bounded areas is 2.
  - Cyclomatic complexity = 2+1=3.

Cyclomatic Complexity

- McCabe's metric provides:
  - A quantitative measure of testing difficulty and the ultimate reliability
- Intuitively,
  - Number of bounded areas increases with the number of decision nodes and loops.

Cyclomatic Complexity

- The first method of computing V(G) is amenable to automation:
  - You can write a program which determines the number of nodes and edges of a graph
  - Applies the formula to find V(G).

Cyclomatic Complexity

- The cyclomatic complexity of a program provides:
  - A lower bound on the number of test cases to be designed
  - To guarantee coverage of all linearly independent paths.

Cyclomatic Complexity

- A measure of the number of independent paths in a program.
- Provides a lower bound:
  - for the number of test cases for path coverage.

Cyclomatic Complexity

- Knowing the number of test cases required:
  - Does not make it any easier to derive the test cases,
  - Only gives an indication of the minimum number of test cases required.

Practical Path Testing

- The tester proposes initial set of test data :
  - Using his experience and judgment.
- A dynamic program analyzer used:
  - Measures which parts of the program have been tested
  - Result used to determine when to stop testing.

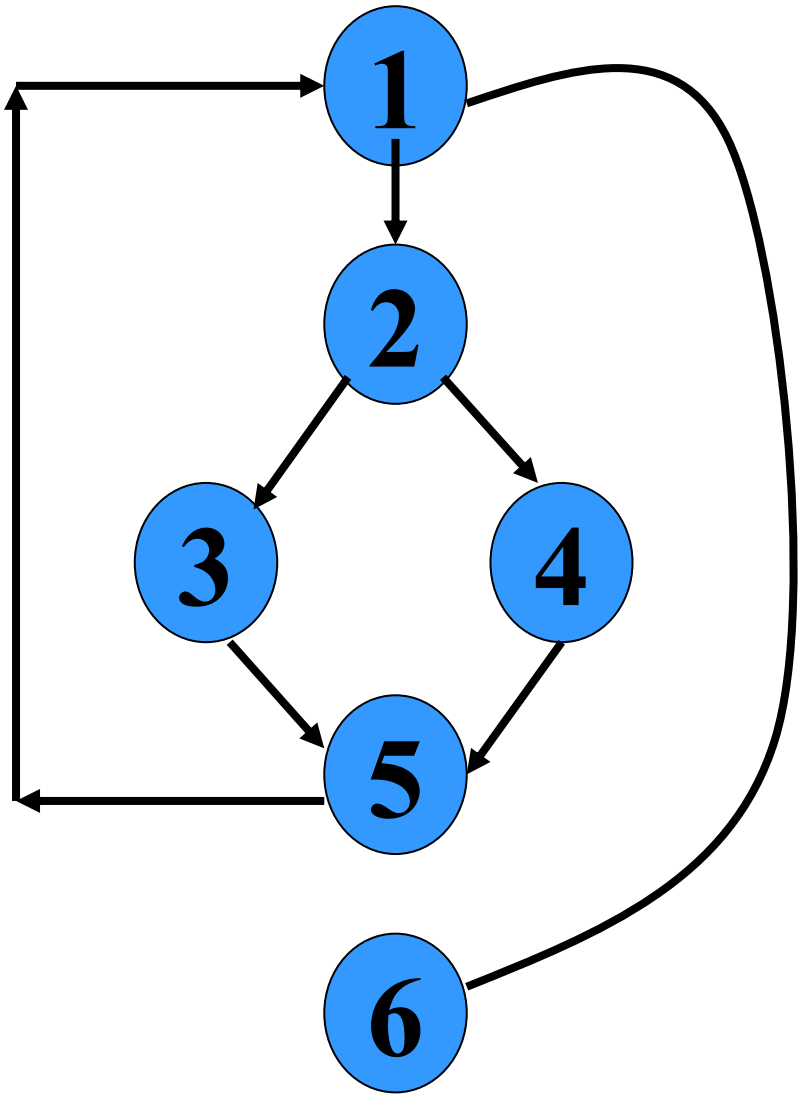
Derivation of Test Cases

- Draw control flow graph.
- Determine V(G).
- Determine the set of linearly independent paths.
- Prepare test cases:
  - to force execution along each path.

Example

```
int f1(int x,int y){
1 while (x != y){
2     if (x>y) then
3           x=x-y;
4     else y=y-x;
5 }
6 return x;          }
```

Example Control Flow Diagram



Derivation of Test Cases

- **Number of independent paths: 3**
  - **1,6** test case (x=1, y=1)
  - **1,2,3,5,1,6** test case(x=2, y=1)
  - **1,2,4,5,1,6** test case(x=1, y=2)

An Interesting Application of Cyclomatic Complexity

- Relationship exists between:
  - McCabe's metric
  - The number of errors existing in the code,
  - The time required to find and correct the errors.

Cyclomatic Complexity

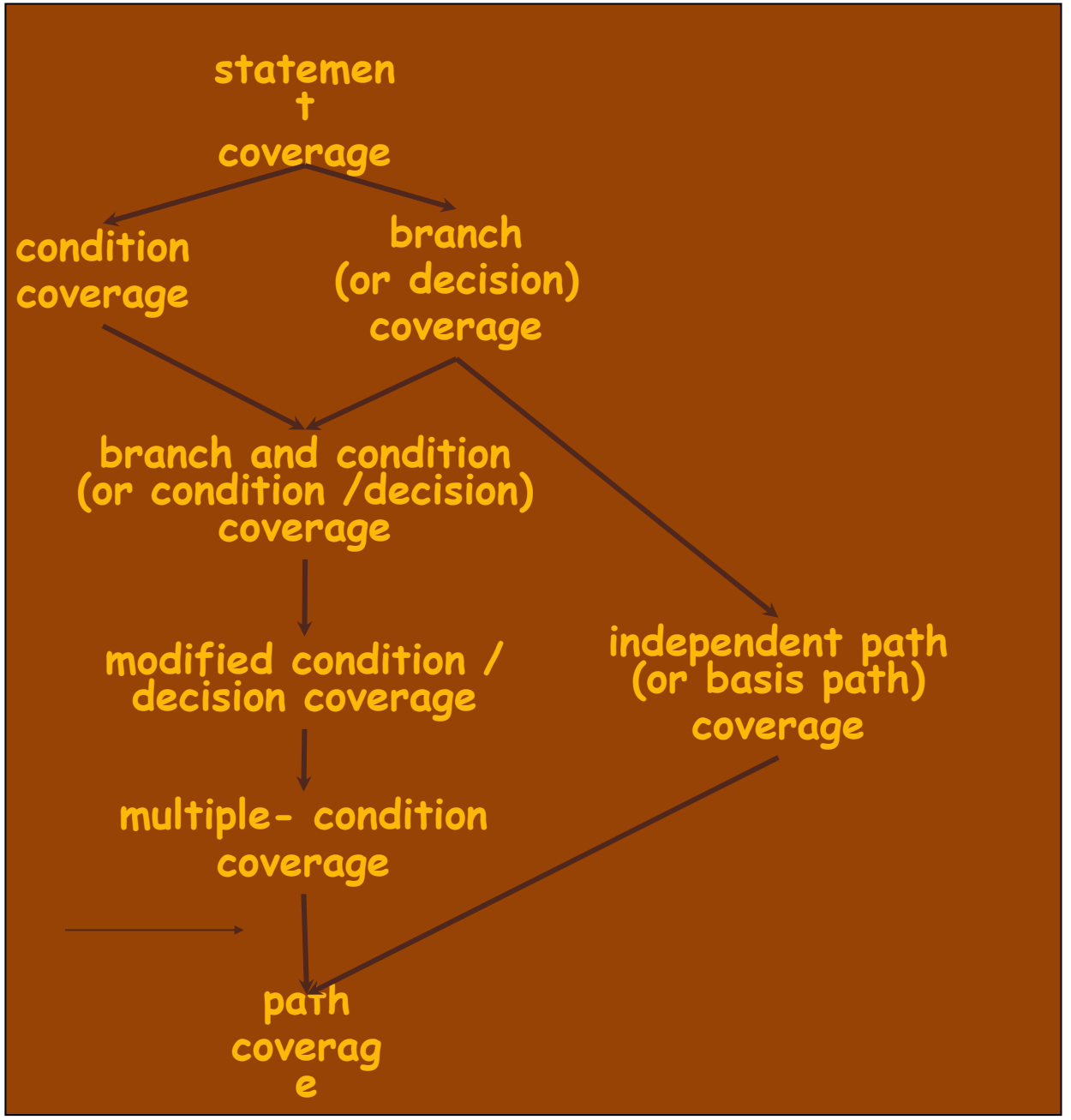
- Cyclomatic complexity of a program:
  - Also indicates the psychological complexity of a program.
  - Difficulty level of understanding the program.

Cyclomatic Complexity

- From maintenance perspective,
  - Limit cyclomatic complexity of modules
    - To some reasonable value.
  - Good software development organizations:
    - Restrict cyclomatic complexity of functions to a maximum of ten or so.

White-Box Testing : Summary

weakest



only if paths across composite conditions are distinguished

strongest

Summary

- There are two approaches to testing:
  - black-box testing and
  - white-box testing.
- Designing test cases for black box testing:
  - does not require any knowledge of how the functions have been designed and implemented.
  - Test cases can be designed by examining only SRS document.

Summary

- White box testing:
  - Requires knowledge about internals of the software.
  - Design and code is required.
- We have discussed a few white-box test strategies.
  - Statement coverage
  - Branch coverage
  - Condition coverage
  - MC/DC coverage
  - Path coverage

Summary

- A stronger testing strategy:
  - Provides more number of significant test cases than a weaker one.
  - Condition coverage is strongest among strategies we discussed.
- We discussed McCabe's Cyclomatic complexity metric:
  - Provides an upper bound for linearly independent paths
  - Correlates with understanding, testing, and debugging difficulty of a program.

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 5), Second Edition, Oxford University Press, 2016.

Thank you

Concolic Testing: A modern software testing technique

Dr. Durga Prasad Mohapatra
Professor
Department of CSE, NIT Rourkela



Seminar Outline

1 Introduction

2 Fundamental Ideas

3 Survey of Related works



Introduction

Software Testing is an important phase in SDLC.

- Helps to achieve software dependability and improve quality.
- Time consuming: Approximately 40% of software development time is devoted to testing.
- Testing can be done in two ways – Manual vs. Automated.
- **An automation tool** for test case generation can effectively reduce the time required in testing.
- **Automation tool** can be designed to generate test



Reliability and Coverage Model -By Prof. Aditya Mathur

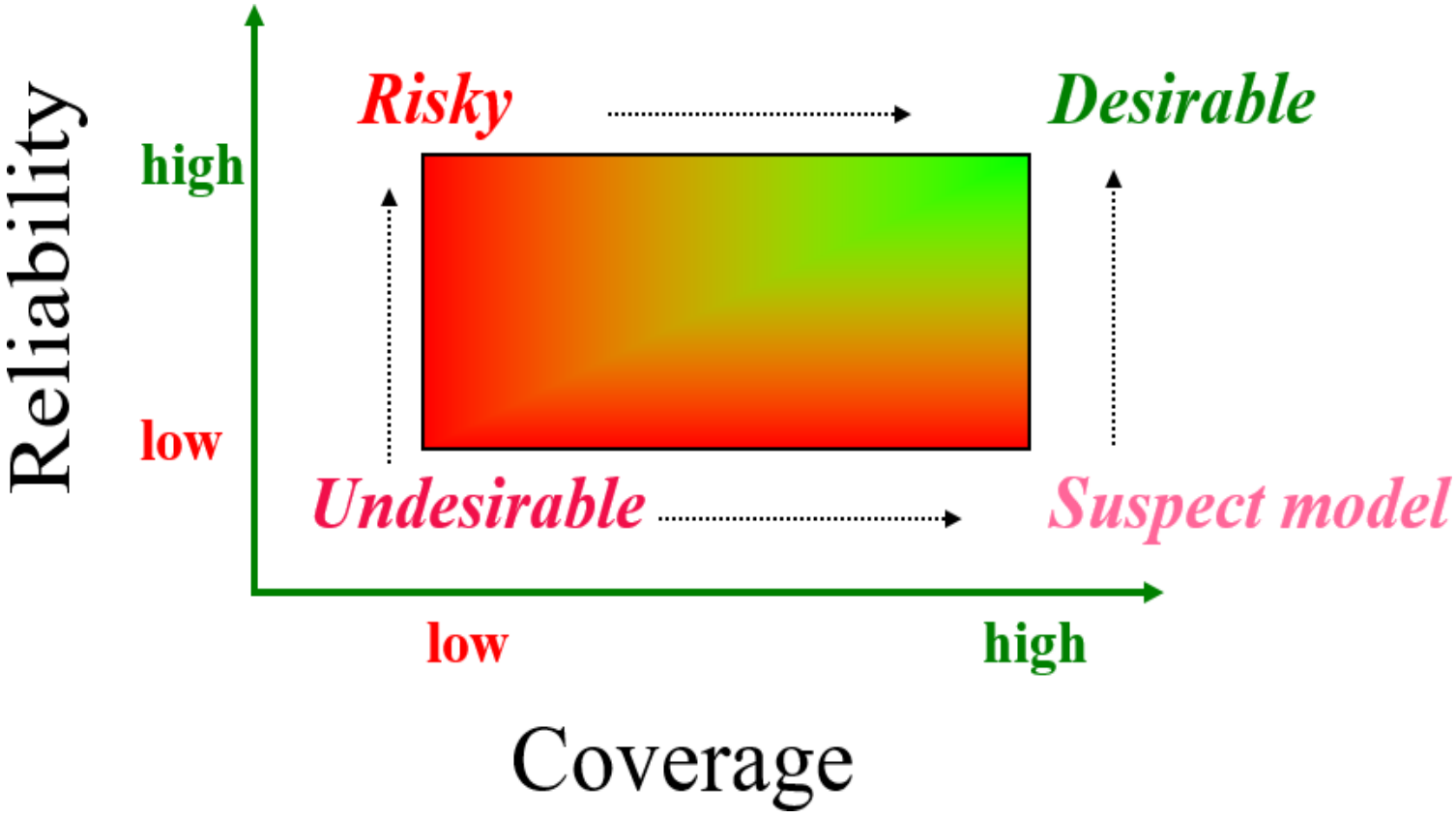


Figure 1: Reliability and Coverage



Software Testing

"No product of human intellect comes out right the first time. We rewrite sentences, rip out knitting stitches, replant gardens, remodel houses, and repair bridges. Why should software be any different?"

- By Wiener, Ruth: Digital Woes, Why We Should Not Depend on Software.



Software Testing

- The purpose of the verification process is to detect and report errors that have been introduced in the development process.
- The verification process must ensure that the produced software implements intended function completely and correctly, while avoiding unintended function.

Verification is an integral process, which is coupled with every development step. Testing quality at the end of the life cycle is impractical.



Software Testing: Coverage

- Coverage refers to the extent to which a given verification activity has satisfied its objectives: in essence, providing an exit criteria for when to stop. That is, what is “enough”is defined in terms of coverage.
- Coverage is a measure, not a method or a test. As a measure, coverage is usually expressed as the percentage of an activity that is accomplished.
- **Our goal, then, should be to provide enough testing to ensure that the probability of failure due to hibernating bugs is low enough to accept. “Enough”implies judgment.**



RTCA/DO178-B/C standards

- RTCA stands for Radio Technical Commission for Aeronautics.
- DO-178B (and DO-278) are used to assure safety of avionics software.
- DO-178C includes the coverage analysis of Object-Oriented Programs used in safety of avionics software.
- These documents provide guidance in the areas of SW development, configuration management, verification and the interface to approval authorities (e.g., FAA, EASA).



Levels of Software

- Different failure conditions require different software conditions → 5 levels

Failure Condition	Software Level
Catastrophic	Level A
Hazardous/Severe - Major	Level B
Major	Level C
Minor	Level D
No Effect	Level E

Figure 2: Levels of Software



Relation of Coverages

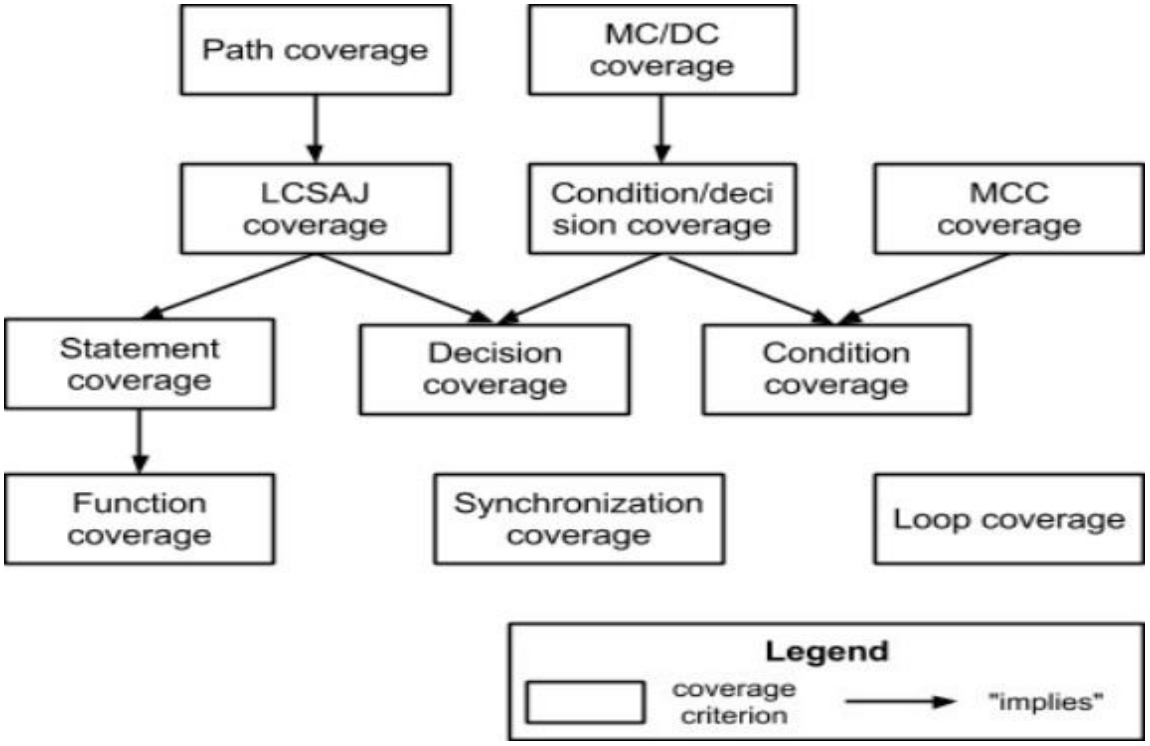


Figure 3: Relation of Coverages

LCSAJ Coverage

- **LCSAJ** stands for Linear Code Sequence and Jump.
- It is a white box **testing** technique to identify the **code coverage**.
- It begins at the start of the program or branch and ends at the end of the program or the branch.
- **LCSAJ** is ordinarily equivalent to **statement coverage**.



Types of Structural Coverage

Coverage Criteria	Statement Coverage	Decision Coverage	Condition Coverage	Condition/ Decision Coverage	MC/DC	Multiple Condition Coverage
Every point of entry and exit in the program has been invoked at least once		•	•	•	•	•
Every statement in the program has been invoked at least once	•					
Every decision in the program has taken all possible outcomes at least once		•		•	•	•
Every condition in a decision in the program has taken all possible outcomes at least once			•	•	•	•
Every condition in a decision has been shown to independently affect that decision's outcome					•	•[8]
Every combination of condition outcomes within a decision has been invoked at least once						•

weakest → strongest

Figure 4: Types

Definitions

Predicate / Boolean Expression

A predicate is an expression that evaluates to a boolean value, and which is required for our approach.

A simple example is: $((a > b) \vee C) \wedge p(x)$. Predicates may contain boolean variables, non-boolean variables that are compared with relational operators, and calls to function that return a boolean value, all three of which may be joined with logical operators.



Definitions

Predicate Coverage

For each p $\in$ P, Test Requirement (TR) for predicate coverage contains two requirements: p evaluates to true, and p evaluates to false.



Definitions

Clause / Atomic Condition

A clause is a predicate that does not contain any of the logical operators.

The predicate $((a > b) \vee C) \wedge p(x)$ contains three clauses; a relational expression $(a > b)$, a boolean variable C and a boolean function call p(x).



Definitions

Clause Coverage

For each c $\in$ C, TR for clause coverage contains two requirements: c evaluates to true, and c evaluates to false.



Definitions

Statement Coverage

- The statement coverage based strategy aims to design the test cases so as to execute every statement in a program at least once.
- The principle idea governing the statement coverage strategy is that a unless a statement is executed, there is no way to determine whether as error exists in that statement.



Definitions

Branch Coverage

Each decision should take all possible outcomes at least once either true or false.

For example if(m >n), the test cases are (1) m<=n, (2) m>n



Example

```
1   int computeGCD(x,y)
2       int x,y;
3       {
4           while (x!=y){
5               if(x>y) then
6               x=x-y;
7               else
8               y=y-x;
9           }
10          return x;
11      }
```

Figure 5: Euclid's GCD computation program



Test cases for Statement Coverage

- To design the test cases for the statement coverage, the conditional expression of the while statements needs to be made true and the conditional expression of the if statement needs to be made both true and false.
- By choosing the test set {(x=3,y=3), (x=4,y=3),(x=3,y=4)}, all the statements of the program would be executed at-least once.



Test cases for Statement Coverage

- For the GCD program, the test cases for branch coverage can be {(x=3,y=3), (x=3,y=2), (x=4,y=3),(x=3,y=4)}.



Observation

- It is easy to show that branch coverage based testing is a stronger testing than statement coverage-based testing.

  We can prove this by showing that branch coverage ensures statement coverage, but not vice-versa.



Definitions

Concolic Testing

- The concept of CONCOLIC testing combines the CONCrete constraints execution and symBOLIC constraints execution to automatically generate test cases for full path coverage.
- This testing generates test suites by executing the program with random values.
- At execution time both concrete and symbolic values are saved for execution path.
- During execution, the variables are stored in some symbolic values such as x_0, and y_0, instead of x and y.
- The next iteration of the process forces the selection of different paths.



Definitions

Concolic Testing

- The tester selects a value from the path constraints and negates the values to create a new path value. Then the tester finds concrete constraints to satisfy the new path values.
- The selection of values is responsible by the Constraint Solver which is a part of Concolic tester.
- These constraints are inputs for all next executions. This concolic testing is performed iteratively until exceeds the threshold value or sufficient code coverage is obtained.



Concolic Testing Example

Concolic Testing | Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```



Concolic Testing Example

Concolic Testing | Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Reads input values

We want to violate this



Concolic Testing Example

Concolic Testing | Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```



Concolic Testing Example

Concolic Testing | Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

← 5
← 10

Assign Random Values



Concolic Testing Example

Concolic Testing Example

```
1:    x = input();   ← 5
2:    y = input();   ← 10
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x		
y		

Execution Tree



Concolic Testing Example

Concolic Testing — Example

```
1:    x = input();   ← 5
2:    y = input();   ← 10
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	5	i_0
y		

Execution Tree



Concolic Testing Example

Concolic Testing — Example

```
1:   x = input();   ← 5
2:   y = input();   ← 10
3:   if (x > y) {
4:       assert(x != 100);
5:   }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	5	i_0
y	10	i_1

Execution Tree



Concolic Testing Example

Concolic Testing — Example

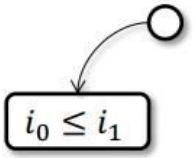
```
1:    x = input();   ← 5
2:    y = input();   ← 10
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	5	i_0
y	10	i_1

Execution Tree

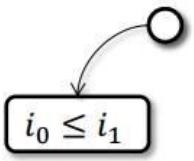


Concolic Testing Example

Concolic Testing — Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Execution Tree

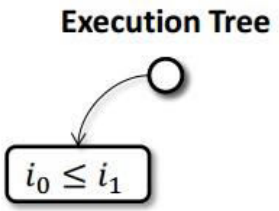




Concolic Testing Example

Concolic Testing | Example

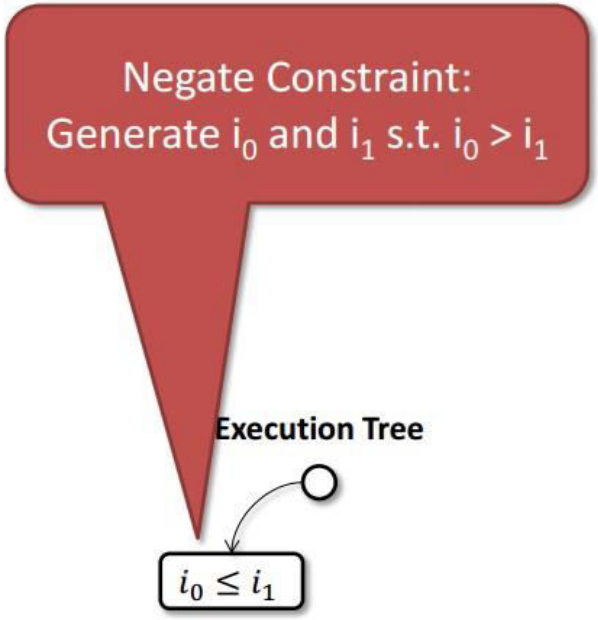
```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```



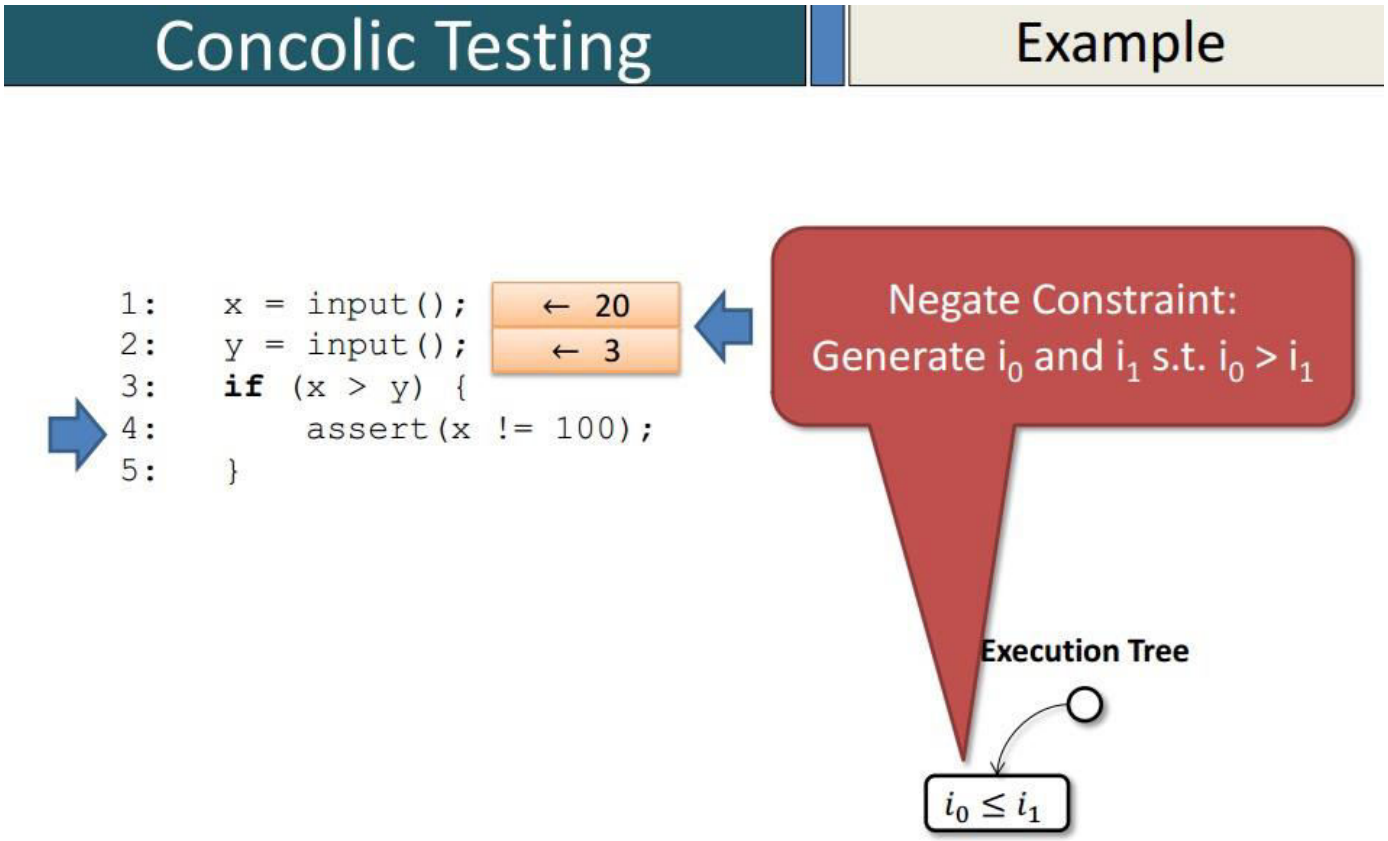
Concolic Testing Example

Concolic Testing — Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```



Concolic Testing Example



Concolic Testing Example

Concolic Testing — Example

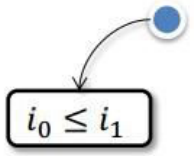
```
1:    x = input();   ← 20
2:    y = input();   ← 3
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x		
y		

Execution Tree



Concolic Testing Example

Concolic Testing | Example

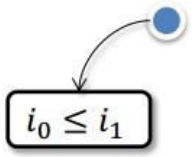
```
1:    x = input();   ← 20
2:    y = input();   ← 3
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	20	i_0
y		

Execution Tree



Concolic Testing Example

Concolic Testing — Example

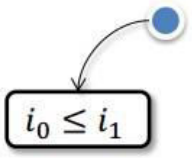
```
1:    x = input();     ← 20
2:    y = input();     ← 3
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	20	i_0
y	3	i_1

Execution Tree



Concolic Testing Example

Concolic Testing — Example

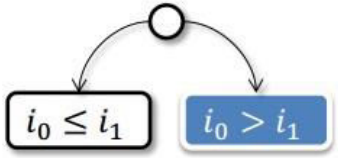
```
1:    x = input();   ← 20
2:    y = input();   ← 3
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	20	i_0
y	3	i_1

Execution Tree





Concolic Testing Example

Concolic Testing — Example

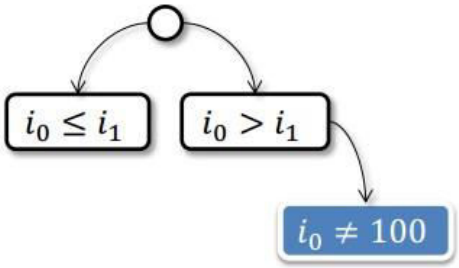
```
1:    x = input();   ← 20
2:    y = input();   ← 3
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	20	i_0
y	3	i_1

Execution Tree

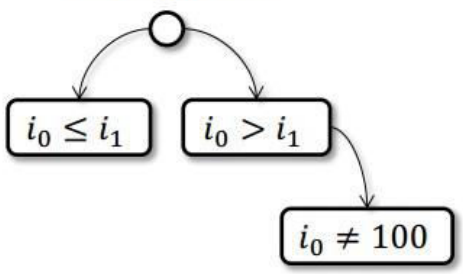


Concolic Testing Example

Concolic Testing — Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

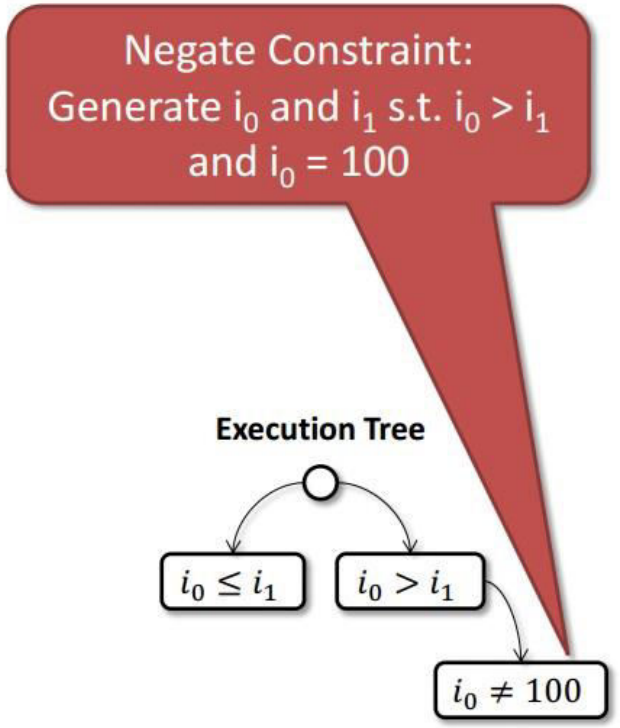
Execution Tree



Concolic Testing Example

Concolic Testing | Example

```
1:    x = input();
2:    y = input();
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

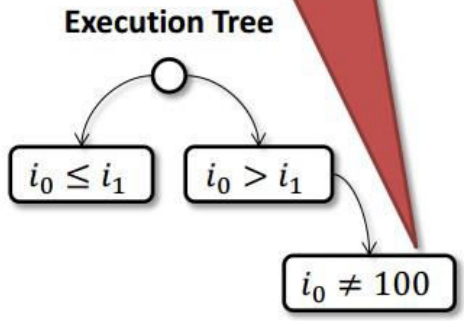


Concolic Testing Example

Concolic Testing — Example

```
1:    x = input();   ← 100
2:    y = input();   ← 4
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Negate Constraint:
Generate i_0 and i_1 s.t. $i_0 > i_1$
and $i_0 = 100$



Concolic Testing Example

Concolic Testing — Example

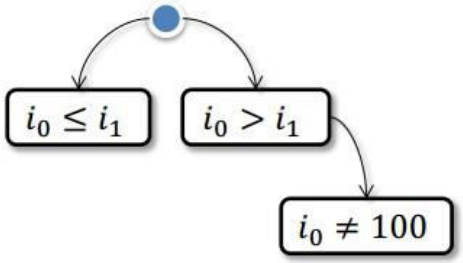
```
1:    x = input();   ← 100
2:    y = input();   ← 4
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x		
y		

Execution Tree



Concolic Testing Example

Concolic Testing — Example

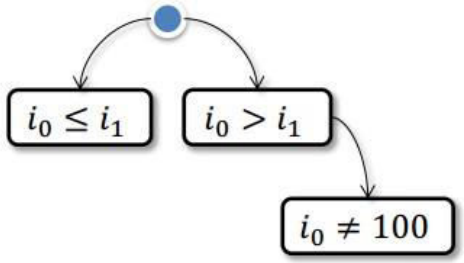
```
1:    x = input();   ← 100
2:    y = input();   ← 4
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	100	i_0
y		

Execution Tree



Concolic Testing Example

Concolic Testing — Example

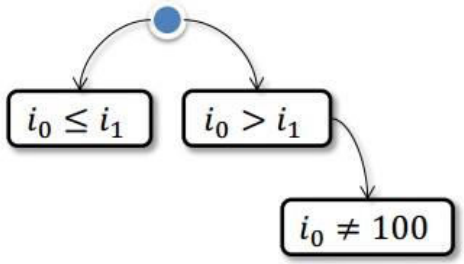
```
1:   x = input();   ← 100
2:   y = input();   ← 4
3:   if (x > y) {
4:       assert(x != 100);
5:   }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	100	i_0
y	4	i_1

Execution Tree



Concolic Testing Example

Concolic Testing — Example

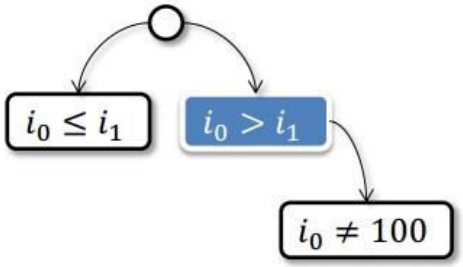
```
1:    x = input();   ← 100
2:    y = input();   ← 4
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

Concolic Execution

Memory

	Concrete	Symbolic
x	100	i_0
y	4	i_1

Execution Tree



Concolic Testing Example

Concolic Testing | Example

```
1:    x = input();   ← 100
2:    y = input();   ← 4
3:    if (x > y) {
4:        assert(x != 100);
5:    }
```

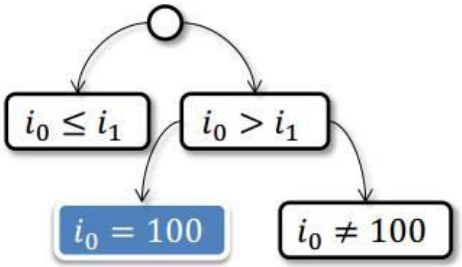


Concolic Execution

Memory

	Concrete	Symbolic
x	100	i_0
y	4	i_1

Execution Tree



Definitions

Concolic Testing: Final report

- Concolic testing technique explored total 3 paths for the example program.
- It has generated total three test cases for the variables x and y. These are shown in Table 1.

Table 1: Test cases

Test cases	TC1	TC2	TC3
x	5	20	100
y	10	3	4



Definitions

Distributed Concolic Testing

- *Distributed Concolic Testing (DCT)* is an extension of the original concolic testing approach that uses several computing nodes in a distributed manner.
- It significantly reduces the time to generate test cases with better efficiency.
- Concolic testing is the combination of concrete and symbolic testing. In addition, distributed concolic testing is scalable and achieves a linear speedup by using a large number of computing nodes for test case generation.



Concolic Testers

In Table 1, we have compared concolic testing tools with respect to the following parameters/features: 1) variables types; 2) pointers; 3) native calls; 4) non-linear arithmetic operations; 5) bitwise operations; 6) array offsets and 7) function pointers.

The abbreviation used in Table 1 are the following: "Y "means the tool supports the feature. "N"means the tool does not support the feature.

"P"means the tool can partially support the feature. "NA"means unknown.



Concolic Testers

Table 2: Summary of concolic tester with their properties.

Tool Name	Supporting Language	Supporting Platform	Support Constraints Solver.	Support for float/double	Support for pointer	Support for native call	Support for non-linear arithmetic op.	Support for bitwise op.	Support for offset	Support for function pointer
DART	C	NA	LP SOLVER	N	N	N	NA	NA	N	N
SMART	C	LINUX	LP SOLVER	N	N	N	NA	NA	N	N
CUTE	C	LINUX	LP SOLVER	N	Y	N	NA	NA	N	N
jCUTE	**JAVA**	**LINUX/WINDOWS**	**NA**	**N**	**-**	**N**	**NA**	**NA**	**N**	**N**
CREST	**C**	**LINUX**	**YICES**	**N**	**N**	**N**	**P**	**P**	**N**	**N**
EXE	C	LINUX	STP	N	Y	N	Y	Y	Y	N
KLEE	C	LINUX	STP	N	Y	P	Y	Y	Y	NA
RWSET	C	LINUX	STP	N	Y	N	Y	Y	Y	NA
jFUZZ	JAVA	LINUX	BUILT ON JPF	N	NA	N	N	N	NA	NA
PATH CRAWLER	C	NA	NA	NA	NA	N	NA	NA	NA	NA
PEX	.NET	WINDOWS	Z3	N	NA	N	NA	NA	NA	NA
SAGE	MACHINE CODE	WINDOWS	DISOLVER	NA	N	Y	NA	NA	NA	NA
APOLLO	PHP	WINDOWS	CHOCO	NA	NA	N	NA	N	NA	NA



Comparison of related works

Table 3: Summary of different work on concolic testing.

S.No	Authors	Testing Type	FrameWork Type	Input Type	Output Type
1	Das et al. [16]	Concolic Testing, MC/DC	BCT,CREST, CA	C-Program	MC/DC%
2	Bokil et al. [24]	SC, DC, BC,MC/DC	AutoGen	C-Program	Test data, Time
3	Kim et al. [20]	HCT	SMT Solver, CREST	Flash storage Platform Software	Reduction Ratio
4	Majumdar et al. [17]	HCT, BC	CUTE	Editor in C-Language	Test Cases
5	Burnim et al. [21]	Heuristics Concolic Testing, BC	CREST	Software Application in C	Branch Covered
6	Kim et al. [23]	Concolic Testing	CREST	Embedded C Application	Branch Covered
7	Kim et al. [22]	Concolic Testing	CONBOL	Embedded Software	BC%, Time
8	Kim et al. [19, 25]	Distributed Concolic Testing	SCORE	Embedded C Program	BC%, Effectiveness
9	Sen et al. [26]	Concolic Testing, BC	CUTE, JCUTE	C and Java Programs	Test Cases, BC%, Time



Characteristics of different approaches

Table 4: Characteristics of different approaches on concolic testing.

Sl.No	Authors	Generated Test Cases	Measuring Coverage%	Determined Time Constraints	Computed Speed
1	Das et al. [16]	C	C	X	X
2	Bokil et al. [24]	C	X	C	X
3	Kim et al. [20]	C	X	X	X
4	Majumdar et al. [17]	C	X	X	X
5	Burnim et al. [21]	C	X	X	X
6	Kim et al. [23]	C	X	X	X
7	Kim et al. [22]	C	C	C	X



References » I

1. Palacios, M., Garc´ıa-Fanjul, J., Tuya, J. and Spanoudakis, G., 2015. Coverage-based testing for service level agreements. IEEE Transactions on Services Computing, 8(2), pages 299-313.
2. Fraser, G. and Arcuri, A., 2014. A large-scale evaluation of automated unit test generation using EvoSuite. ACM Transactions on Software Engineering and Methodology (TOSEM), 24(2), page 8.
3. Jones, JA. and Harrold, MJ. 2013. Test-suite reduction and prioritization for modified condition/decision coverage. IEEE Transactions on Software Engineering. 29(3), pages 195-209.
4. Baluda, M., Braione, P., Denaro, G. and Pezz`e, M., 2011. Enhancing structural software coverage by incrementally computing branch executability. Software Quality Journal, 19(4), pages 725-751.



References » II

5. Jiang B, Tse TH, Grieskamp W, Kicillof N, Cao Y, Li X, Chan WK. 2011. *Assuring the model evolution of protocol software specifications by regression testing process improvement*. Software: Practice and Experience. 41(10). pages 1073-1103.
6. McMinn, P. 2004. Search-based software test data generation: a survey: Research articles. *Software Testing, Verification and Reliability, 14(2)*, pages 105–156.
7. Harman M., Hu L., Hierons R., Wegener J., Sthamer H., Baresel A., and Roper M., 2004. Testability Transformation. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING*, 30(1), pages 1-14.
8. Wegener, J., Baresel, A. and Sthamer, H., 2001. Evolutionary test environment for automatic structural testing. Information and Software Technology, 43(14), pages 841-854.
9. Duran, J.W. and Ntafos, S.C., 1984. An evaluation of random testing. IEEE transactions on software engineering, (4), pages 438-444.



References » III

[10] Miller, W., and Spooner, DL. 1976. Automatic generation of floating-point test data. *IEEE Transactions on Software Engineering*, 2(3):223-226,

[11] Kuhn, DR. 1999. Fault classes and error detection capability of specification-based testing. *ACM Transactions on Software Engineering Methodology*, 8(4), pages 411-424.

[12] Ferguson, R. and Korel, B., 1996. The chaining approach for software test data generation. ACM Transactions on Software Engineering and Methodology (TOSEM), 5(1), pages. 63-86.

[13] DeMillo, R., and Offutt, J., 1993. Experimental results from an Automatic Test Case Generation. *ACM Transaction on Software Engineering Methodology*, 2(2), pages 109-175.

[14] Ntafos, S.C., 1988. A comparison of some structural testing strategies. IEEE Transactions on software engineering, 14(6), pages 868-874.

[15] Chilenski, J., and Miller. S. 1994 Application of Modified Condition/Decision Coverage to Software Testing. *Software Engineering Journal*, pages 193



References » IV

[16] Das A., and Mall R., 2013. Automatic Generation of MC/DC Test Data. *International Journal of Software Engineering, Acta Press* 2(1).

[17] Majumder R., and Sen K., 2007. Hybrid Concolic Testing. *Proceedings of the 29th International Conference on Software Engineering,IEEE Computer Society,Washington, DC, USA* pages 416-426.

[18] Hayhurst, KJ., Veerhusen DS., Chilenski, JJ., Rierson, LK. 2001. A Practical Tutorial on Modified Condition/Decision Coverage, NASA/TM-2001-210876.

[19] Kim Y., and Kim M., 2011. SCORE: a scalable concolic testing tool for reliable embedded software. *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering* pages 420-423.

[20] Kim, M., Kim, Y. and Choi, Y., 2012. Concolic testing of the multi-sector read operation for flash storage platform software. Formal Aspects of Computing, 24(3), pages 355-374.



References ≫ V

[21] Burnim J., and Sen K., 2008. Heuristics for scalable dynamic test generation. *In Proc. ASE*, pages 443-446, Washington, D.C., USA.

[22] Kim Y., Kim Y., Kim T., Lee G., Jang Y., and Kim M., 2013. Automated unit testing of large industrial embedded software using concolic testing. *IEEE/ACM 28th International Conference on Automated Software Engineering (ASE)* pages 519-528.

[23] Kim M., and Kim Y.,and Jang Y., 2012. Industrial application of concolic testing on embedded software: Case studies. *IEEE Fifth International Conference on Software Testing, Verification and Validation (ICST)*, pages 390-399.

[24] Bokil, P., Darke, P., Shrotri, U., and Venkatesh, R., 2009. Automatic Test Data Generation for C Programs. *In proceedings 3rd IEEE International Conference on Secure Software Integration and Reliability Improvement*.



References ≫ VI

[25] Kim M., Kim Y., and Rothermel G., 2012. A Scalable Distributed Concolic Testing Approach: An Empirical Evaluation. *IEEE Fifth International Conference on Software Testing, Verification and Validation (ICST)*, pages 340-349.

[26] Sen K., and Agha G., 2006. CUTE and jCUTE: Concolic Unit Testing and Explicit Path Model-Checking Tools (Tools Paper). *DTIC Document*.




Thank
You!

Data Flow Testing

Dr. Durga Prasad Mohapatra
Professor
CSE Department
NIT Rourkela

Introduction cont…

- Data flow testing is a white-box technique that can be used to detect improper use of data values due to coding errors.
- Errors may be unintentionally introduced in a program by programmers,
  - e.g. a programmer might use a variable without defining it.
- Data flow testing gives a chance to look out for
  - inappropriate data definition, their use in predicate, computations, and termination.

Introduction cont…

- It identifies the potential bugs
  - by examining the patterns in which that piece of data is used.

- Example: If an out-of-scope data is being used in a computation, then it is a bug. There may be several patterns like this which indicate data anomalies.

Introduction cont…

- To examine the patterns, the control flow graph of a program is used.
- This test strategy selects the paths in the module's control flow such that various sequences of data objects can be chosen.
- The major focus is on the points at which the data receives values and the places at which the data initialized has been referenced.
- Thus, we have to choose enough paths in the control flow to ensure that every data is initialized before use and all the defined data have been used somewhere.

Data Flow-Based Testing

- Selects test paths of a program:
  - According to the locations of
    - Definitions and Uses of different variables in a program.

Example

```
1   X(){
2    int a=5; /* Defines variable a */
        ….
3    While(c>5) {
4     if (d<50)
5           b=a*a;   /*Uses variable a */
6           a=a-1; /* Defines variable a */
…
7     }
8    print(a); } /*Uses variable a */
```

Data Flow-Based Testing cont …

- For a statement numbered S,
  - DEF(S) = {X / statement S contains a definition of X}
  - USES(S)= {X / statement S contains a use of X}
  - Example: **1: a=b;** DEF(1)={a}, USES(1)={b}.
  - Example: 2: **a=a+b;** DEF(1)={a}, USES(1)={a,b}.

Data Flow-Based Testing cont …

- A variable X is said to be live at statement S1, if
  - X is defined at a statement S, and
  - there exists a path from S to S1 not containing any definition of X.

DU Chain Example

```
1 X(){
2  int a=5; /* Defines variable a */
3  While(c>5) {
4     if (d<50)
5           b=a*a;   /*Uses variable a */
6           a=a-1; /* Defines variable a */
7     }
8   print(a); } /*Uses variable a */
```

Definition-use chain (DU chain)

- [X,S,S1],
  - S and S1 are statement numbers,
  - X ∈ DEF(S),
  - X ∈ USES(S1), and
  - the definition of X in the statement S is live at statement S1.

Data Flow-Based Testing Strategy

- One simple data flow testing strategy:
  - **Every DU chain in a program be covered at least once.**
- Data flow testing strategies:
  - Useful for selecting test paths of a program containing nested if and loop statements.

Example

```
1 X(){
2  B1;      /* Defines variable a */
3  While(C1) {
4     if (C2)
5           if(C4) B4; /*Uses variable a */
6       else B5;
7           else  if (C3) B2;
8       else B3;     }
9  B6 }
```

Example cont …

- [a,1,5]: a DU chain.
- Assume:
  - DEF(X) = {B1, B2, B3, B4, B5}
  - USES(X) = {B2, B3, B4, B5, B6}
  - There are 25 DU chains.
- However only 5 paths are needed to cover these chains.

Data Flow Testing cont…

- It also closely examines the state of the data in the CFG resulting in a richer test suite
  - than the one obtained from CFG based path testing strategies like statement coverage, branch coverage, etc.

States of a Data Object

- Defined (d):
- Killed / Undefined / Released (k):
- Usage (u):
- Computational use (c-use) or
- Predicate use (p-use).

State of a Data Object cont …

A data object can be in the following states:

- ***Defined (d)*** A data object is called defined when it is initialized, i.e., when it is on the left side of an assignment statement. Defined state can also be used to mean that a file has been opened, a dynamically allocated object has been allocated, something is pushed onto the stack, a record written, and so on.

State of a Data Object cont…

- ***Kill/Undefined/Released (k)***

- When the data has been reinitialized or the scope of a loop control variable finishes, i.e., exiting the loop or memory is released dynamically or a file has been closed.

State of a Data Object cont…

- ***Usage (u)*** When the data object is on the right side of assignment or the control variable in a loop, or in an expression used to evaluate the control flow of a case statement, or as a pointer to an object, etc.
- In general, we say that the usage is either computational use (c-use) or predicate use (p-use).

Data-Flow Anomalies

- Data-flow anomalies represent the patterns of data usage which may lead to an incorrect execution of the code.

- An anomaly is denoted by a two-character sequence of actions.

Data-Flow Anomalies cont…

- Example: ‘dk’ means a variable is defined and killed without any use, which is a potential bug.

- There are nine possible two-character combinations out of which only four are data anomalies, as shown in next Table.

Table 1: Two-character data-flow anomalies

Anomaly	Explanation	Effect of Anomaly
du	Define-use	Allowed, Normal case.
dk	Define-Kill	Potential bug. Data is killed without use after definition.
ud	Use-define	Data is used and then redefine. Allowed, Usually not a bug because the language permits reassignment at almost any time.
uk	Use-Kill	Allowed, Normal situation.
ku	Kill-use	Serious bug because the data is used after being killed.
kd	Kill-define	Data is Killed and then redefined, Allowed
dd	Define-define	Redefining a variable without using it . Harmless bug, but not allowed.
uu	Use-use	Allowed Normal case.
kk	Kill-kill	Harmless bug, but not allowed.

- Not all data-flow anomalies are harmful, but most of them are suspicious and indicate that an error can occur.
- There may be single-character data anomalies also.
- To represent these types of anomalies, we take the following conventions:

  - ~x: indicates all prior actions are not of interest to x.
  - x~: indicates all post actions are not listed of interest to x.

Table 2: Single-character data-flow anomalies

Anomaly	explanation	Effect of Anomaly
~d	First definition	Normal situation, Allowed.
~u	First Use	Data is used without defining it. Potential bug.
~k	First Kill	Data is killed before defining it, Potential bug.
D~	Define last	Potential bug.
U~	Use last	Normal case, Allowed.
K~	Kill last	Normal case, Allowed.

Some Terminologies

Suppose P is a program that has a graph G (P) and a set of variables V. The graph has a single entry and exit node.

- ***Definition node*** Defining a variable means assigning value to a variable for the very first time in a program. For example, input statements, assignment statements, loop control statements, procedure calls, etc.

Some Terminologies contd…

- ***Usage node*** It means the variable has been used in some statement of the program. Node n that belongs to G(p) is usage node of variable *v*, if the value of variable v is used at the statements corresponding to node n.

Some Terminologies contd…

- A usage node can be of the following two types:

1) Predicate usage Node: If usage node n is a predicate node, then **n** is a predicate usage node.
2) Computation Usage Node: If usage node **n** corresponds to a computation statement in a program other than predicate, then it is called a computation usage node.

Some Terminologies contd…

- ***Loop-free path segment*** It is a path segment for which every node is visited once at most.

- ***Simple path segment*** It is a path segment in which at most one node is visited twice. A simple path segment is either loop-free or if there is a loop, only one node is involved.

- ***Definition-use path (du-path)*** A du-path with respect to a variable v is a path between the definition node and usage node of that variable, Usage node can either be a p-usage or a c-usage node.

Some Terminologies contd…

- ***Definition-clear path (dc-path)*** A dc-path with respect to a variable v is a path between the definition node and the usage node such that no other node in the path is a defining node of variable *v*.

- The du paths which are not dc paths are important, as these are potential spots for testing persons.

- Those du-paths which are definition-clear are easy to test in comparison to du-paths which are not dc-paths.

- The du-paths which are not dc-paths need more attention.

Static Data Flow Testing

With static analysis, the source code is analysed without executing it.

EXAMPLE:

Consider a program for calculating the gross salary of an employee in an organization. If his basic salary < 1500, then HRA=10% of the Basic and DA=90% of basic. If his salary >= 1500, then HRA=500 and DA=98% of basic. Calculate the gross salary.

```
main()
{
1.  float bs, gs, da, hra=0;
2.  printf("Enter basic salary");
3.  scanf("%f",&bs);
4.  if(bs < 1500)
5.  {
6.      hra=bs * 10/100;
7.      da= bs * 90/100;
8.  }
9.  else
10. {
11.     hra = 500;
12.     da= bs * 98/100;
13. }
14.  gs= bs+ hra+ da;
15.  printf("Gross Salary =  Rs. %f", gs);
16. }
```

Find out the define-use-kill patterns for all the variables in the program

Solution

Pattern	Line Number	Explanation
~d	3	Normal case. Allowed
du	3-4	Normal case. Allowed
uu	4-6,6-7,7-12,12-14	Normal case. Allowed
uk	14-16 Define-use-kill patterns for variable 'bs'	Normal case. Allowed
K~	16	Normal case. Allowed

Solution cont…

Pattern	Line Number	Explanation
~d	14	Normal case.Allowed
du	14-15	Normal case.Allowed
uk	15-16	Normal case.Allowed
K~	16	Normal case.Allowed

Define-use-kill patterns for variable 'gs'

Solution cont…

Pattern	Line Number	Explanation
~d	7	Normal case. Allowed
du	7-14	Normal case. Allowed
uk	14-16	Normal case. Allowed
K~	16	Normal case. Allowed

Define-use-kill patterns for variable ‘da’

Solution cont…

Pattern	Line Number	Explanation
~d	1	Normal case. Allowed
dd	1-6 or 1-11	Double definition. Not allowed. Harmless bug.
du	6-14 or 11-14	Normal case. Allowed
uk	14-16	Normal case. Allowed
K~	16	Normal case. Allowed

Define-use-kill patterns for variable 'hra'

From the above static data flow testing, only one bug is found, i.e in variable HRA of double definition.

Summary

- Discussed the basic concepts of data flow testing.
- Explained DU Chain.
- Presented the different states of a data object.
- Explained the different data-flow anomalies.
- Explained static data flow testing with an example.

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 5), Second Edition, Oxford University Press, 2016.

Thank you

Data Flow Testing cont…

Dr. Durga Prasad Mohapatra
Professor
CSE Department
NIT Rourkela

Static Analysis is not Enough

- All anomalies using static analysis cannot be determined and this is an unsolvable problem.

- For example, if the variable is used in an array as the index, we cannot determine its state by static analysis.

- Or it may be the case that the index is generated dynamically during execution, therefore we cannot guarantee what the state of the array element is referenced by that index.

Dynamic Data Flow Testing

- The test cases are designed in such a way that every definition of data variable to each of its use is traced to each of its definition.
- Various strategies are employed for the creation of test cases.
- All these strategies are defined below.

Dynamic Data Flow Testing cont…

- ***All-du Paths (ADUP)*** It states that every definition of every variable to every use of that definition should be exercised under some test. It is the strongest data flow testing strategies.

Dynamic Data Flow Testing cont…

- ***All-uses (AU)*** This states that for every use of the variable, there is a path from the definition of that variable (nearest to the use in backward direction) to the use.

Dynamic Data Flow Testing cont…

- ***All-p-uses/Some-c-uses (APU + C)*** This strategy states that for every variable and every definition of that variable, include at least one dc-path from the definition to every predicate use. If there are definitions of the variable with no p-use following it, then add computational use (c-use) test cases as required to cover every definition.
- Note: A dc-path (definition-clear path) with respect to a variable v is a path between the definition node & the usage node s.t. that no other node in the path is a defining node of variable *v.*

Dynamic Data Flow Testing cont…

- ***All-c-uses/Some-p-uses (ACU + P)*** This strategy states that for every variable and every definition of that variable, include at least one dc-path from the definition to every computational use. If there are definitions of the variable with no c-use following it, then add predicate use (p-use) test cases as required to cover every definition.

Dynamic Data Flow Testing cont…

- ***All-Predicate-uses (APU)*** It is derived from the APU + C strategy and states that for every variable, there is a path from every definition to every p-use of that definition. If there is a definition with no p-use following it, then it is dropped from contention.

Dynamic Data Flow Testing cont…

- ***All-Computational-Uses (ACU)*** It is derived from the strategy ACU + P strategy and states that for every variable, there is a path form every definition to every c-use of that definition. If there is a definition with no c-use following if, then it is dropped form contention.

Dynamic Data Flow Testing cont…

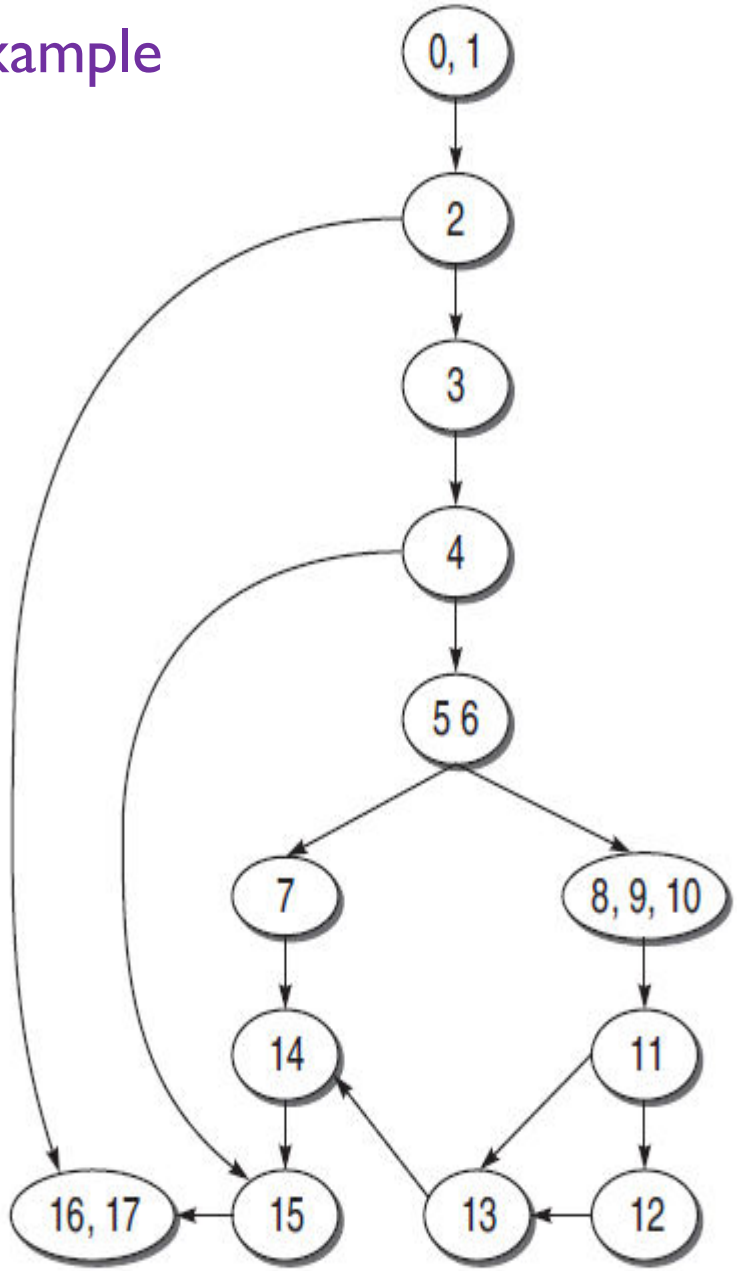
- ***All-Definition (AD)*** It states that every definition of every variable should be covered by at least one use of that variable, be that a computational use or a predicate use.

Example

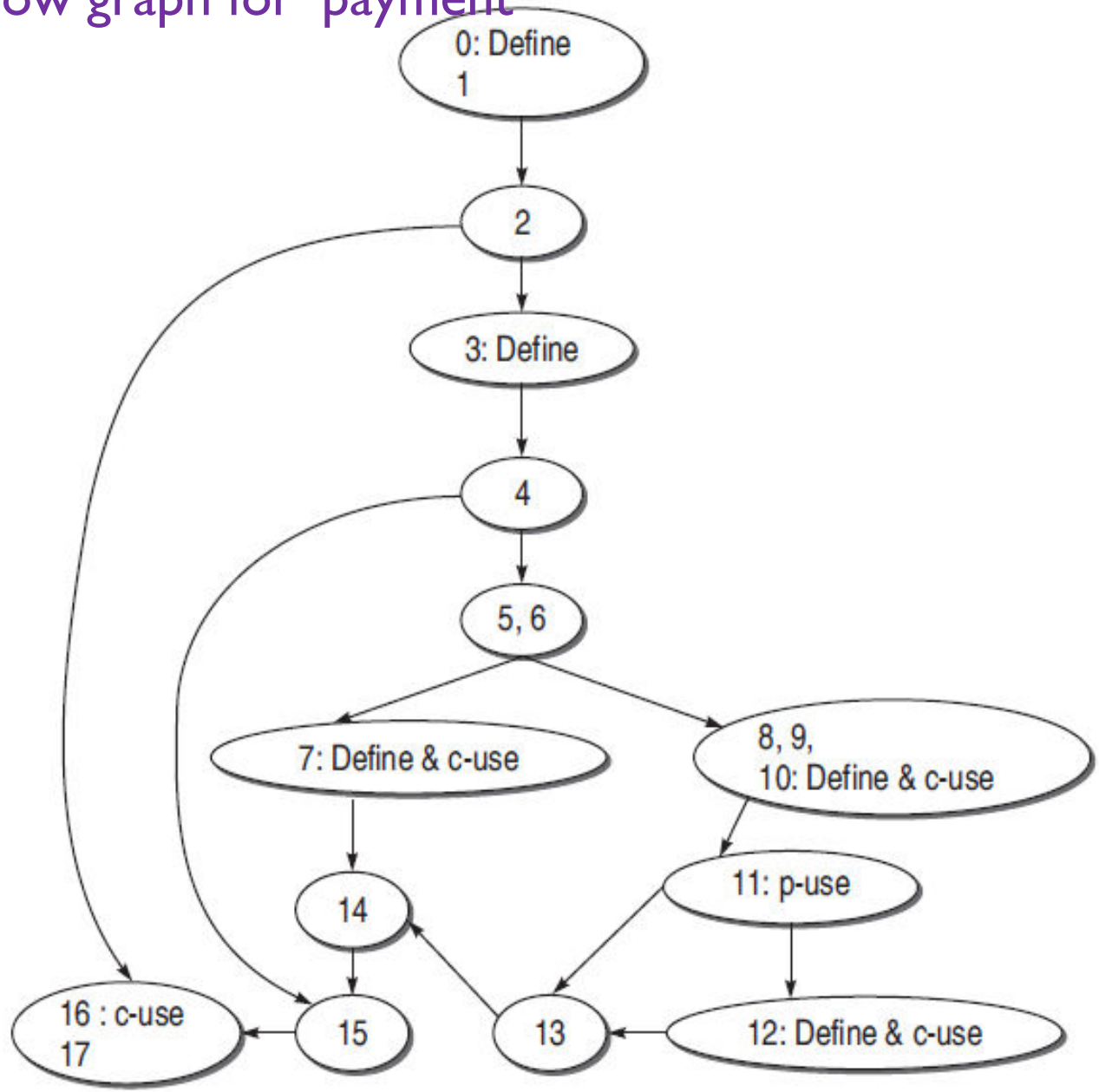
- Consider a program given below. Draw its control flow graph and data flow graph for each variable used in the program, and derive data flow testing paths with all the strategies discussed above.

```
main()
 {
   int work;
0.   double payment =0;
1.   scanf("%d", work);
2.    if (work > 0) {
3.    payment = 40;
4.   if (work > 20)
5.    {
6.    if(work <= 30)
7.    payment = payment + (work – 25) * 0.5;
8.     else
9.      {
10.      payment = payment + 50 + (work –30) * 0.1;
11.     if (payment >= 3000)
12.     payment = payment * 0.9;
13.    }
14.     }
15.   }
16.     printf("Final payment", payment);
```

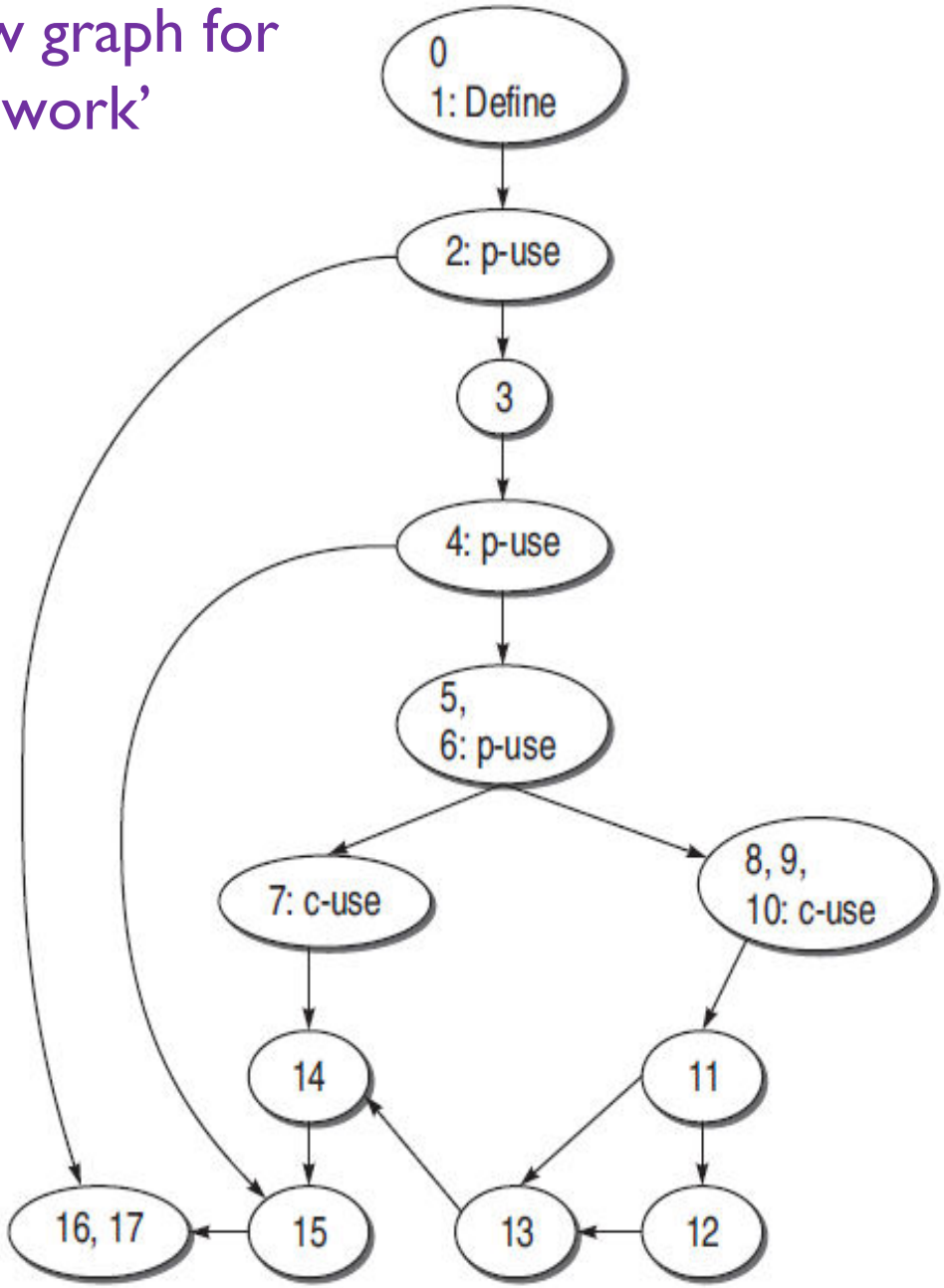
CFG for the Example



Data flow graph for 'payment'



Data flow graph for variable 'work'



- Prepare a list of all the definition nodes and usage nodes for all the variables in the program.

Variable	Defined At	Used At
Payment	0,3,7,10,12	7,10,11,12,16
Work	1	2,4,6,7,10

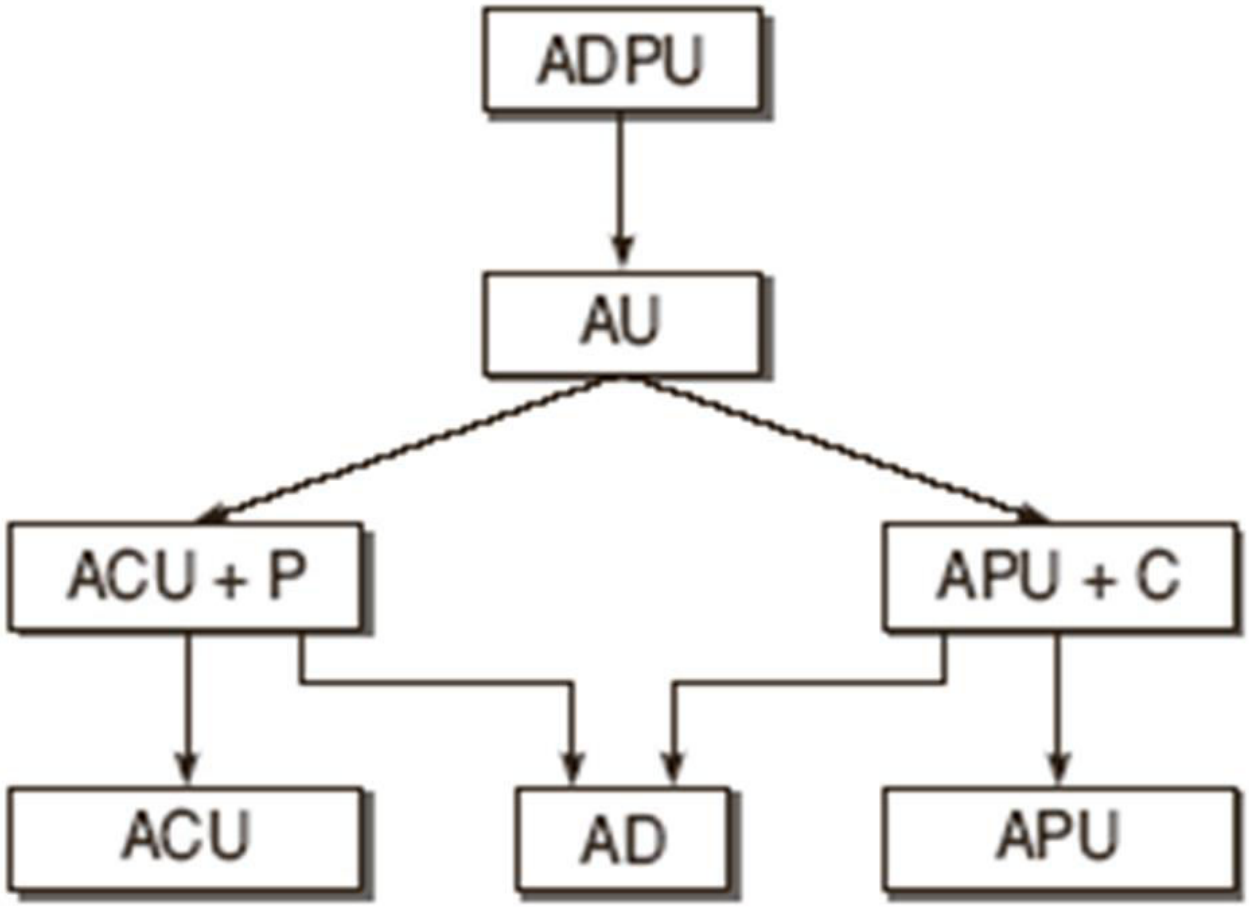
Strategy	Payment	Work
All Uses(AU)	3-4-5-6-7	1-2
	10-11	1-2-3-4
	10-11-12	1-2-3-4-5-6
	12-13-14-15-16	1-2-3-4-5-6-7
	3-4-5-6-8-9-10	1-2-3-4-5-6-8-9-10
All p-uses(APU)	0-1-2-3-4-5-6-8-9-10-11	1-2
		1-2-3-4
		1-2-3-4-5-6
All c-uses(ACU)	0-1-2-16	1-2-3-4-5-6-7
	3-4-5-6-7	1-2-3-4-5-6-8-9-10
	3-4-5-6-8-9-10	
	3-4-15-16	
	7-14-15-16	
	10-11-12	
	10-11-13-14-15-16	
	12-13-14-15-16	

Strategy	Payment	Work
All-p-uses / Some-c-uses (APU + C)	0-1-2-3-4-5-6-8-9-10-11	1-2
	10-11-12	1-2-3-4
	12-13-14-15-16	1-2-3-4-5-6
		1-2-3-4-5-6-8-9-10
All-c-uses / Some-p-uses (ACU + P)	0-1-2-16	1-2-3-4-5-6-7
	3-4-5-6-7	1-2-3-4-5-6-8-9-10
	3-4-5-6-8-9-10	1-2-3-4-5-6
	3-4-15-16	
	7-14-15-16	
	10-11-12	
	10-11-13-14-15-16	
	12-13-14-15-16	
	0-1-2-3-4-5-6-8-9-10-11	

Strategy	Payment	Work
All-du-paths (ADUP)	0-1-2-3-4-5-6-8-9-10-11	1-2
	0-1-2-16	1-2-3-4
	3-4-5-6-7	1-2-3-4-5-6
	3-4-5-6-8-9-10	1-2-3-4-5-6-7
	3-4-15-16	1-2-3-4-5-6-8-9-10
	7-14-15-16	
	10-11-12	
	10-11-13-14-15-16	
	12-13-14-15-16	
All Definitions (AD)	0-1-2-16	1-2
	3-4-5-6-7	
	7-14-15-16	
	10-11	
	12-13-14-15-16	

Ordering of data flow testing strategies

- While selecting a test case, we need to analyse the relative strengths of various data flow testing strategies.



Ordering of data flow testing strategies cont…

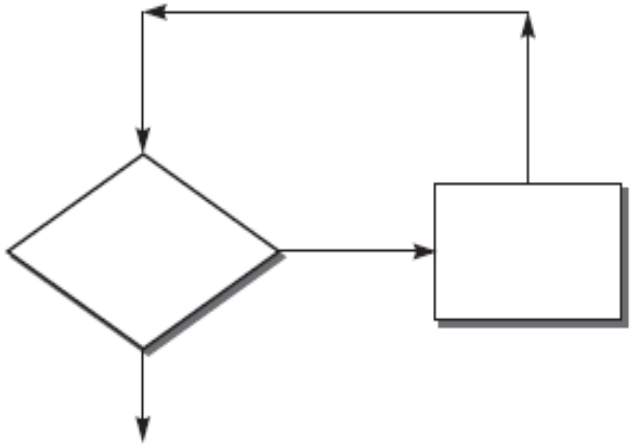
- The previous Figure depicts the relative strength of the data flow strategies.
- In this figure, the relative strength of testing strategies reduces along the direction of the arrow.
- It means that all-du-paths (ADUP) is the strongest criterion for selecting the test cases.

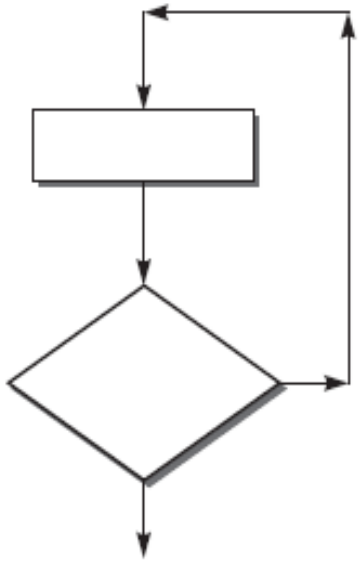
Loop testing

- Loop testing can be viewed as an extension to branch coverage.
- Loops are important in the software from the testing view point. If loops are not tested properly, bugs can go undetected.
- Loop testing can be done effectively while performing development testing (unit testing by the developer) on a module.
- Sufficient test cases should be designed to test every loop thoroughly.
- There are four different kinds of loops. How each kind of loop is tested, is discussed below.

Simple loops

- Simple loops mean, we have a single loop in the flow, as shown below.





Testing Simple Loops

- Check whether you can bypass the loop or not. If the test case for bypassing the loop is executed and, still you enter inside the loop, it means there is a bug.
- Check whether the loop control variable is negative.
- Write one test case that executes the statements inside the loop.

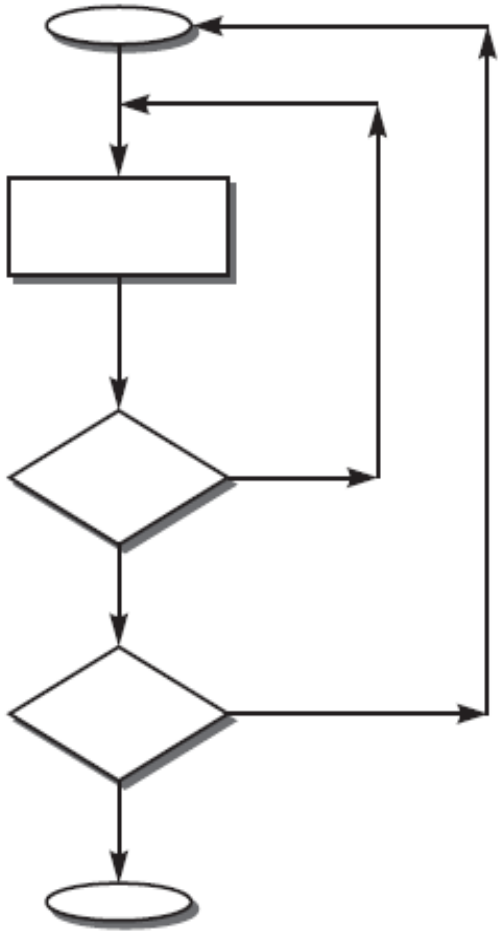
Testing Simple Loops cont …

- Write test cases for a typical number of iterations through the loop.
- Write test cases for checking the boundary values of the maximum and minimum number of iterations defined (say max and min) in the loop. It means we should test for min, min+1, min−1, max−1, max, and max+1 number of iterations through the loop.

Nested loops

- When two or more loops are embedded, it is called a nested loop, as shown in next slide. If we have nested loops in the program, it becomes difficult to test.

Nested loops



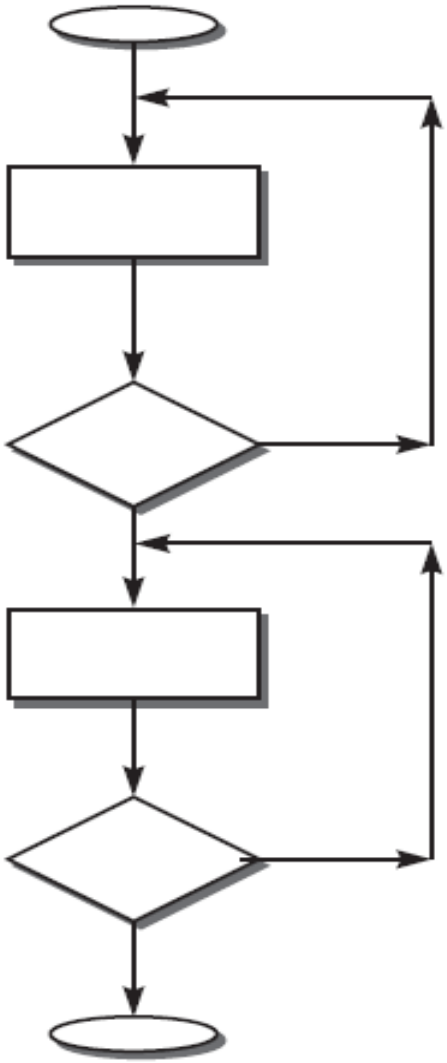
Testing Nested Loops

- If we adopt the approach of simple tests to test the nested loops, then the number of possible test cases grows geometrically.
- Thus, the strategy is to start with the innermost loops while holding outer loops to their minimum values. Continue this outward in this manner until all loops have been covered

Concatenated loops

- The loops in a program may be concatenated (shown in next slide).
- Two loops are concatenated if it is possible to reach one, after exiting the other, while still on a path from entry to exit.
- If the two loops are not on the same path, then they are not concatenated.
- The two loops on the same path may or may not be independent.
- If the loop control variable for one loop is used for another loop, then they are concatenated, but nested loops should be treated like nested only.

Concatenated loops



Testing Concatenated Loops

- The concatenated loop may be treated as a sequence of two or more numbers of simple loops.
- So, the strategy for testing of simple loops may be extended to testing of concatenated loops.

Unstructured loops

- This type of loops is really impractical to test.
- They must be redesigned or at least converted into simple or concatenated loops.

Summary

- Explained dynamic data flow testing strategies with an example.
- Presented the ordering of different data flow testing strategies.
- Discussed different loop testing strategies.

References

1. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 5), Second Edition, Oxford University Press, 2016.

Thank you

Static Testing

Drawbacks of Dynamic Testing

- Uncovers the bug at a later stage of SDLC
  - Costly to debug
- It is expensive and time consuming, as it needs to create, run, validate and maintain test cases.
- dynamic testing can miss a bug because test cases are incomplete, not because dynamic testing is incapable of finding the bug

Drawbacks of Dynamic Testing cont…

- The efficiency of code coverage decreases with the increase in size of the system.
- It provides information about the bugs-
  - It is difficult and time-consuming to trace a failure from a test case back to its root cause.

- Dynamic testing cannot detect all the potential bugs.

Static Testing

- Static testing is a complimentary to dynamic testing technique to acquire high quality software.
- Static testing techniques do not execute the software and do not require the bulk of test cases.
- Static testing reveals the errors which are not shown by dynamic testing.
- This type of testing is also known as non-computer based testing or human testing.

Static Testing cont…

- Static testing can be applied for most of the verification activities.
- They check the software product at each SDLC stage for conformance with the required specification or standards.
- Static testing can identify defects in requirements, design, and code before execution, while dynamic testing identifies failures by executing the software.
- This is why static and dynamic testing complement each other rather than replace each other.

Static Testing cont…

- Requirements, design specifications, test plans, source code, user's manuals, maintenance procedures are some of the items that can be statically tested.
- Static testing has proved to be a cost-effective technique of error detection.
  - 60% of errors in a program can be detected

Benefits of Static Testing

- As defects are found and fixed, the quality of the product increases.
- A more technically correct base is available for each new phase of development.
- Overall software life cycle cost becomes lower.
- Effectiveness of dynamic test activity is increased & less time needs to be devoted for testing the product.
- Immediate evaluation and feedback about quality improvement is available.

Objectives of Static Testing

- To identify errors in any phase of SDLC as early as possible.
- To verify that the components of software are in conformance with its requirements.
- To provide information for project monitoring.
- To improve the software quality and increase productivity.

Types of Static Testing

- Software Inspections
- Walkthroughs
- Technical reviews

Types of Static Testing cont …

- After a module has been coded:
  - code inspection and code walk through are carried out
  - ensures that coding standards are followed
  - helps detect as many errors as possible before testing.

Types of Static Testing cont …

- Detect as many errors as possible during inspection & walkthrough:
  - detected errors require less effort for correction
    - much higher effort needed if errors were to be detected during integration or system testing.

Inspections

- Software inspections were first introduced at IBM by Fagan in the early 1970s.
- These can be used to tackle software quality problems because they allow the detection and removal of defects after each phase of SDLC.

Inspection

- Code inspection aims mainly at discovery of commonly made errors.
- During code inspection:
  - the code is examined for the presence of certain kinds of errors.

Inspection

- For instance, consider:
  - classical error of writing a procedure that modifies a formal parameter
  - while the calling routine calls the procedure with a constant actual parameter.
- It is more likely that such an error will be discovered:
  - by looking for this kind of mistakes in the code,
  - rather than by simply hand simulating execution of the procedure.

Inspection

- Good software development companies:
  - collect statistics of errors committed by their engineers
  - identify the types of errors most frequently committed.
- A list of common errors:
  - can be used during code inspection to look out for possible errors.

Commonly made errors

- Use of uninitialized variables.
- Nonterminating loops.
- Array indices out of bounds.
- Incompatible assignments.
- Improper storage allocation and deallocation.
- Actual and formal parameter mismatch in procedure calls.
- Jumps into loops.

Inspections cont…

- Inspection process is an in-process manual examination of an item to detect bugs.
- It may be applied to any product or partial product of the software development process.
- These are done in the early stages of each product development.

Inspections cont…

- This process does not require executable code or test cases
  - With inspection, bugs can be found on infrequently executed paths that are not likely to be included in test cases.
- It does not execute the code, so
  - It is machine independent
  - Requires no target system resources
  - Does not require to change program's operational behavior.

Inspections cont…

- The inspection process is carried out by a group of peers.
- The group first inspects the product at individual level.
- After this, they discuss the potential defects of the product observed in a formal meeting.

Inspections cont…

- The second important thing about inspection process is that it is a formal process of verifying a software product.
- The documents which can be inspected are SRS, SDD, Code, and test plan.

Inspections cont…

- An inspection process involves the interaction of the following elements:
  - Inspection steps,
  - Role for participants,
  - Item being inspected.

Inspections cont…

- The entry and exit criteria are used to determine whether an item is ready to be inspected.
  - Entry criteria mean that the item to be inspected is matured enough to be used.
  - Exit criterion is that once the item has been given for inspection, it should not be updated,
    - Otherwise it will not know how many bugs have been reported and corrected and the whole purpose of inspection is lost.

Inspection team

- For the inspection process, a minimum of the following four team members are required:
  - Author/Owner/Producer
    - Is a programmer or designer responsible for producing the program or documents.
    - He is also responsible for fixing defects discovered during the inspection process.

Inspection team cont…

- Inspector
  - A peer member of the team.
  - He is not directly related to the product under inspection and may be concerned with some other products.
  - He finds errors, omissions, and inconsistencies in programs and documents.

Inspection team cont…

- Moderator
  - A team member who manages the whole inspection process.
  - He schedules, leads and controls the inspection session.
  - He is the key person with the responsibility of planning and successful execution of the inspection.

Inspection team cont…

- Recorder

  Who records all the results of the inspection meeting.

Inspection process

- A general inspection process has the following stages:
  - Planning
  - Overview
  - Individual preparation
  - Inspection meeting
  - Rework
  - Follow-up

Inspection process cont…

- Planning:

  During this phase the following is executed:

  - The product to be inspected is identified.
  - A moderator is assigned
  - The objective of the inspection is stated.

Inspection process cont…

During planning the moderator performs the following activities

- Assures that the product is ready for inspection
- Selects the inspection team and assigns their roles.
- Schedules the meeting venue and time.
- Distributes the inspection materials like the item to be inspected, check lists, etc.

Inspection process cont…

- Overview:
  - The inspection team is provided with the background information for inspection.
  - The author presents the rationale for the product, its relationship to the rest of the product being developed, its function and intended use, and the approach used to develop it.
  - The opening meeting may also be called by the moderator to explain the objective of inspection to the team members

Inspection process cont…

- Individual preparation:
  - After the overview, the reviewers individually prepare themselves for the inspection.
  - They point out potential errors or problems found and record them in a log.
  - The moderator compiles the logs of different members and gives a copy to the author of the inspected team.
  - The inspector reviews the product for general problems as well as specific problems.

Inspection process cont…

- The product being inspected is also checked against standard documents to assure correctness.
- After reviewing, the inspector records the defects found and the time spent for preparation.
- Completed preparation logs are then submitted to the moderator prior to the inspection meeting.

Inspection process cont…

- The moderator reviews the logs to determine whether the team is adequately prepared.
- Moderator also checks for trouble spots and the areas of major concern that may require extra attention .
- If the team is not adequately prepared then the meeting is rescheduled.
- After this, the compiled log file is submitted to the author.

Inspection process cont…

- Inspection meeting:
  - The inspection meeting starts with the author of the inspected team.
  - Author discusses every issue raised by different members in the compiled log file.
  - They arrive at a consensus to accept the concerns as errors or not.
  - If a new error is found during the meeting then it is also recorded and discussed.

Inspection process cont…

- During the meeting effort is put only to identify the bug and not to fix them.
- The author later fixes the identified bugs.
- It is the duty of the moderator that the meeting remains focused towards its objective and the author is not discouraged in any way.
- At the end the moderator concludes the meeting and produces the summary of the inspection meeting.

Inspection process cont…

- Rework:
  - The summary list of the bugs that arise during the inspection meeting needs to be reworked by the author.
  - The author fixes all these bugs and reports back to the moderator.

Inspection process cont…

- Follow-up:
  - Moderator checks that all the bugs found have been addressed and fixed.
  - He prepares a report, the document is then approved for release.
  - If this is not the case, then the unresolved issues are mentioned in a report and another inspection meeting is called.

Benefits of Inspection process

- Bug reduction
- Bug prevention
- Productivity
- Real-time feedback to software engineers
- Reduction in development resource
- Quality improvement
- Project management
- Checking coupling and cohesion
- Learning through inspection
- Process improvement
  - Finding most error prone modules
  - Distribution of error types

Effectiveness of inspection process

- The effectiveness of the inspection process lies in the rate of inspection
- Refers to how much evaluation of an item has been done.
  - Rate is high
    - Coverage of item to be evaluated is high.
  - Rate is slow
    - Means coverage is not much.
- Rate should be considered in the perspective of detection of errors
  - Too high means less errors detected, and too slow increases the cost of project.

Effectiveness of inspection process cont…

- It may be calculated as the error detection efficiency of the inspection process, as given below:

  Error detection efficiency =

$$\frac{Errorfoundbyaninspection}{totalerror\sin theitembeforeinspection}\times 100$$

- It also depends on the experience of the team, programming language and the application domain.

Cost of Inspection

- With at least four members involved the cost of inspecting 100 lines of code is roughly equivalent to one person-day of effort.
  - Assuming that inspection takes 1 hr &
  - Each member spends1-2hrs preparation
- Testing costs are variable and depend on number of faults in the software.
- Effort required for inspection is half the effort that would be required for dynamic testing.
- It is estimated that, the cost of inspection can be 5-10%of the total cost of the project.

Thank You

Static Testing cont…

Variants of inspection process

Active Design Reviews (ADR)

- Several reviews are conducted targeting a particular type of bug and conducted by the experts in that area.
- These are conducted in the following stages:
  - Overview - The overview explains the modular structure in case it is unfamiliar to reviewers, and shows them where the module belongs in the structure.
  - Review- Reviewers are assigned sections of the document to be reviewed and questionnaires based on the bug type.
  - Meeting - The designers read the completed questionnaires and meet the reviewers to resolve any queries that the designers may have about the reviewer's answers to the questionnaires.

FTArm

- Stands for Formal Technical Asynchronous review method (FTArm)
  - It is a type of asynchronous inspection process that is carried out without having a meeting of the members.
  - This process consists of the following steps
    - Setup (choosing the members and preparing the document)
    - Orientation (same as overview step of inspection process)
    - Private review (same as preparation phase of inspection process)
    - Public review (all comments provided privately are made public and all inspectors are able to see each other's comments and put forward their suggestions.)
    - Consolidation (the moderator analyses the result of private and public reviews and lists the findings)
    - Group meeting (any unresolved issues are discussed in this step)
    - Conclusion (final report of the inspection with analysis is produced)

Gilb inspection

- Defect detection is carried out by individual inspector at his level rather than in a group.
- Three different roles identified:
  - Leader is responsible for planning and running the inspection
  - Author of the document
  - Checker is responsible for finding and reporting the defects in the document.

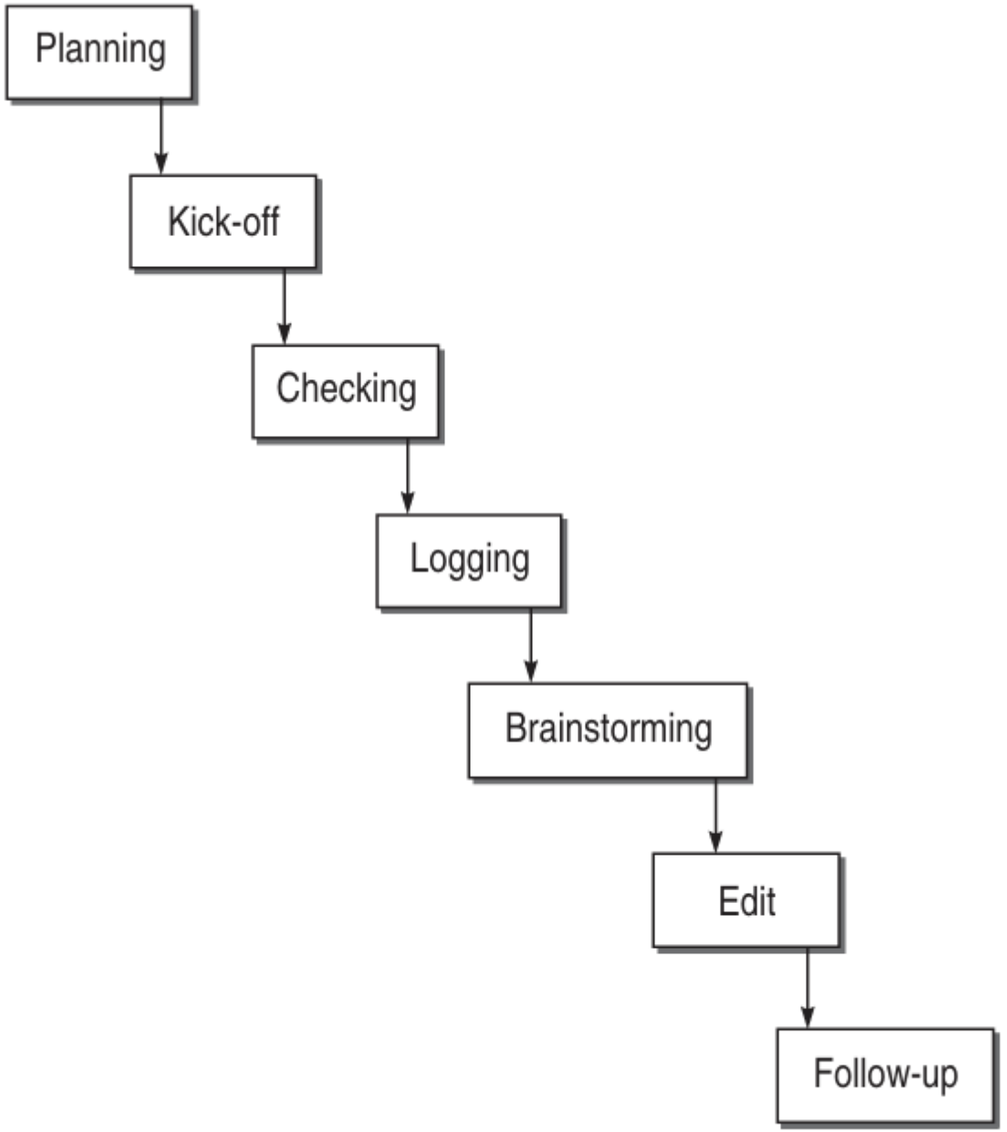


Figure 6.4 Gilb inspection process

Steps of Gilb inspection

- **Entry -** The document must pass through an entry criteria so that the inspection time is not wasted on a document which is fundamentally flawed.
- **Planning -**The leader determines the inspection participants and schedules the meeting.
- **Kick-off -** The relevant documents are distributed, participants are assigned roles and briefed about the agenda of the meeting.
- **Checking -** Each checker works individually and finds defects.

Steps of Gilb inspection cont …

- **Logging -** Potential defects are collected and logged.
- **Brainstorming -** In this stage, process improvement suggestions are recorded based on the reported bugs.
- **Edit -** After all the defects have been reported, the author takes the list and works accordingly.
- **Follow-up -** The leader ensures that the edit phase has been executed properly.
- **Exit -** The inspection must pass the exit criteria as fixed for the completion of the inspection process.

Humphrey's inspection

- Preparation phase emphasizes the finding and logging of bugs, unlike Fagan inspection.
- Includes an analysis phase wherein individual logs are analyzed & combined into a single list.
- The steps are:
  - Planning
  - Overview
  - Preparation
  - Analysis
  - Inspection
  - Rework
  - Follow-up

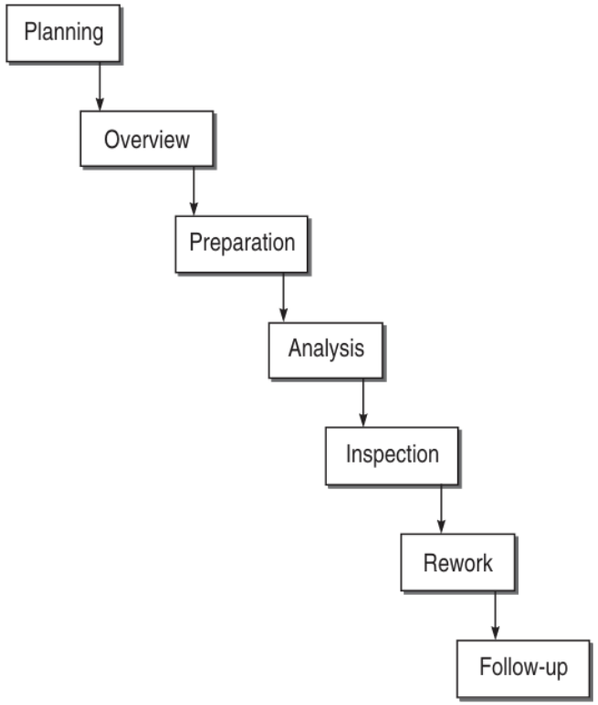


Figure 6.5 Humphrey's process

N-Fold inspections

- Inspection process's effectiveness can be increased by replicating it by having multiple inspection teams.
- This process consists of the following stages:
  - Planning and overview
  - Inspection stages
  - Collation phase
  - Rework and follow-up

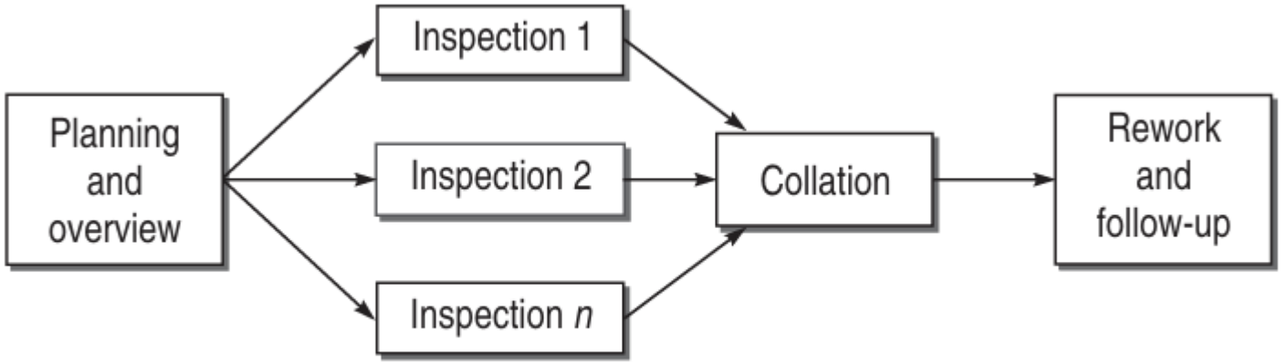


Figure 6.6 N-fold inspection

Phased inspection

- Phased inspections are designed to verify the product in a particular domain by experts in that domain only.
- There are two types of phases:
  - Single inspector
    - A rigorous checklist is used to verify the features specified.
  - Multiple inspector
    - Checklist cannot be used.
    - Each inspector prepares their self developed questionnaire.

Reading techniques

- Reading technique can be defined as a series of steps whose purpose is to guide an inspector to acquire a deep understanding of the inspected product.

- It can be regarded as a mechanism or strategy for the individual inspector to detect defects in the inspected product.

Reading techniques cont …

- The various reading techniques are:
  - Adhoc method
    - No direction or guidelines provided for inspection
  - Checklists
  - Scenario-based reading
  - Perspective-based reading
  - Usage-based reading
  - Abstraction driven reading
  - Task driven reading
  - Function-point based scenarios

Reading techniques

- Checklists
  - Are in the form of questions and are used by the inspection team members to verify the item.
  - Drawbacks:
    - The questions are not sufficiently tailored to take into account a particular development environment.
    - Instructions for using them are missing.
    - Probability of missing defects, not detected earlier.

Structured Walkthroughs

- The idea of structured walkthrough was proposed by Yourdon.
- An informal code analysis technique:
  - Undertaken after coding of a module is complete.
- A few members of the development team select some test cases:
  - simulate execution of the code by hand using these test cases.

Structured Walkthroughs cont …

- Even though an informal technique:
  - several guidelines have evolved over the years
  - making this naive but useful analysis technique more effective.
  - These guidelines are based on
    - personal experience, common sense, and several subjective factors.

Structured Walkthroughs cont …

- The guidelines should be considered as examples:
  - rather than accepted as rules to be applied strictly.
- Discussion should focus on discovery of errors:
  - and not on how to fix the discovered errors.
- To foster cooperation:
  - avoid the feeling among engineers that they are being evaluated in the code walk through meeting,
  - managers should not attend the walk through meetings.

Structured Walkthroughs

- The team performing code walk through should not be either too big or too small. Ideally, it should consist of between three to seven members.
- A typical structured walkthrough team consists of the following members:
  - Coordinator
    - Organizes, moderates, and follows up the walkthrough activities.
  - Presenter/developer
    - Optional member, introduces the item to be inspected.

Structured Walkthroughs cont…

- Scribe/recorder
  - Notes down the defects found and suggestions proposed by the members.
- Reviewer/tester
  - Finds the defects in the item
- Maintenance Oracle
  - Focuses on the long term implications
- Standards Bearer
  - Assesses adherence to standards
- User representative/ Accreditation agent
  - Reflects the needs and concerns of the user.

Structured Walkthroughs cont…

- The steps of the walkthrough process are:
  - Organization
  - Preparation
  - Walkthrough
  - Rework and follow-up

Walkthrough vs Inspection

- Walkthrough is less formal and rigorous than inspection, has fewer steps and does not use a checklist, whereas the inspection team uses a checklist for uncovering errors.

- In walkthrough, the tester brings in test cases that are mentally executed in the meeting, i.e. the test data are walked through the logic of the program, whereas in inspection, the program is simply read and discussed during meeting.

Technical reviews

- A technical review is intended to evaluate the software in the light of development standards, guidelines and specifications.
- A review is similar to an inspection and walkthrough, except that the review team also includes management.
- Therefore considered to be a higher level technique.

Technical reviews cont…

- Review agendas focus less on technical issues and more on foresight.
- The purpose is to evaluate the system relative to specifications and standards.
- The moderator should gather and distribute the documentation to all team members.

Technical reviews cont…

- A set of indicators is prepared to measure the following points:
  - Appropriateness of the problem definition and requirements
  - Adequacy of all underlying assumptions
  - Adherence to standards
  - Consistency
  - Completeness
  - documentation

Technical reviews cont…

- The moderator may also prepare check list to help team focus on key points.
- The result of the review is a document recording the events of the meeting, deficiencies and recommendations.
- Appropriate actions are then taken to correct any deficiency and address recommendations.

Clean Room Testing

- The term **cleanroom** was first coined at IBM by drawing analogy to the semiconductor fabrication units where defects are avoided by manufacturing in an ultra-clean atmosphere.
- This type of testing relies heavily on walkthroughs, inspection, and formal verification.
- The programmers are not allowed to test any of their code by executing the code other than doing some syntax testing using a compiler.

Clean Room Testing cont…

- This technique reportedly produces documentation and code that is more reliable and maintainable than other development methods relying heavily on code execution-based testing.
- Software testing in the cleanroom process is carried out as a statistical experiment.
- Based on the formal specification, a representative subset of software input/output trajectories/paths is selected and tested.
- This sample is then statistically analysed to produce an estimate of the reliability of the software, and a level of confidence in that estimate.

Limitations of Clean Room Testing

- The main problem with this approach is that testing effort is increased as walkthroughs, inspection, and verification are time consuming for detecting all simple errors.
- Also testing-based error detection is efficient for detecting certain errors that escape manual inspection.

Summary

- Discussed variants of inspection technique.
- Discussed about structured walkthrough – a kind of static testing technique.
- Briefly explained technical reviews - a kind of static testing technique.
- Presented the concept of clean room testing.

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 6), Second Edition, Oxford University Press, 2016.

Thank You

Software Debugging

Debugging

- Once errors are identified:
  - it is necessary to identify the precise location of the errors and to fix them.
- Each debugging approach has its own advantages and disadvantages:
  - each is useful in appropriate circumstances.

Some Debugging Approaches

- Brute Force method
- Symbolic Debugger
- Backtracking
- Cause-Elimination Method
- Program Slicing

Brute-force method

- This is the most common method of debugging:
  - least efficient method.
  - program is loaded with print statements
  - print the intermediate values
  - hope that some of printed values will help identify the error.

Symbolic Debugger

- A symbolic debugger (or source-level debugger) bridges the gap between binary code and human logic by allowing developers to inspect and manipulate runtime execution using the original source code variables and names rather than raw memory addresses.

Symbolic Debugger

- Using a symbolic debugger:
  - values of different variables can be easily checked and modified
  - single stepping to execute one instruction at a time
  - break points and watch points can be set to test the values of variables.

Backtracking

- When a test case fails, backtracking is a logical, reverse-engineering approach used by developers to locate the source code error.
- This is a fairly common approach.
- Beginning at the statement where an error symptom has been observed:
  - source code is traced backwards until the error is discovered.
- Stop backtracking the moment the point where the data or state first became corrupted is found. That point is the bug.

Example

```
int main( ){
 int i, j, s;
 i=1;
 while(i<=10){
          s=s+i;
          i++;
         j=j++;}
printf("%d",s);
}
```

Backtracking

- Unfortunately, as the number of source lines to be traced back increases,
  - the number of potential backward paths increases
  - becomes unmanageably large for complex programs.

Cause-elimination method

- In this method, once a failure is observed, the symptoms of the failure (e.g. certain variable is having a negative value though it should be positive) are noted.
- Determine a list of causes:
  - which could possibly have contributed to the error symptom.
  - tests are conducted to eliminate each.
- A related technique of identifying errors by examining error symptoms:
  - software fault tree analysis.

Program Slicing

- This technique is similar to back tracking.
- However, the search space is reduced by defining slices.
- A slice is defined for a particular variable at a particular statement:
  - set of source lines preceding this statement which can influence the value of the variable.

Program Slicing cont …

- Slice of a program w.r.t. program point **p** and variable **x**:

  - All statements and predicates that might affect the value of **x** at point **p**.

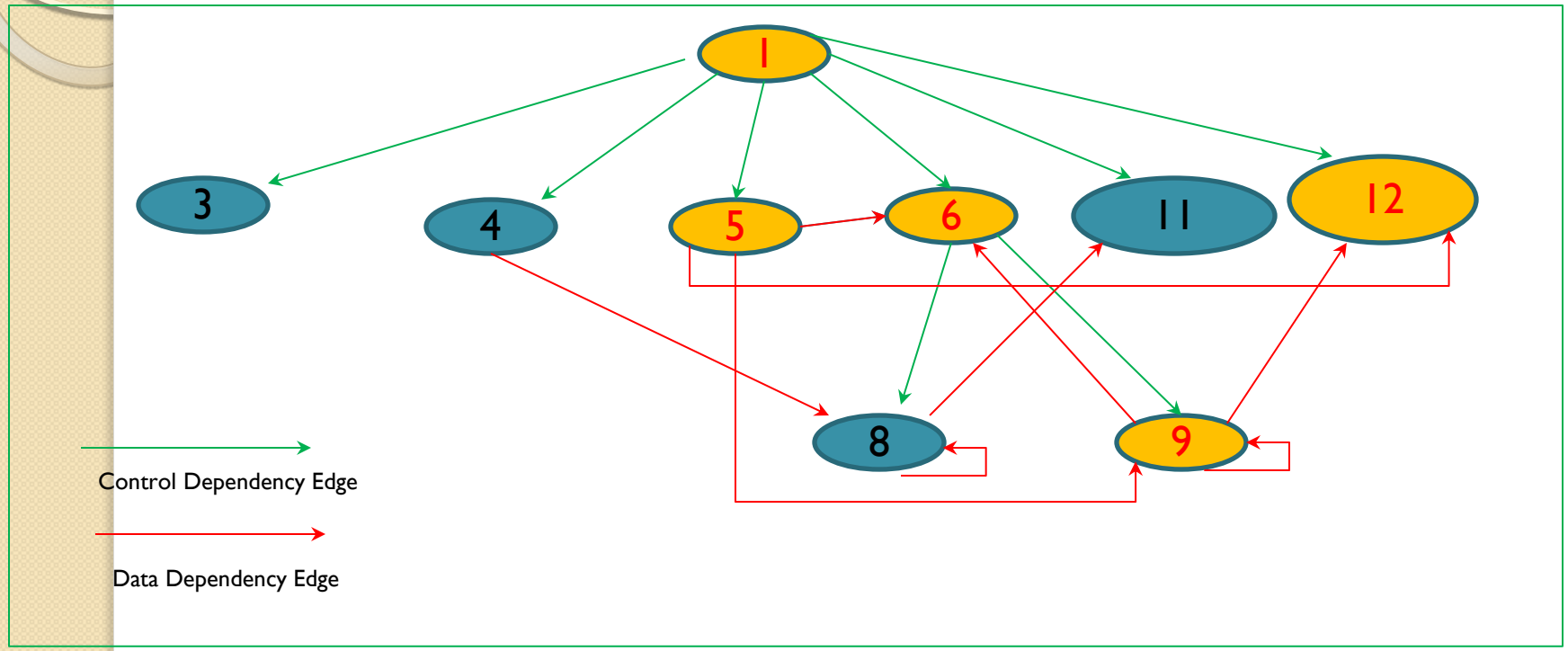
- <**p**, **x**> known as slicing criterion.

Example

```
1 main( )
2 {
3   int i, sum;
4   sum = 0;
5   i = 1;
6   while(i <= 10)
7         {
8       Sum = sum + 1;
9       ++ i;
10        }
11      printf("%d", sum);
12      printf("%d", i);
13      }
```

An Example Program & its backward slice w.r.t. <12, i>

Program Dependence Graph



Example

An Example Program

& its slice w.r.t. <9, i>

```
1.  int main( ){
2.  int  i, s;
3.   i=1;
4.  s=1;
5.   while(i<=10){
6.        s=s+i;
7.        i++;}
8.  printf("%d",s),
9.  printf("%d",i);
10. }
```

Types of Slices

Static Slice: Statements that may affect value of a variable at a program point for *all possible executions*.

Dynamic Slice: Statements that actually affect value of a variable at a program point for *that particular execution*.

Backward Slice: Statements that *might have* affected the variable at a program point.

Forward Slice: Statements that *might be* affected by the variable at a program point.

Example of Forward Slice

```
1 main( )
2 {
3   int i, sum;
4   sum = 0;
5   i = 1;
6   while(i <= 10)
7         {
8         sum = sum + 1;
9         ++ i;
10         }
11        printf("%d", sum);
12        printf("%d", i);
13        }
```

An Example Program & its forward slice w.r.t. <5, i>

Types of Slices cont …

- **Intra-Procedural Slice:** for programs having only one procedure
  - Not applicable for OOPs

- **Inter-Procedural Slice:** for programs having more than one procedure
  - Applicable for OOPs

Applications of Slicing

- Debugging
- Program understanding
- Testing
- Software maintenance
- Complexity measurement
- Program integration
- Reverse engineering
- Software reuse

Debugging Guidelines

- Debugging usually requires a thorough understanding of the program design.
- Debugging may sometimes require full redesign of the system.
- A common mistake novice programmers often make:
  - not fixing the error but the error symptoms.

Debugging Guidelines

- Be aware of the possibility:
  - an error correction may introduce new errors.
- After every round of error-fixing:
  - regression testing must be carried out.

Program Analysis Tools

- An automated tool:
  - takes program source code as input
  - produces reports regarding several important characteristics of the program,
  - such as size, complexity, adequacy of commenting, adherence to programming standards, etc.

Program Analysis Tools

- Some program analysis tools:
  - produce reports regarding the adequacy of the test cases.
- There are essentially two categories of program analysis tools:
  - Static analysis tools
  - Dynamic analysis tools

Static Analysis Tools

- Static analysis tools:
  - Assess properties of a program without executing it.
  - Analyze the source code
    - provide analytical conclusions.

Static Analysis Tools

- Whether coding standards have been adhered to?
  - Commenting is adequate?
- Programming errors such as:
  - uninitialized variables
  - mismatch between actual and formal parameters.
  - Variables declared but never used, etc.

Static Analysis Tools

- Code walk through and inspection can also be considered as static analysis methods:
  - however, the term static program analysis is generally used for automated analysis tools.

Dynamic Analysis Tools

- Dynamic program analysis tools require the program to be executed:
  - its behaviour recorded.
  - Produce reports such as, extent of coverage achieved, adequacy of test cases, etc.

Summary

- Discussed different debugging approaches.
  - Brute Force method
  - Symbolic Debugger
  - Backtracking
  - Cause-Elimination Method
  - Program Slicing
- Presented some debugging guidelines.
- Explained the Program Analysis Tools
  - Static analysis tools
  - Dynamic analysis tools

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.

Thank You

Unit Validation Testing

Testing Levels

4 Testing Levels

- Software tested at 4 levels:
  - Unit testing
  - Integration testing
  - System testing
  - Regression testing

Unit testing

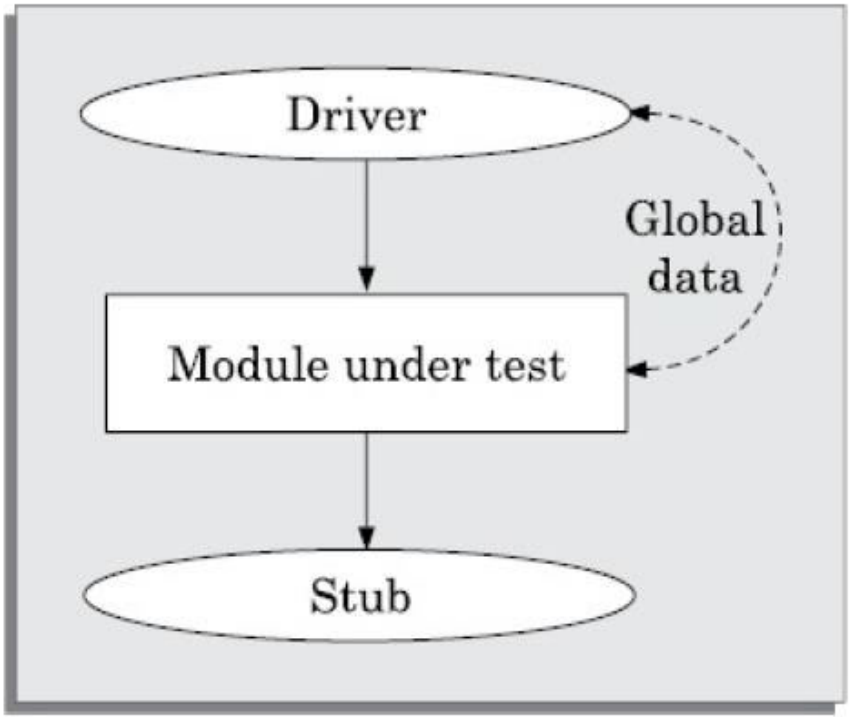
- Unit testing is undertaken after a module has been coded and reviewed.
- This activity is typically undertaken by the coder of the module himself in the coding phase.
- Before carrying out unit testing, the unit test cases have to be designed and the test environment for the unit under test has to be developed.

Driver and stub modules

- In order to test a single module, we need a complete environment to provide all relevant code that is necessary for execution of the module.
- Besides the module under test, the following are needed to test the module:
  - The procedures belonging to other modules that the module under test calls.
  - Non-local data structures that the module accesses.
  - A procedure to call the functions of the module under test with appropriate parameters.

- Modules required to provide the necessary environment (which either call or are called by the module under test) are usually not available until they too have been unit tested.
- In this context, stubs and drivers are designed to provide the complete environment for a module so that testing can be carried out.
- The role of stub and driver modules is pictorially shown in the following Figure.

Unit testing with the help of driver and stub modules



Verification versus Validation

- Verification is the process of determining:
  - Whether output of one phase of development conforms to its previous phase.

- Validation is the process of determining:
  - Whether a fully developed system conforms to its SRS document, i.e. the module that has been prepared till now is in conformance with the requirements which were set initially in the SRS or user manual.

Verification versus Validation

- Verification is concerned with phase containment of errors:
  - Whereas, the aim of validation is that the final product is error free.

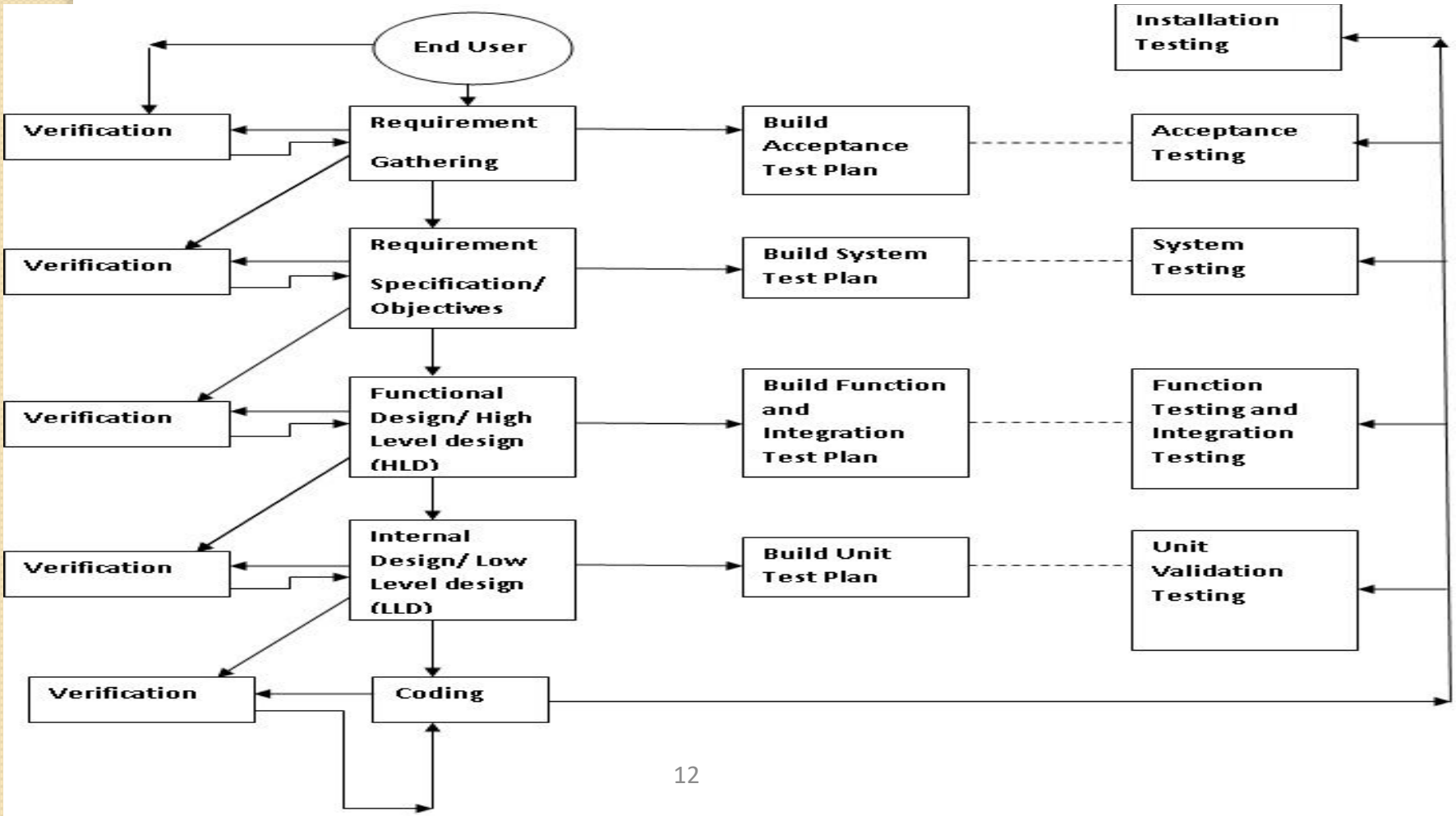
Verification and Validation Techniques

- Review
- Simulation
- Unit testing
- Integration testing

- System testing

- Software validation is achieved by a series of black-box tests that demonstrate conformity with requirements.
- Both test plan and test procedure are designed to ensure that
  - Functional requirements are satisfied
  - Behavioral characteristics achieved
  - Performance requirements attained

V&V activities



Unit Validation Testing

- Since unit is the smallest building block of the software system, it is the first piece of system to be validated. Before we validate the entire software, units or modules must be validated.
- Unit testing is normally considered an adjunct to the coding step.
- Units must also be validated to ensure that every unit of software has been built in the right manner in conformance with user requirements.

Unit Validation Testing Cont…

- Unit tests ensure that the software meets at least a baseline level of functionality prior to integration and system testing.
- While developing the software, if the developer detects and removes the bug, it offers significant savings of time and costs.
- This type of testing is largely based on black-box techniques.

Module

- Software is divided into modules but a module is not an isolated entity.
- The module under consideration might be getting some inputs from another module or the module is calling some other module.
- It means that a module is not independent and cannot be tested in isolation.
- While testing the module, all its interfaces must be simulated if the interfaced modules are not ready at the time of testing the module under consideration.

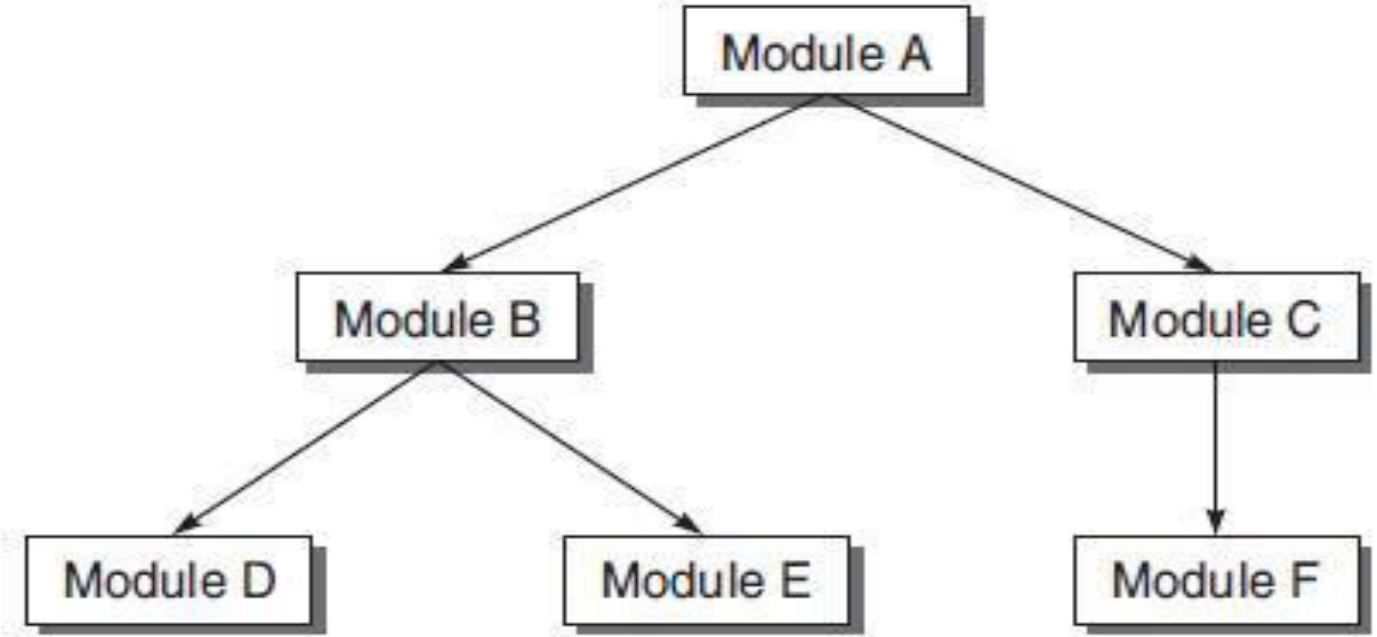
Drivers

- Software test drivers are programs which simulate the behaviors of software components (or modules) that are the control modules of a under test module.
- Control the test cases.
- A driver module should contain the non-local data structures accessed by the module under test.
- Additionally, it should also have the code to call the different functions of the unit under test with appropriate parameter values for testing.
- The driver module may print or interpret the results produced by the module under testing.

Example

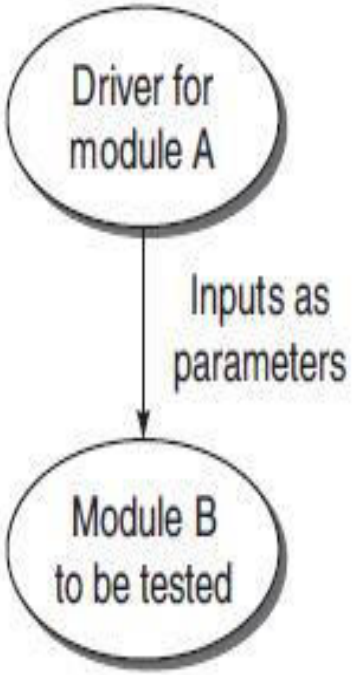
- For example, see the design hierarchy of the modules in Figure 1.
- Suppose module B is under test. In the hierarchy, module A is a superordinate of module B.
- Suppose module A is not ready and B has to be unit tested.

Design Hierarchy of an example system



Example

- A driver module is needed which will simulate module A in the sense that it passes the required inputs to module B and acts as main program for module B.



Drivers

- Test driver is supporting the code and data used to provide an environment for testing a part of a system in isolation.

- In fact, it drives the unit being tested by creating necessary ‘inputs’ required for the unit and then invokes the unit.

Drivers

- A test driver may take inputs in the following form and call the unit to be tested:

1. It may hard-code the inputs as parameters of the calling unit.
2. It may take the inputs from the user.
3. It may read the inputs from a file.

Drivers

- A test driver provides the following facilities to a unit to be tested:

1. Initializes the environment desired for testing.
2. Provides simulated inputs in the required format to the units to be tested.

Drivers

- Projects where commercial drivers are not available, specialized drivers need to be developed.

- This happens mainly in defense projects where projects are developed for a special application.

Stubs

- The module under testing may also call some other module which is not ready at the time of testing.

- Therefore, these modules need to be simulated for testing.

- In most cases, dummy modules instead of actual modules, which are not ready, are prepared for these subordinate modules.

- These dummy modules are called *stubs*.

Stub

- A stub is a dummy procedure that has the same I/O parameters as the function called by the unit under test but has a highly simplified behaviour.

- For example, a stub procedure may produce the expected behaviour using a simple table look up mechanism.

Stubs

- Thus, a stub can be defined as a piece of software that works similar to a unit which is referenced by the unit being tested, but it is much simpler than the actual unit.

- A stub works as a ‘stand-in’ for the subordinate unit and provides the minimum required behavior for that unit.

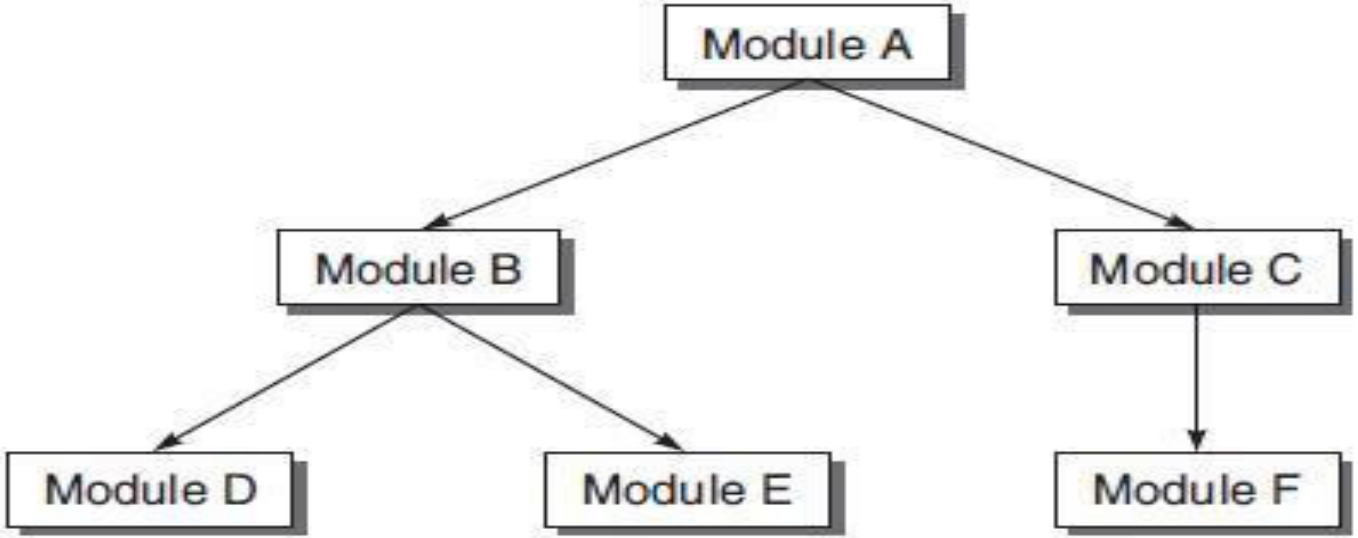
Stubs

- For example, consider Figure 1 again. Module B under test needs to call module D and module E. But they are not ready.

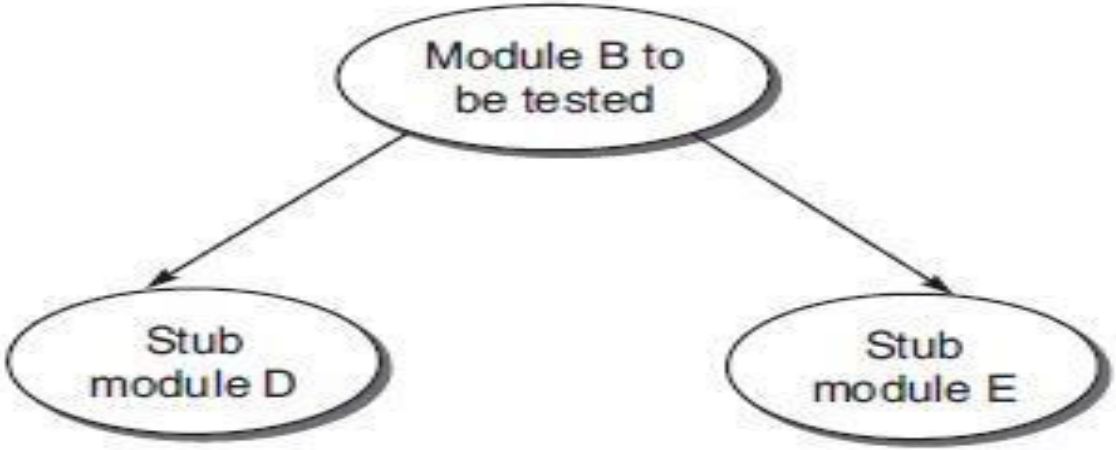
- So there must be some skeletal structure in their place so that they act as dummy modules in place of the actual modules.

- Therefore, stubs are designed for module D and module E, as shown in Figure 3.

Example



Example



Stubs

- Stubs have the following characteristics:

1. Stub is a place holder for the actual module to be called. Therefore, it is not designed with the functionalities performed by the actual module. It is a reduced implementation of the actual module.

Stubs

2. It does not perform any action of its own and returns to the calling unit (which is being tested).

3. We may include a display instruction as a trace message in the body of stub. The idea is that the module to be called is working fine by accepting the input parameters.

Stubs

4. A constant or null must be returned from the body of stub to the calling module.

5. Stub may simulate exceptions or abnormal conditions, if required.

- However, drivers and stubs represent overheads also. Overhead of designing them may increase the time and cost of the entire software system.

- Therefore, they must be designed simple to keep overheads low.

- Stubs and drivers are generally prepared by the developer of the module under testing.

- Developers use them at the time of unit verification.

- But they can also be used by any other person who is validating the unit.

Example

Consider the following program:

```
main()
  {
        Int  a, b, c, sum, diff, mul;
        scanf("%d %d %d", &a, &b, &c);
        sum = calsum(a,b,c);
        diff = caldiff(a,b,c);
        mul = calmul(a,b,c);
        printf("%d %d %d", sum, diff, mul);
  }
calsum(int x, int y, int z)
   {
        int d;
        d = x + y + z;
        return(d);
   }
```

Example

(a) Suppose main() module is not ready for the testing of calsum() module. Design a driver module for main().

(b) Modules caldiff() and calmul() are not ready when called in main(). Design stubs for these two modules.

Solution

Solution

(a) **Driver for `main()` module:**

```
main()
   {
          int a, b, c, sum;

          scanf("%d %d %d", &a, &b, &c);
          sum = calsum(a,b,c);
          printf("The output from calsum module is %d", sum);
}
```

Benefits of Designing Stubs and Drivers

1. Stubs allow the programmer to call a method in the code being developed, even if the method does not have the desired behavior yet.

2. By using stubs and drivers effectively, we can cut down our total debugging and testing time by testing small parts of a program individually, helping us to narrow down problems before they expand.

3. Stubs and drivers can also be an effective tool for demonstrating progress in a business environment.

Solution

(b) **Stub for `caldiff()` Module**

```
caldiff(int x, int y, int z)
{
        printf("Difference calculating module");
        return 0;
}
```

Stub for `calmul()` Module

```
calmul(int x, int y, int z)
{
        printf("Multiplication calculation module");
        return 0;
}
```

Summary

- Discussed different levels of testing.
- Explained Verification vs. Validation.
- Presented V & V activities.
- Discussed unit validation testing.
- Explained the concept of Drivers and Stubs with an example.

References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 7), Second Edition, Oxford University Press, 2016.

Thank You

Integration Testing

Introduction

- In the modular design of a software system where the system is composed of different modules, integration is the activity of combining the modules together when all the modules have been prepared.
- Integration of modules is done according to the design of software specified earlier.
- Integration aims at constructing a working software system.
- But a working software demands full testing and thus, integration testing comes into the picture.

Introduction cont …

- Why do we need integration testing? When all modules have been verified independently, then why is integration testing necessary?

- As discussed earlier, modules are not standalone entities.

- They are a part of a software system which comprises of many interfaces. Even if a single interface is mismatched, many modules may be affected.

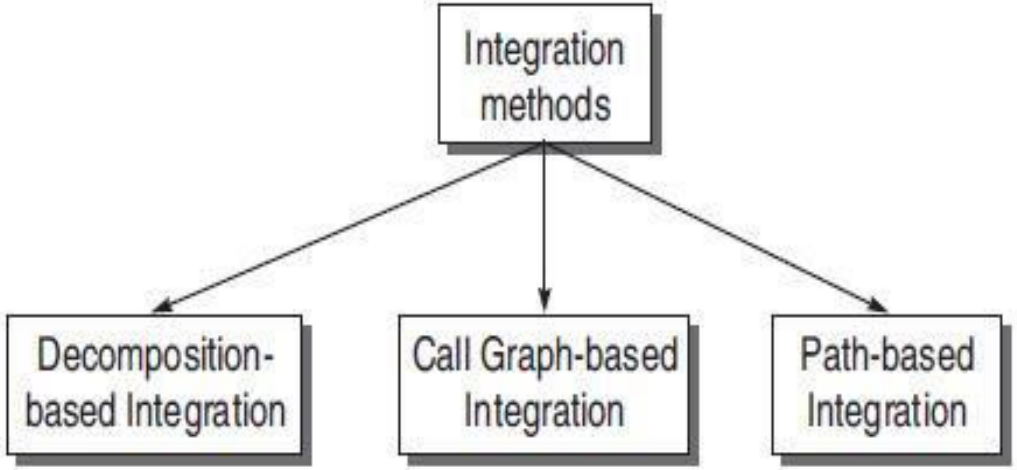
Introduction cont …

Thus, integration testing is necessary for the following reasons:

1. Integration testing exposes inconsistency between the modules such as improper call or return sequences.
2. Data can be lost across an interface.
3. One module when combined with another module may not give the desired result.
4. Data types and their valid ranges may mismatch between the modules.

Approaches for integration testing

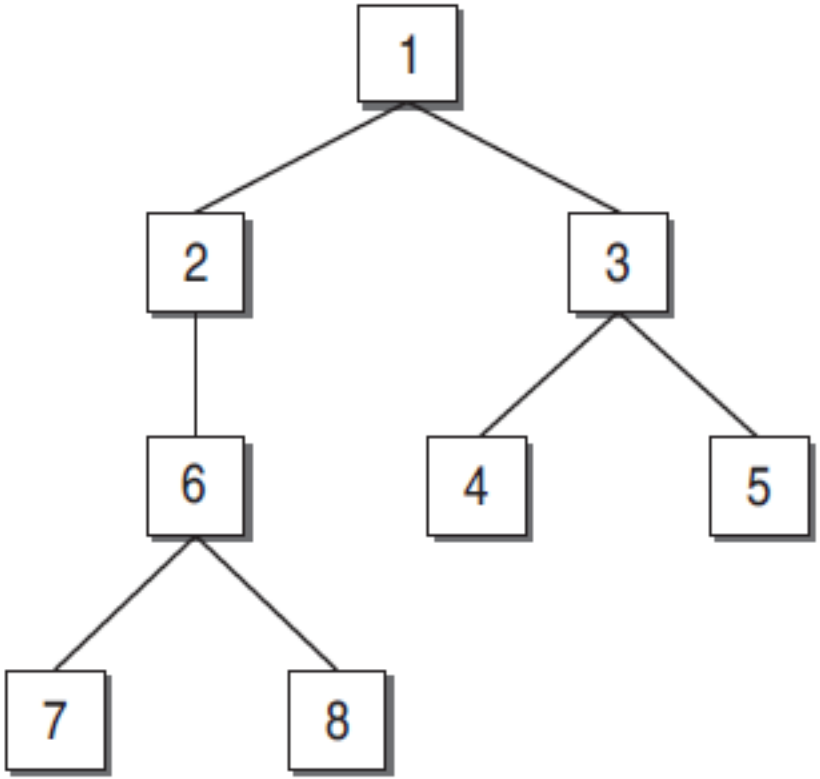
- Thus, integration testing focuses on bugs caused by interfacing between the modules while integrating them.
- There are three approaches for integration testing:



Decomposition-based integration

- Idea is based on decomposition into functional modules
- Functional decomposition is shown as a tree
- Classified into
  - Non-incremental
  - incremental

Integration Testing



- Non incremental Approach
  - –Also known as big-bang integration

  - –Discarded for the following reasons:
    - Big-bang requires more work
      - –With increase in the number of modules, more number of drivers and stubs will be required to test the modules independently.
    - Actual modules are not interfaced directly until the end of the software system.
    - It will be difficult to localize the errors since the exact location of bugs cannot be found easily.

- Incremental Approach

  – It is beneficial for the following reasons:
    - Does not require many drivers and stubs.
    - Interfacing errors are uncovered earlier.
    - It is easy to localize the errors since modules are combined one by one, thus debugging is easier.
    - Incremental testing is a more thorough testing

Types of incremental integration testing

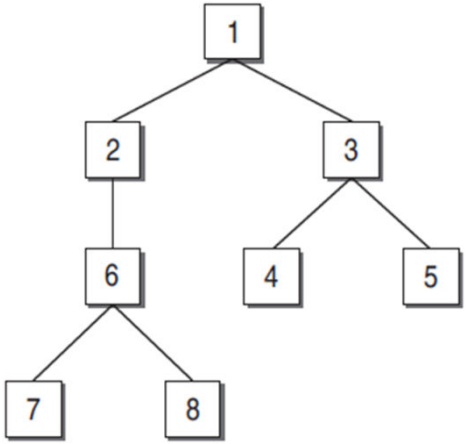
1. Top-down integration
2. Bottom-up integration

Top-down integration Testing

- The strategy in top-down integration is to look at the design hierarchy from top to bottom.
- Start with the high-level modules and move downward through the design hierarchy.
- Modules subordinate to the top module are integrated in the following two ways:
  - –Depth first integration
  - –Breadth first integration

Depth first integration

- In this type, all modules on a major control path of the design hierarchy are integrated first.
- In the example shown in below figure, modules 1, 2, 6, 7/8 will be integrated first. Next, modules 1, 3, 4/5 will be integrated.



Breadth first integration

- In this type, all modules directly subordinate at each level, moving across the design hierarchy horizontally, are integrated first.
- In the example shown in previous figure, modules 2 and 3 will be integrated first.
- Next, modules 6, 4, and 5 will be integrated. Modules 7 and 8 will be integrated last.

However, in practice, these two sequences of top-down integration cannot be used every time. In general, there is no best sequence, but the following guidelines can be considered:

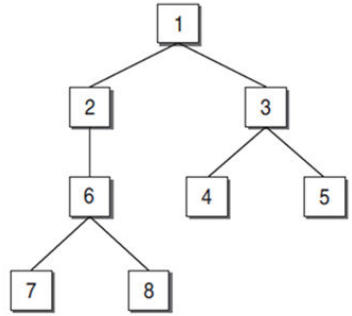
1. In practice, the availability of modules matters the most. The module which is ready to be integrated, will be integrated and tested first. We should not wait to test it according to depth first or breadth first sequence, but use the availability of modules.

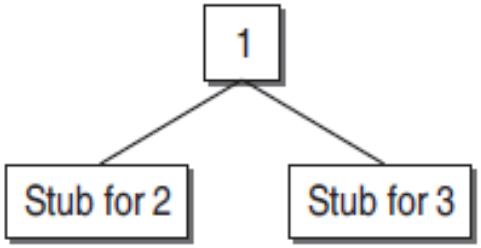
2. If there are critical sections of the software, design the sequence such that these sections will be added and tested as early as possible. A critical section might be a complex module, a module with a new algorithm or a module suspected to be error prone.
3. Design the sequence such that the I/O modules are added as early as possible, so that all interface errors will be detected earlier.

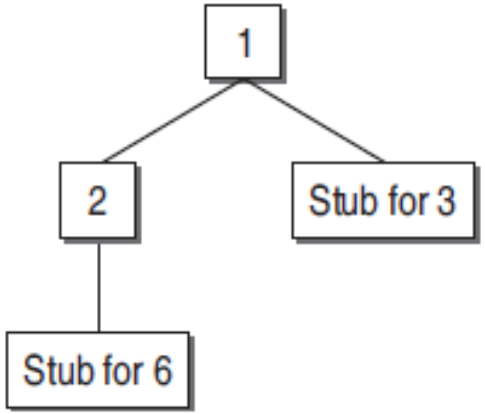
Top-Down Integration Procedure

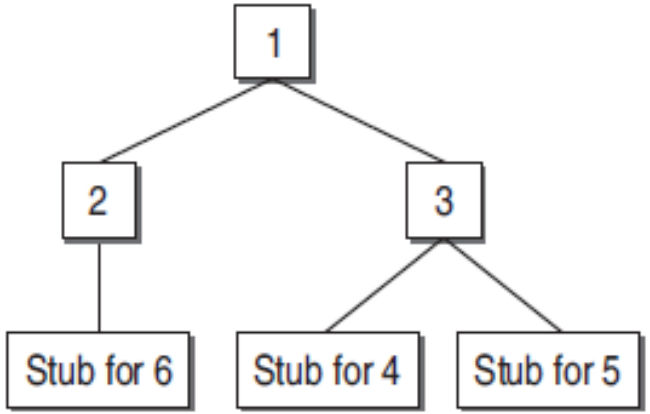
1. Start with the top or initial module in the software. Substitute the stubs for all the subordinate modules of top module. Test the top module.
2. After testing the top module, stubs are replaced one at a time with the actual modules for integration.
3. Perform testing on this recent integrated environment.
4. Regression testing may be conducted to ensure that new errors have not appeared.
5. Repeat steps 2-4 for the whole design hierarchy.

Example

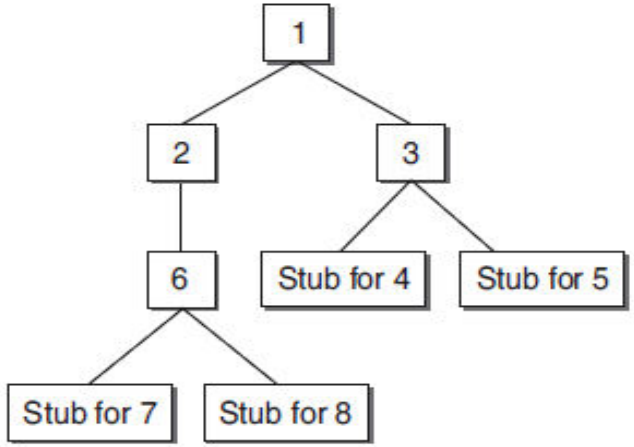

1
2
3
6
4
5
7
8

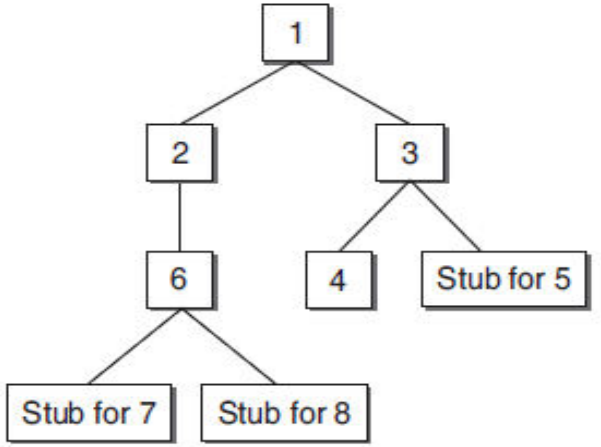

1
Stub for 2
Stub for 3

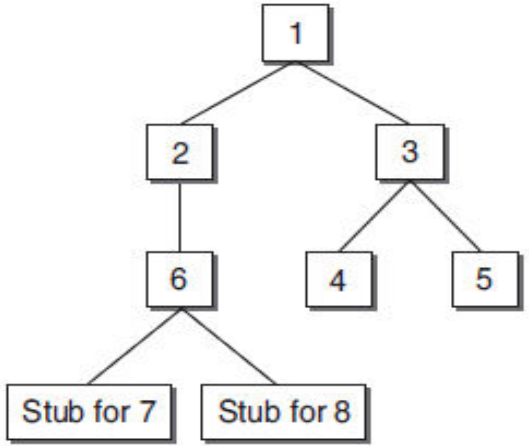

1
2
Stub for 3
Stub for 6

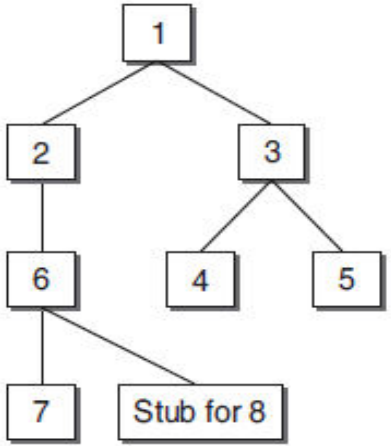

1
2
3
Stub for 6
Stub for 4
Stub for 5

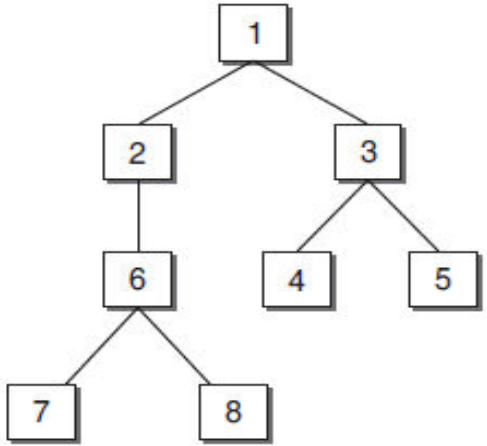
Example cont…











Drawbacks of top-down integration

- Stubs must be prepared as required
- Stubs are often more complicated than they appear
- Before the I/O functions are added, the representation of test cases in stubs can be difficult

Bottom-up Integration Testing

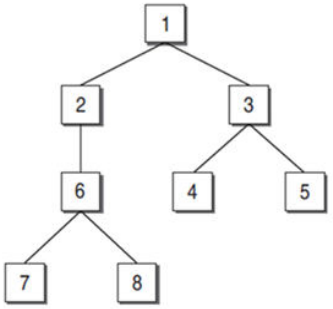
- The bottom-up strategy begins with the terminal or modules at the lowest level in the software structure. After testing these modules, they are integrated and tested moving from bottom to top level.
- Since the processing required for modules subordinate to a given level is always available, stubs are not required in this strategy.

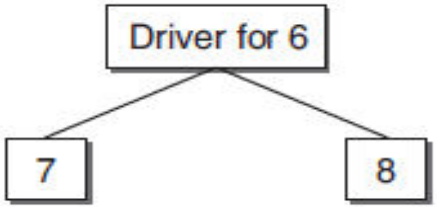
- Bottom-up integration can be considered as the opposite of top-down approach.
- Unlike top-down strategy, this strategy does not require the architectural design of the system to be complete.
- Thus, bottom-up integration can be performed at an early stage in the developmental process.
- It may be used where the system reuses and modifies components from other systems.

Steps in bottom-up integration

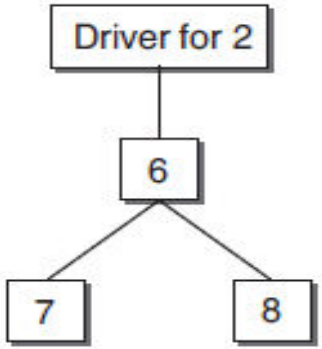
1. Start with the lowest level modules (modules from which no other module is being called), in the design hierarchy.
2. Look for the super-ordinate module which calls the module selected in Step 1. Design the driver module for this super-ordinate module.
3. Test the module selected in Step 1 with the driver designed in Step 2.
4. The next module to be tested is any module whose subordinate modules (modules it calls) have all been tested.
5. Repeat Steps 2 to 4 and move up in the design hierarchy.
6. Whenever, the actual modules are available, replace stubs and drivers with the actual one and test again.

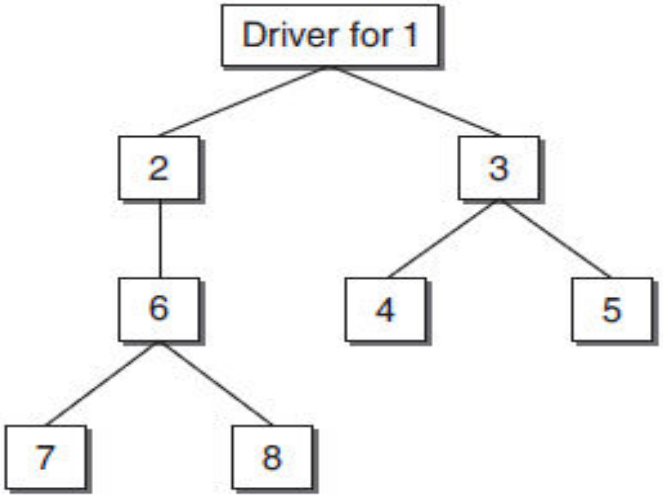
Example

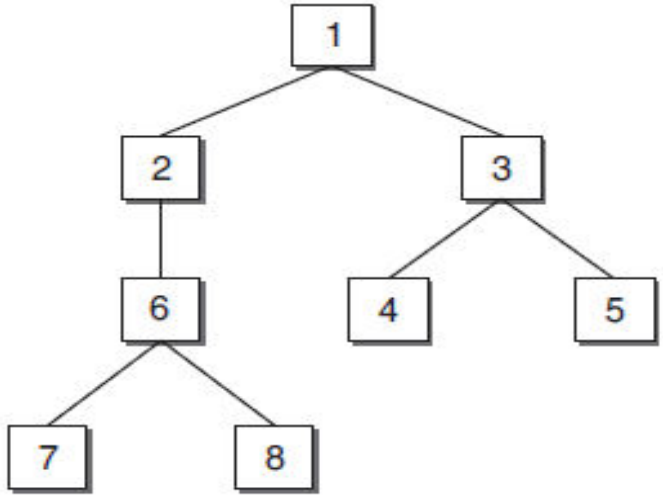

1
2
3
6
4
5
7
8


Driver for 6
7
8


Driver for 3
4
5


Driver for 2
6
7
8


Driver for 1
2
3
6
4
5
7
8


1
2
3
6
4
5
7
8

Comparison between top-down and bottom-up testing

Issue	Top-Down Testing	Bottom-Up Testing
Architectural Design	It discovers errors in high-level design, thus detects errors at an early stage.	High-level design is validated at a later stage.
System Demonstration	Since we integrate the modules from top to bottom, the high-level design slowly expands as a working system. Therefore, feasibility of the system can be demonstrated to the top management.	It may not be possible to show the feasibility of the design. However, if some modules are already built as reusable components, then it may be possible to produce some kind of demonstration.
Test Implementation	(nodes – 1) stubs are required for the subordinate modules.	(nodes – leaves) test drivers are required for super-ordinate modules to test the lower-level modules.

Practical Approach for Integration Testing

- There is no single strategy adopted for industry practice.
- For integrating the modules, one cannot rely on a single strategy.
- There are situations depending on the project in hand which will force to integrate the modules by combining top-down and bottom-up techniques.
- This combined approach is sometimes known as **sandwich integration testing**.

- Selection of an integration testing strategy depends on software characteristics and sometimes project schedules.
- In general, sandwich testing strategy that uses top-down tests for upper levels of the program structure with bottom-up tests for subordinate levels is the best compromise.
- The practical approach for adopting sandwich testing is driven by the following factors:

– Priority

- First putting together those subsystems with more important requirements.
- Follow top-down approach if the module has high level of control on its sub-ordinate modules.
- Modules with more user interfaces should be tested first, as they are more error prone.
- Module having high cyclomatic complexity should be tested first.

– Availability

- The module that is ready to be integrated. Will be tested first.

Pros and Cons of Decomposition Tech.

- Debugging is easy in decomposition based integration
- Better for monitoring the progress of integration.
- But, more effort is required as stubs and drivers are needed.

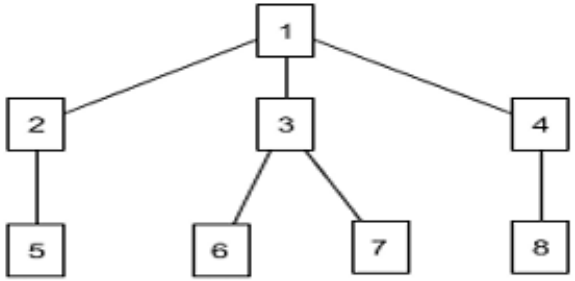
Integration Testing Effort

- The integration testing effort is computed as the number of test sessions.
- A test session is one set of test cases for a specific configuration.
- The total number of test sessions in a decomposition-based integration is computed as:
- Number of test sessions = nodes - leaves + edges

Summary

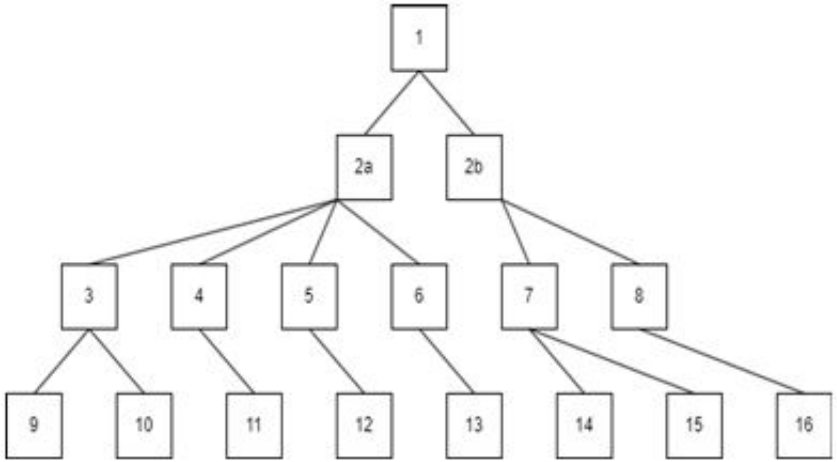
- Discussed basics of different approaches for integration testing.
- Discussed decomposition-based integration in detail.
  - Big bang integration
  - Top-down integration
    - Breadth First Integration
    - Depth First Integration
  - Bottom-up integration
  - Sandwich integration
- Explained how to compute the integration testing effort .

Perform top-down and bottom-up integration procedures from the following system hierarchy.

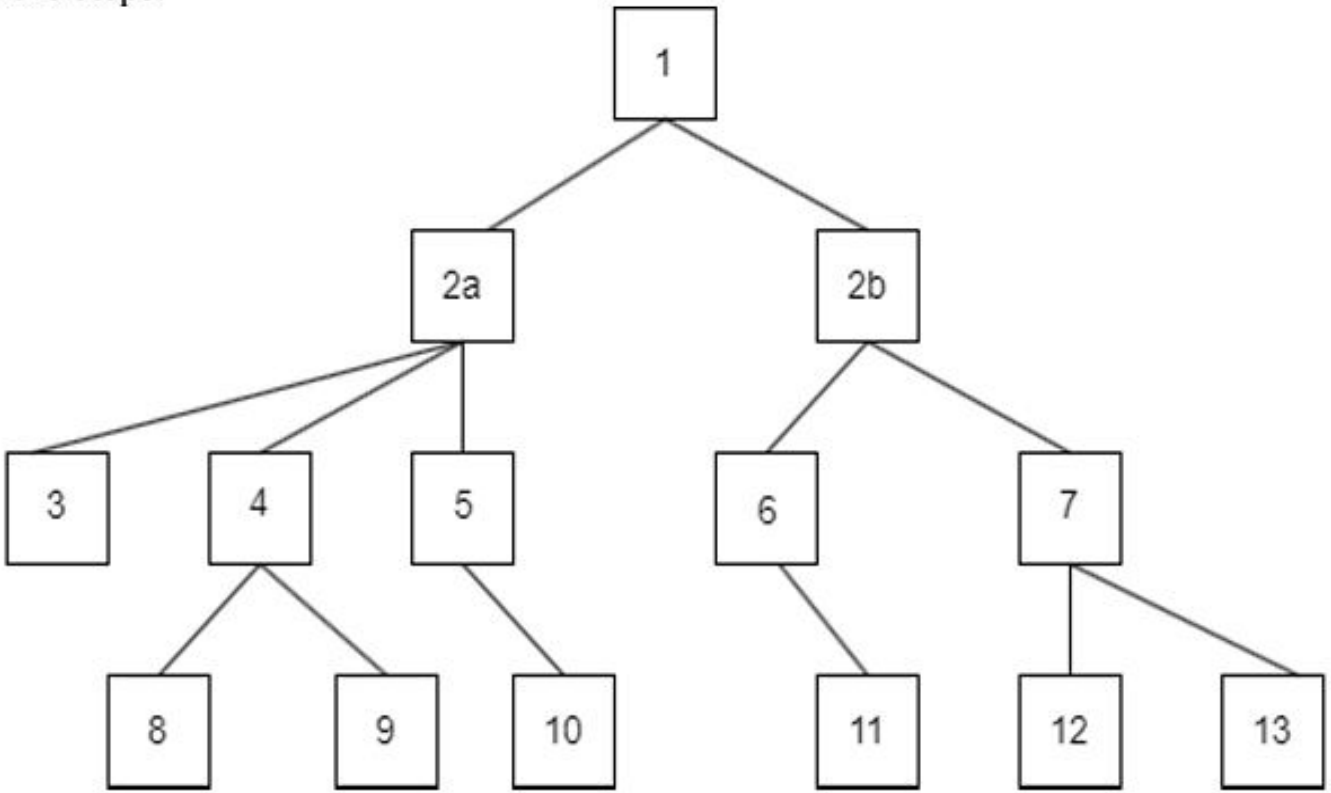


Calculate the number of test sessions.

Perform top-down integration procedure for the following system hierarchy. Show all the intermediate steps. Compute the effort required for this.



Perform bottom up integration procedure for the following system hierarchy. Show all the steps.



References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 7), Second Edition, Oxford University Press, 2016.

Thank You

Integration Testing cont …

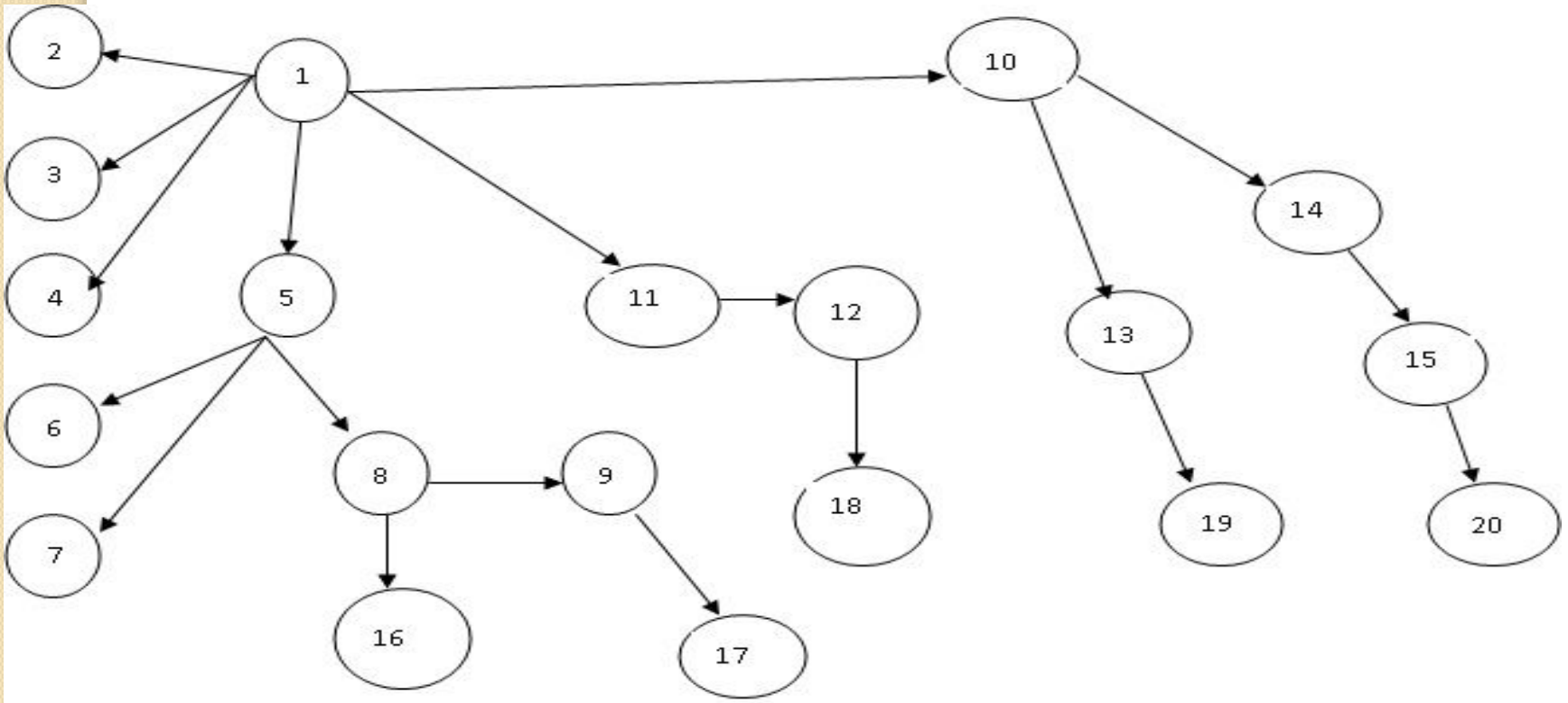
Call Graph-Based Integration

- It is assumed that integration testing detects bugs which are structural.
- However, it is also important to detect some behavioral bugs.
- If we can refine the functional decomposition tree into a form of module calling graph, then we are moving towards behavioral testing at the integration level.
- This can be done with the help of a *call graph* as given by Jorgensen.

Call Graph-Based Integration

- A call graph is a directed graph, wherein the nodes are either modules or units, and a directed edge from one node to another means one module has called another module.
- The call graph can be captured in a matrix form which is known as the adjacency matrix.

Example Call Graph



Adjacency Matrix

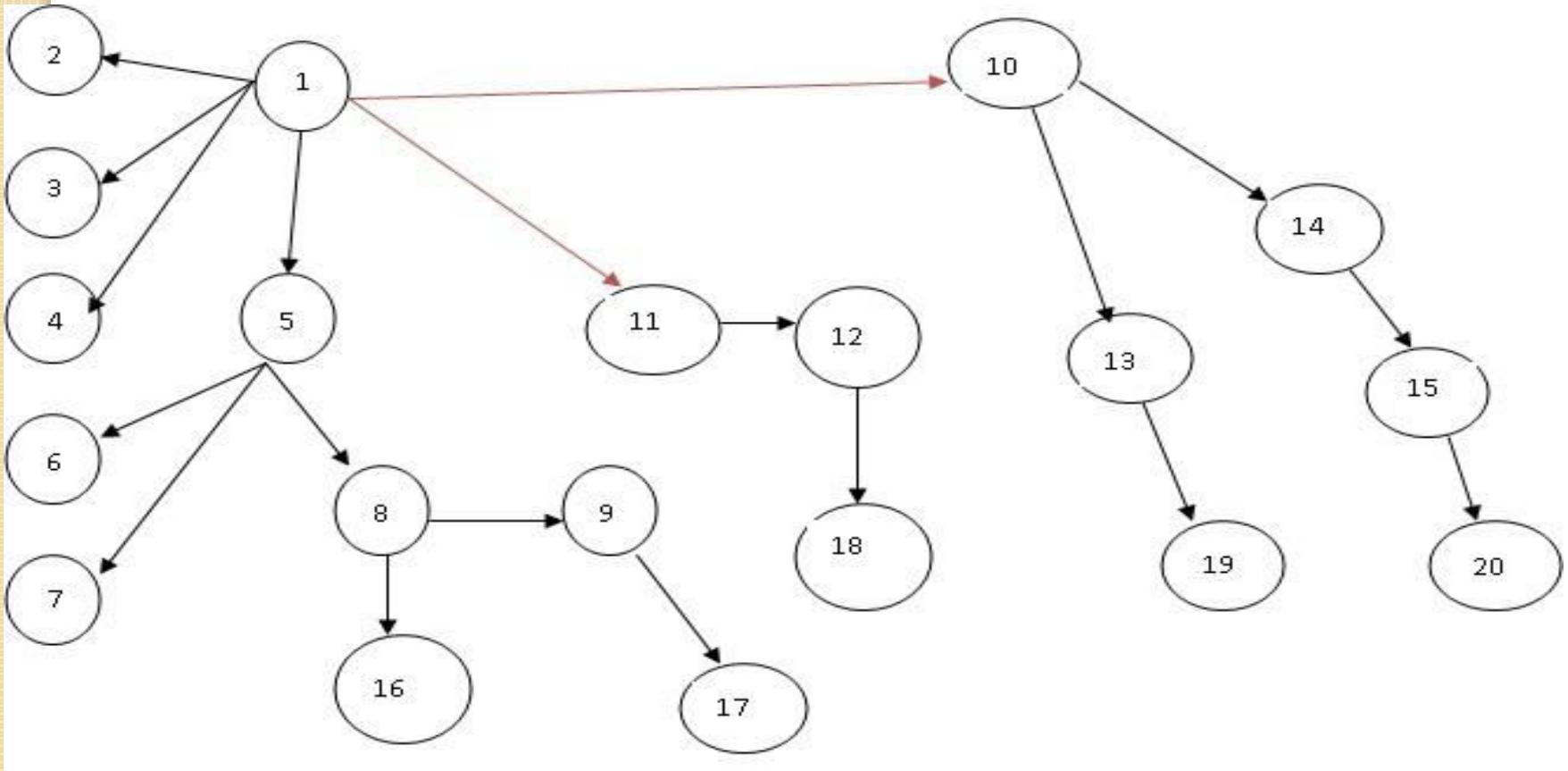
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	x	x	x	x					x	x									
2																			
3																			
4																			
5					x	x	x												
6																			
7																			
8								x							x				
9																x			
10												x	x						
11											x								
12																	x		
13																		x	
14														x					
15																			x
16																			
17																			
18																			
19																			
20																			

- The idea behind using a call graph for integration testing is to avoid the efforts made in developing the stubs and drivers.

- If we know the calling sequence, and if we wait for the called or calling function, if not ready, then call graph based integration can be used.

- There are two types of integration testing based on call graph:
  - –Pair-wise Integration
    - If we consider only one pair of calling and called modules, then we can make a set of pairs for all such modules.
    - The resulting set will be the total test sessions which will be equal to the sum of all edges in the call graph.
  - –Neighborhood Integration

Pair wise Integration



Test sessions = No. of edges = 19

The testing effort for pair-wise integration testing can be estimated as:

$$\boxed{E = I \times T \times t \times (1 + C)}$$

Where:

- E = Total testing effort
- I = Number of interfaces/module pairs
- T = Number of test cases per interface
- t = Average time per test case
- C = Contingency factor for defects and re-testing

Neighborhood Integration

- There is not much reduction in the number of test sessions in pair-wise integration as compared to the decomposition-based integration.
- If we consider neighborhoods of a node in the call graph, then number of test sessions may reduce.
- Neighborhood of a node is the immediate predecessor as well as the immediate successor of the node.
- So, neighborhood of a node can be defined as the set of nodes that are one edge away from the given node

Node	Neighbourhoods	
	Predecessors	Successors
1	.....	2,3,4,5,10,11
5	1	6,7,8
8	5	9,16
9	8	17
10	1	13,14
11	1	12
12	11	18
13	10	19
14	10	15
15	14	20

The total test sessions = nodes – sink nodes= **20 – 10 = 10**

sink node is an instruction in a module at which execution terminates.

If a call graph has:

- V = number of modules (vertices)
- E = number of direct calls (edges)

and every module's neighbourhood consists of itself plus all directly connected modules, then:

$$\boxed{\text{Total Neighbourhood Effort} = V + 2E}$$

Path Based Integration

- In call graph, when a module or unit executes, some path of the source instructions is executed.
- It is possible that in the path execution, there may be a call to another unit.
- At that point, the control is passed from the calling unit to the called unit.

Path Based Integration

- This passing of control from one unit to another unit is important for integration testing.
- It can be done with path based integration.
- We need to understand the following definitions for path-based integration:

– Source node
  - It is an instruction in the module at which the execution starts or resumes.
  - The nodes where the control is being transferred after calling the module are also source nodes.

– Sink node
  - It is an instruction in the module at which the execution terminates.
  - The nodes from which the control is transferred are also sink nodes.

– Module execution path (MEP)
  - It is a path consisting of set of executable statements within a module like in a flow graph.

–Message

- When the control is transferred from one unit to another, then the programming language mechanism used to do this is known as a message.
- For example, a function call (message from one unit to another unit)

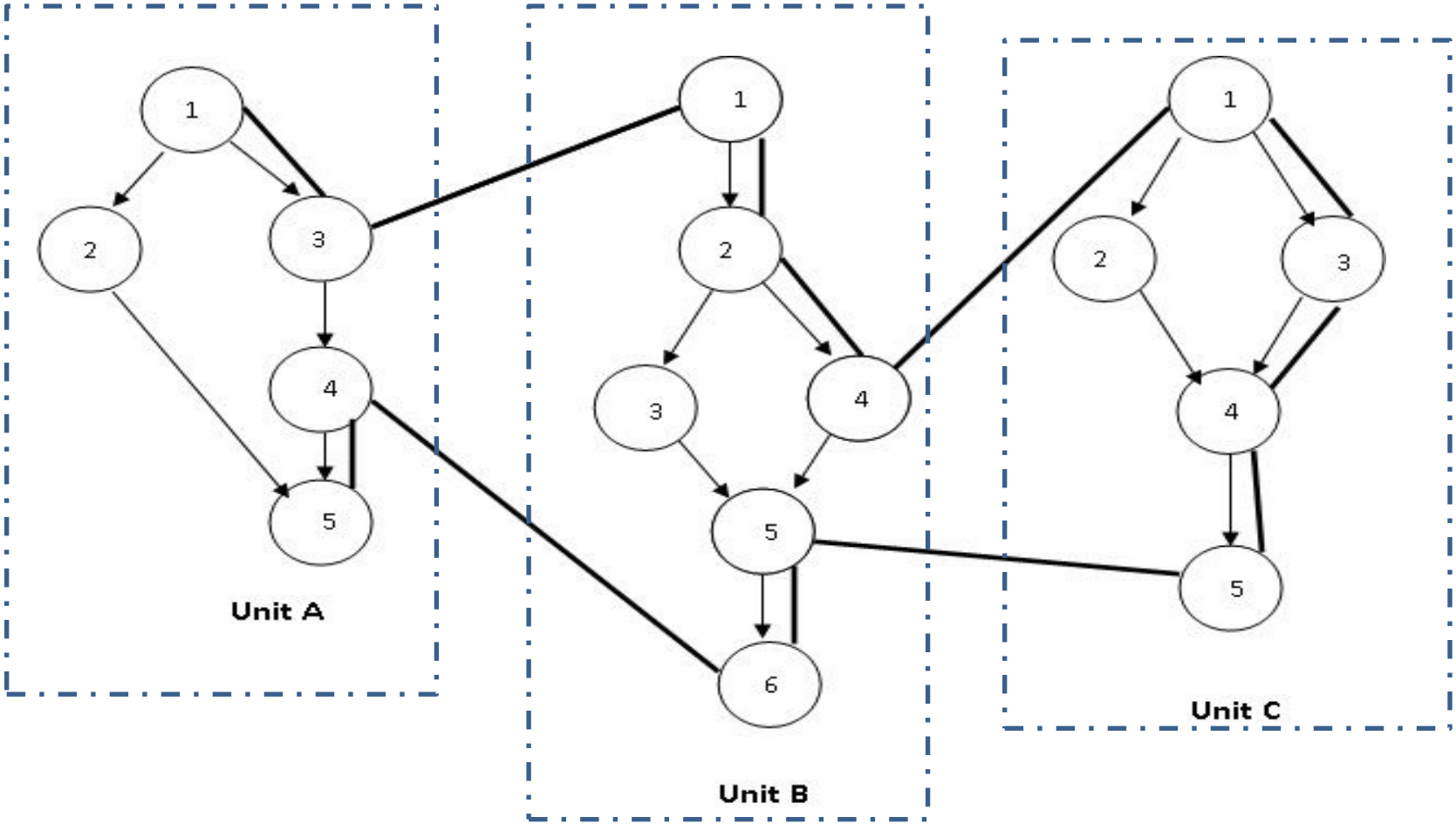
–MM-path

- It is a path consisting of MEPs and messages.
- The path shows the sequence of executable statements; it also crosses the boundary of a unit when a message is followed to call another unit.
- In other words, MM-path is a set of MEPs & transfer of control among different units in the form of messages.

– MM-Path Graph

- It can be defined as an extended flow graph where nodes are MEPs and edges are messages.
- It returns from the last called unit to the first unit where the call was made.
- In this graph (shown in next slide), messages are highlighted with thick lines.

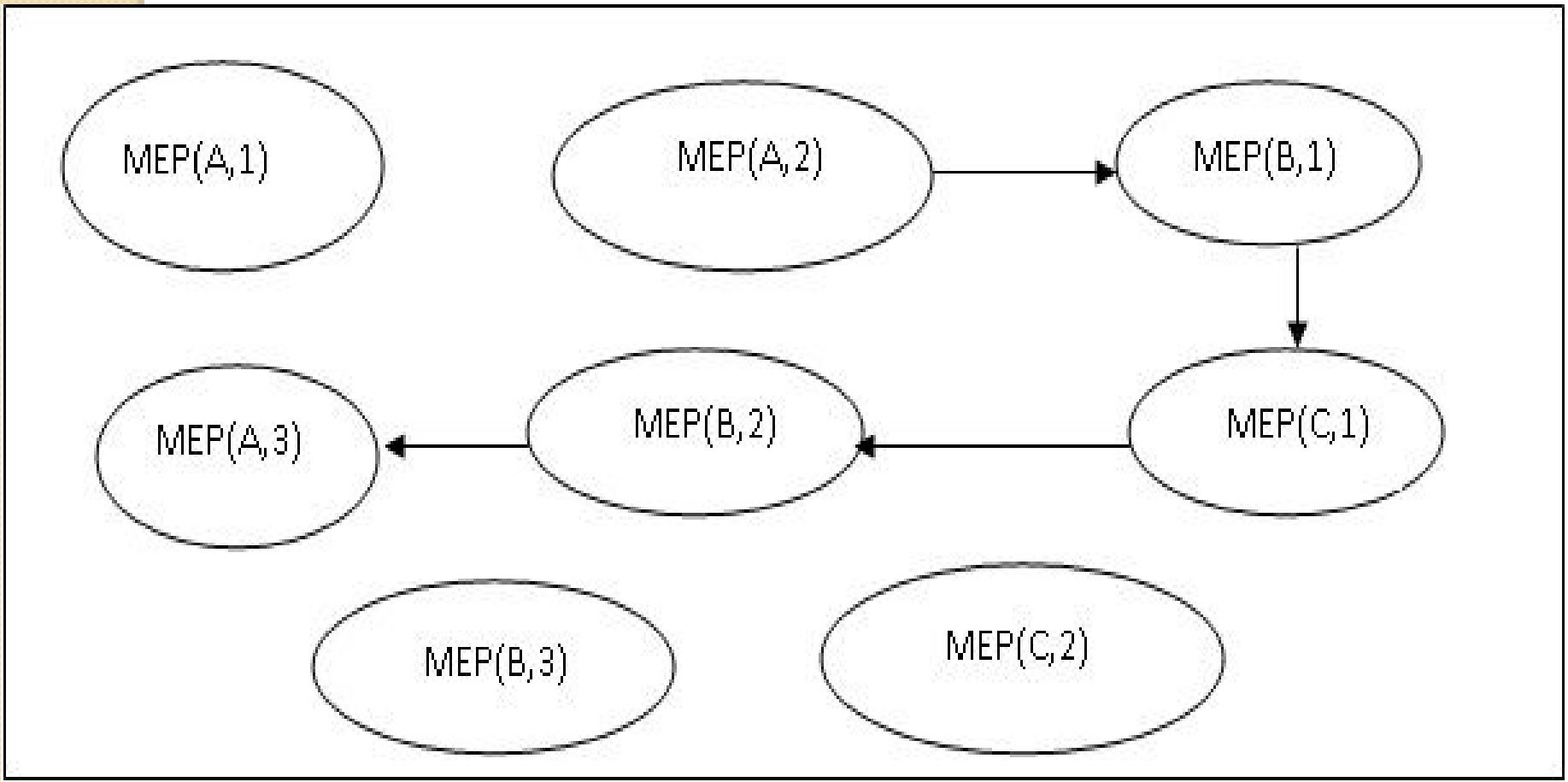
MM-Path



MM-Path Details

	Source nodes	Sink nodes	MEPs
Unit A	1,4	3,5	MEP(A,1)=<1,2,5> MEP(A,2)=<1,3> MEP(A,3)=<4,5>
Unit B	1,5	4,6	MEP(B,1)=<1,2,4> MEP(B,2)=<5,6> MEP(B,3)=<1,2,3,5,6>
Unit C	1	5	MEP(C,1)=<1,3,4,5> MEP(C,2)=<1,2,4,5>

MM-Path Graph



Function Testing

- When an integrated system is tested, all its specified functions and external interfaces are tested on the software.
- Every functionality of the system specified in the functions is tested according to its external specifications.
- An external specification is a precise description of the software behavior from the viewpoint of the outside world.

- Kit has defined function testing as the ***process of attempting to detect discrepancies between the functional specifications of a software and its actual behavior.***
- The objective of the function test is to measure the quality of the functional components of the system.
- Tests verify that the system behaves correctly from the user/business perspectives and functions according to the requirements, models or any other design paradigm.

- The function test must determine if each component or business event:

  - Performs in accordance to the specifications,
  - Responds correctly to all conditions that may present themselves by incoming events/data,
  - Moves data correctly from one business event to the next, and
  - Is initiated in the order required to meet the business objectives of the system

- Function testing can be performed after unit and integration testing, or whenever the development team thinks that the system has sufficient functionality to execute some tests.
- The test cases are executed such that the execution of the given test case against the software will exercise external functionality of certain parts.
- To keep a record of function testing, a function coverage metric is used.

- Function coverage can be measured with a function coverage matrix.
- It keeps track of those functions that exhibited the greatest number of errors.
- An effective function test cycle must have a defined set of processes and deliverables.
- The primary processes/ deliverables for requirement based function test are as follows:

–Test Planning

- The test leader with assistance from the test team defines the scope, schedule, and deliverables for the function test cycle.
- He delivers a test plan and a test schedule- these often undergo several revisions during the testing cycle.

–Partitioning/ functional Decomposition

- It is the process of breaking down a system into its functional components or functional areas.
- Another group in the organization takes responsibility for the functional decomposition of the system.
- If decompositions are deemed insufficient, then testing organization takes up the responsibility of decomposition.

–Requirement definition

- The testing organization needs specified requirements in the form of proper documents to proceed with the function test.
- These requirements need to be itemized under an appropriate functional partition.

–Test case design

- A tester designs and implements a test case to validate that the product performs in accordance with the requirements.
- These requirements need to be itemized under an appropriate functional partition and mapped to the requirements being tested.

–Traceability matrix formation

- Test cases need to be mapped back to the appropriate requirement.
- A function coverage matrix is prepared.
- This matrix is a table, listing specific functions to be tested, the priority for testing each function, and test cases required to test each function.
- Once all the aspects of the function have been tested by one or more test cases, then the test design activity for that function can be considered complete.

Function coverage matrix

Functions/Features	Priority	Test Cases
F1	3	T2, T4, T6
F2	1	T1, T3, T5

–Test case execution

- In all phases of testing, an appropriate set of test cases need to be executed and the result of those test cases recorded.
- So, the test case to be executed should be defined in the test plan.
- If the current application does not support the testing, then it should be deferred.

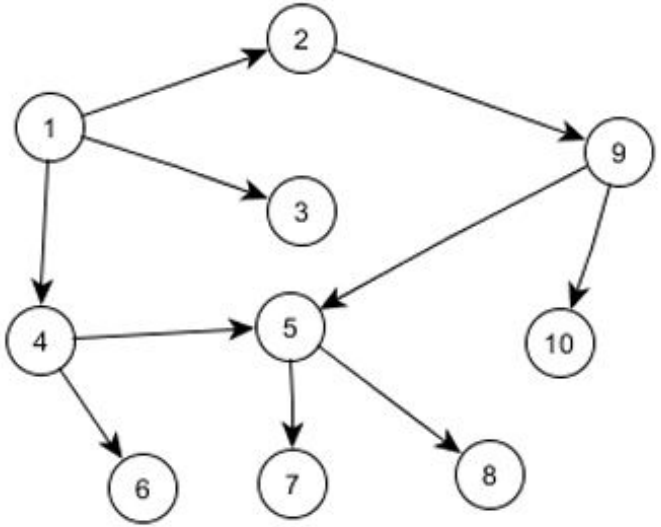
Summary

- Discussed call graph-based integration testing in detail.
  - Pair-wise Integration
  - Neighborhood Integration
- Explained path-based integration testing.
- Explained Function Testing in detail.
- Described the primary processes/ deliverables for requirement based function test.

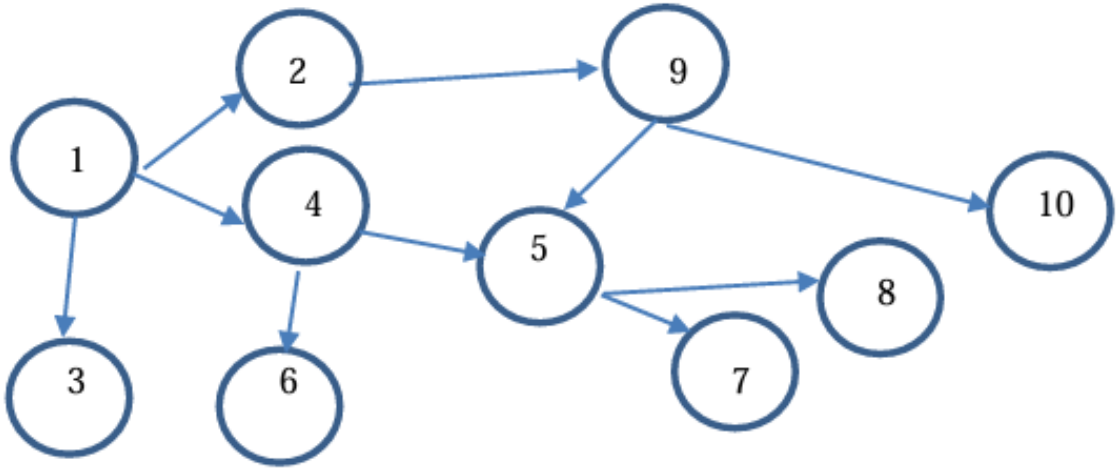
An application passes all unit tests but fails when two modules exchange data. Explain why this can happen and how Integration Testing can identify the problem.

?

Perform neighborhood integration testing for the call graph shown below and calculate the effort required for this testing.



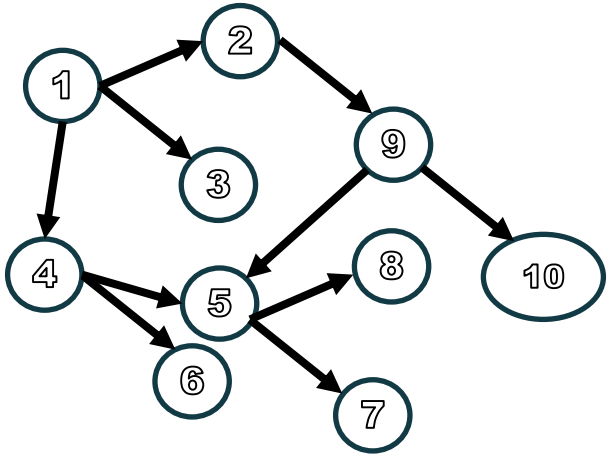
Perform pair-wise integration testing using the following call graph. Show all the intermediate steps. Calculate the testing effort required for pair-wise integration testing for the following call graph.



References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 7), Second Edition, Oxford University Press, 2016.

Thank You


1
2
3
4
5
6
7
8
9
10

Integration Testing cont …

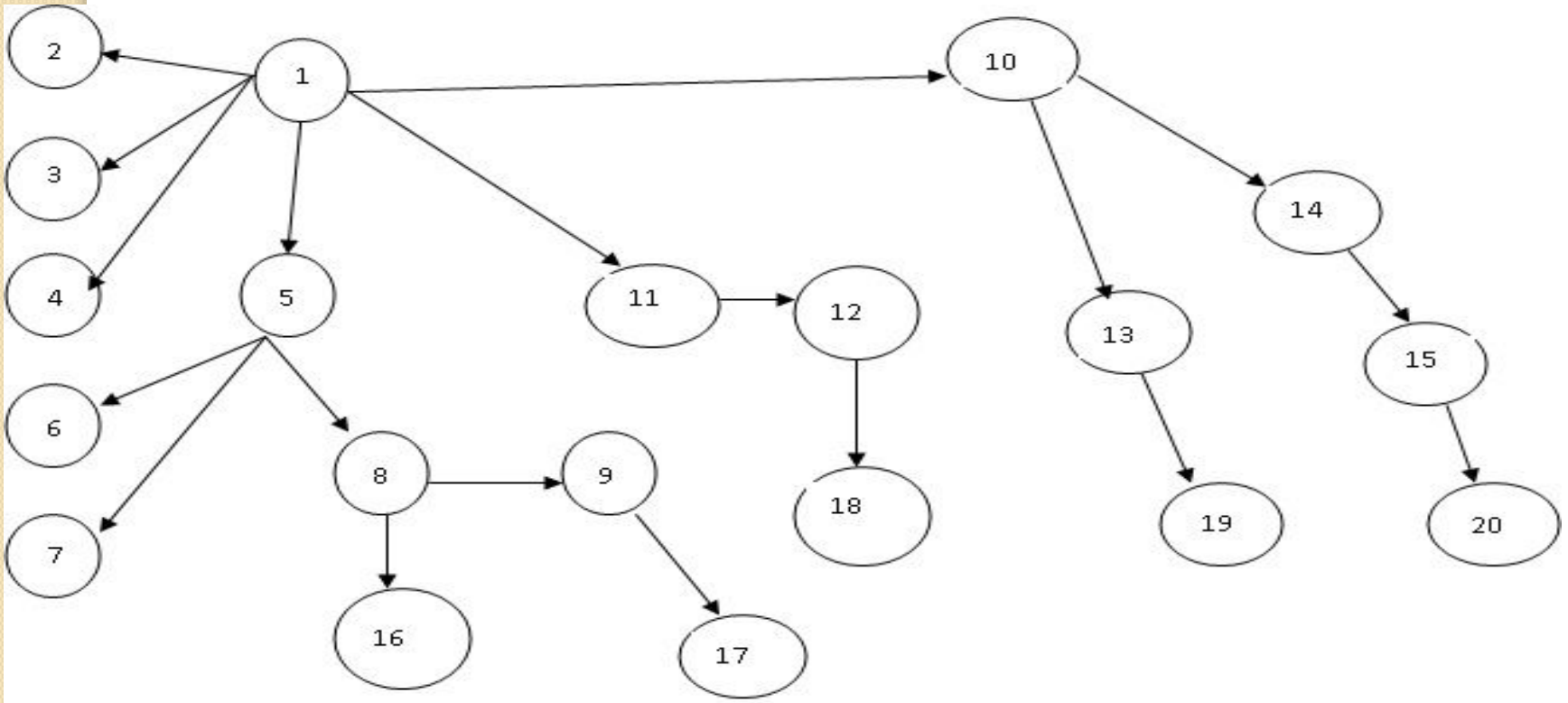
Call Graph-Based Integration

- It is assumed that integration testing detects bugs which are structural.
- However, it is also important to detect some behavioral bugs.
- If we can refine the functional decomposition tree into a form of module calling graph, then we are moving towards behavioral testing at the integration level.
- This can be done with the help of a *call graph* as given by Jorgensen.

Call Graph-Based Integration

- A call graph is a directed graph, wherein the nodes are either modules or units, and a directed edge from one node to another means one module has called another module.
- The call graph can be captured in a matrix form which is known as the adjacency matrix.

Example Call Graph



Adjacency Matrix

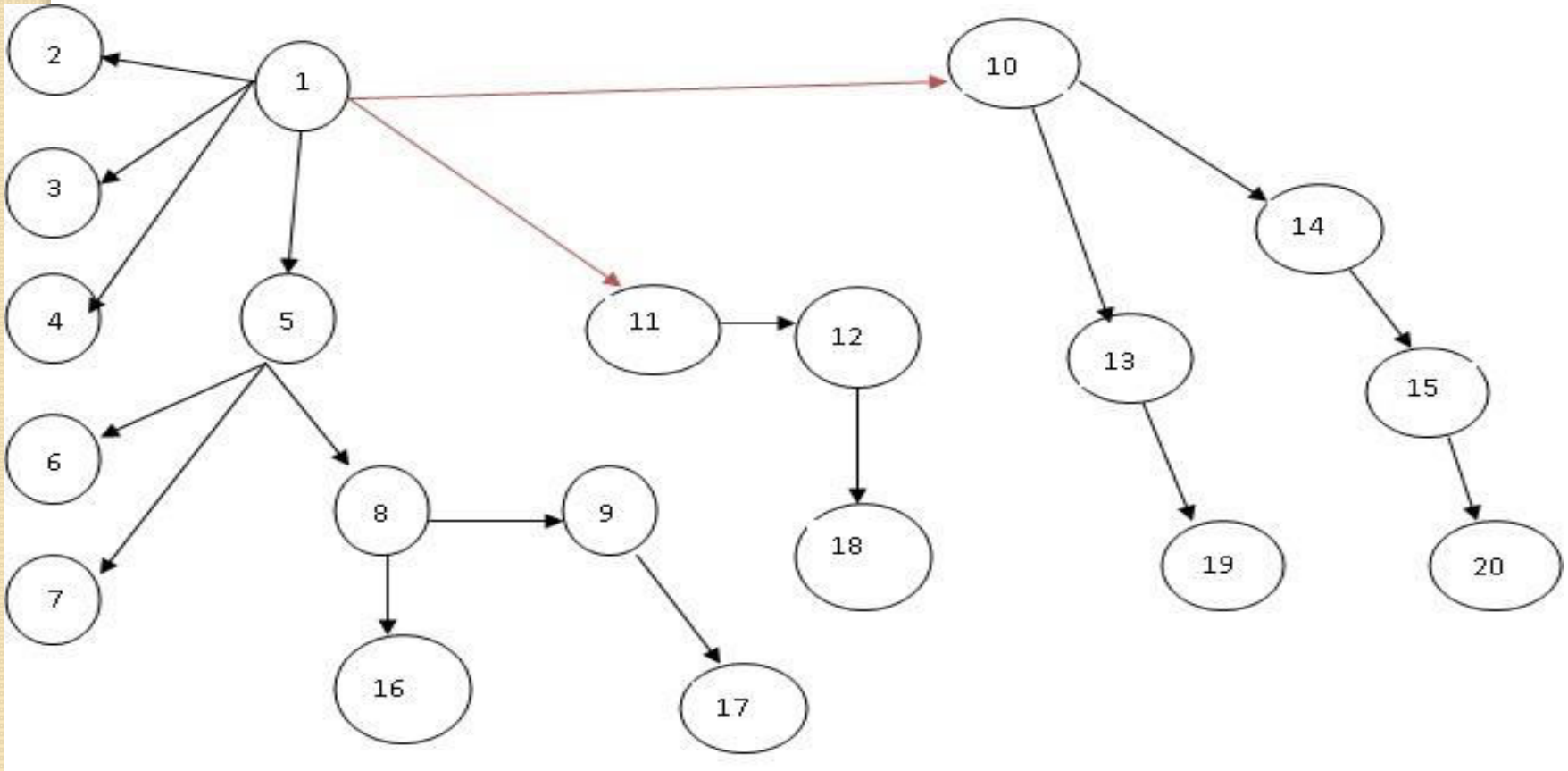
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	x	x	x	x					x	x									
2																			
3																			
4																			
5					x	x	x												
6																			
7																			
8								x							x				
9																x			
10												x	x						
11											x								
12																	x		
13																		x	
14														x					
15																			x
16																			
17																			
18																			
19																			
20																			

- The idea behind using a call graph for integration testing is to avoid the efforts made in developing the stubs and drivers.

- If we know the calling sequence, and if we wait for the called or calling function, if not ready, then call graph based integration can be used.

- There are two types of integration testing based on call graph:
  - –Pair-wise Integration
    - If we consider only one pair of calling and called modules, then we can make a set of pairs for all such modules.
    - The resulting set will be the total test sessions which will be equal to the sum of all edges in the call graph.
  - –Neighborhood Integration

Pair wise Integration



Test sessions = No. of edges = 19

The testing effort for pair-wise integration testing can be estimated as:

$$\boxed{E = I \times T \times t \times (1 + C)}$$

Where:

- E = Total testing effort
- I = Number of interfaces/module pairs
- T = Number of test cases per interface
- t = Average time per test case
- C = Contingency factor for defects and re-testing

Neighborhood Integration

- There is not much reduction in the number of test sessions in pair-wise integration as compared to the decomposition-based integration.
- If we consider neighborhoods of a node in the call graph, then number of test sessions may reduce.
- Neighborhood of a node is the immediate predecessor as well as the immediate successor of the node.
- So, neighborhood of a node can be defined as the set of nodes that are one edge away from the given node

Node	Neighbourhoods	
	Predecessors	Successors
1	.....	2,3,4,5,10,11
5	1	6,7,8
8	5	9,16
9	8	17
10	1	13,14
11	1	12
12	11	18
13	10	19
14	10	15
15	14	20

The total test sessions = nodes – sink nodes= **20 – 10 = 10**

sink node is an instruction in a module at which execution terminates.

If a call graph has:

- V = number of modules (vertices)
- E = number of direct calls (edges)

and every module's neighbourhood consists of itself plus all directly connected modules, then:

$$\boxed{\text{Total Neighbourhood Effort} = V + 2E}$$

Path Based Integration

- In call graph, when a module or unit executes, some path of the source instructions is executed.
- It is possible that in the path execution, there may be a call to another unit.
- At that point, the control is passed from the calling unit to the called unit.

Path Based Integration

- This passing of control from one unit to another unit is important for integration testing.
- It can be done with path based integration.
- We need to understand the following definitions for path-based integration:

– Source node
  - It is an instruction in the module at which the execution starts or resumes.
  - The nodes where the control is being transferred after calling the module are also source nodes.

– Sink node
  - It is an instruction in the module at which the execution terminates.
  - The nodes from which the control is transferred are also sink nodes.

– Module execution path (MEP)
  - It is a path consisting of set of executable statements within a module like in a flow graph.

–Message

- When the control is transferred from one unit to another, then the programming language mechanism used to do this is known as a message.
- For example, a function call (message from one unit to another unit)

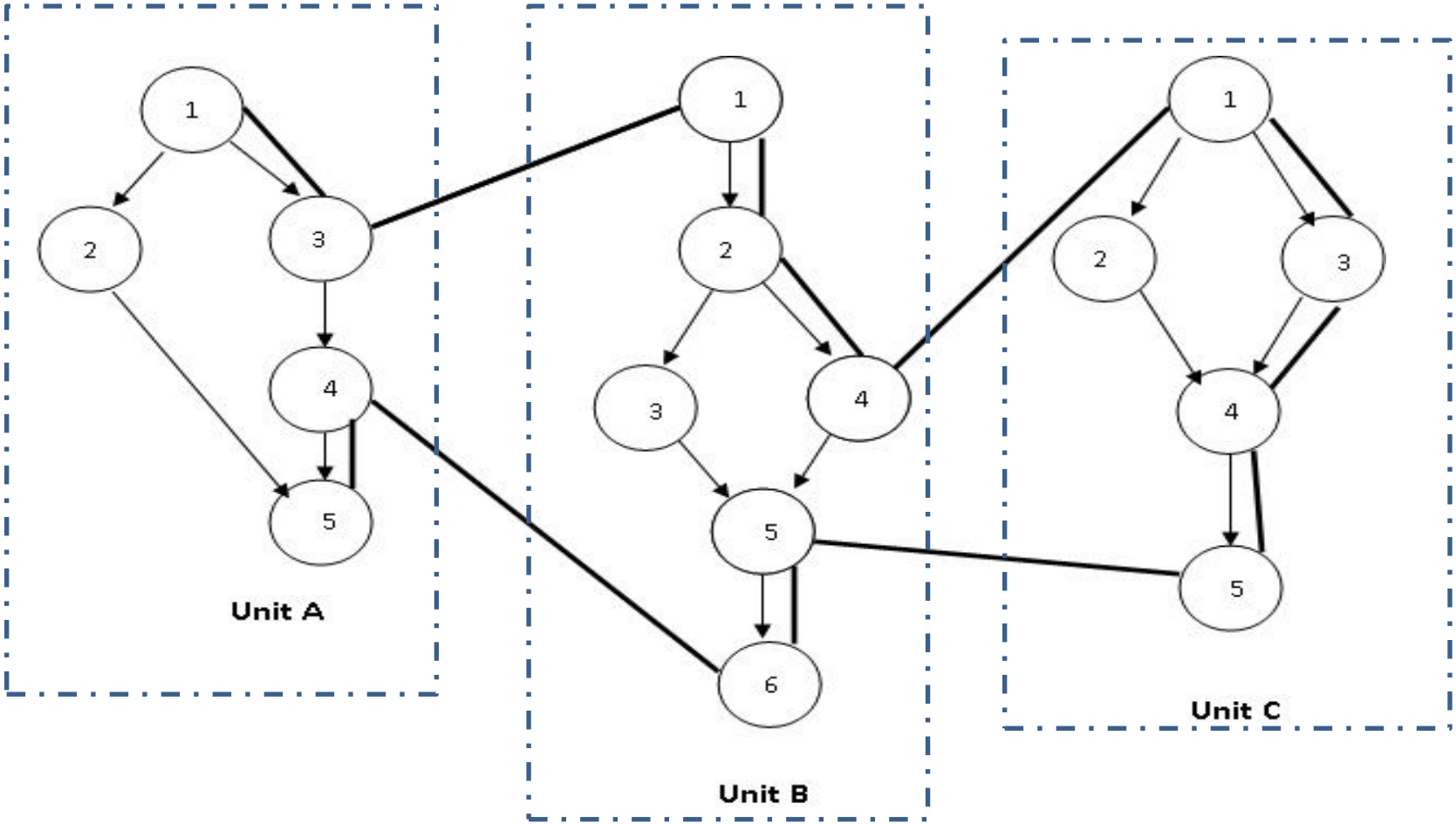
–MM-path

- It is a path consisting of MEPs and messages.
- The path shows the sequence of executable statements; it also crosses the boundary of a unit when a message is followed to call another unit.
- In other words, MM-path is a set of MEPs & transfer of control among different units in the form of messages.

– MM-Path Graph

- It can be defined as an extended flow graph where nodes are MEPs and edges are messages.
- It returns from the last called unit to the first unit where the call was made.
- In this graph (shown in next slide), messages are highlighted with thick lines.

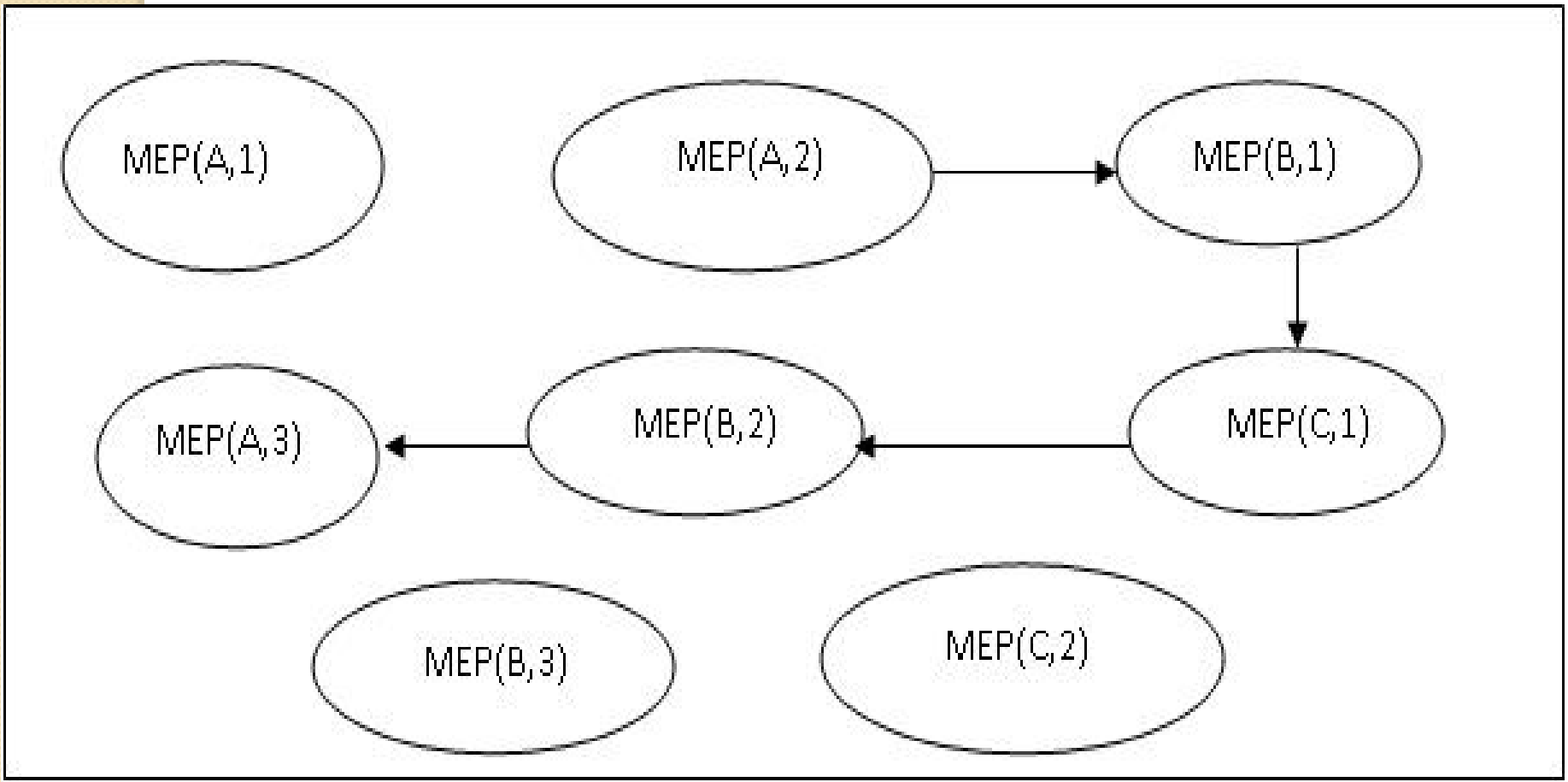
MM-Path



MM-Path Details

	Source nodes	Sink nodes	MEPs
Unit A	1,4	3,5	MEP(A,1)=<1,2,5> MEP(A,2)=<1,3> MEP(A,3)=<4,5>
Unit B	1,5	4,6	MEP(B,1)=<1,2,4> MEP(B,2)=<5,6> MEP(B,3)=<1,2,3,5,6>
Unit C	1	5	MEP(C,1)=<1,3,4,5> MEP(C,2)=<1,2,4,5>

MM-Path Graph



Function Testing

- When an integrated system is tested, all its specified functions and external interfaces are tested on the software.
- Every functionality of the system specified in the functions is tested according to its external specifications.
- An external specification is a precise description of the software behavior from the viewpoint of the outside world.

- Kit has defined function testing as the ***process of attempting to detect discrepancies between the functional specifications of a software and its actual behavior.***
- The objective of the function test is to measure the quality of the functional components of the system.
- Tests verify that the system behaves correctly from the user/business perspectives and functions according to the requirements, models or any other design paradigm.

- The function test must determine if each component or business event:

  - Performs in accordance to the specifications,
  - Responds correctly to all conditions that may present themselves by incoming events/data,
  - Moves data correctly from one business event to the next, and
  - Is initiated in the order required to meet the business objectives of the system

- Function testing can be performed after unit and integration testing, or whenever the development team thinks that the system has sufficient functionality to execute some tests.
- The test cases are executed such that the execution of the given test case against the software will exercise external functionality of certain parts.
- To keep a record of function testing, a function coverage metric is used.

- Function coverage can be measured with a function coverage matrix.
- It keeps track of those functions that exhibited the greatest number of errors.
- An effective function test cycle must have a defined set of processes and deliverables.
- The primary processes/ deliverables for requirement based function test are as follows:

–Test Planning

- The test leader with assistance from the test team defines the scope, schedule, and deliverables for the function test cycle.
- He delivers a test plan and a test schedule- these often undergo several revisions during the testing cycle.

–Partitioning/ functional Decomposition

- It is the process of breaking down a system into its functional components or functional areas.
- Another group in the organization takes responsibility for the functional decomposition of the system.
- If decompositions are deemed insufficient, then testing organization takes up the responsibility of decomposition.

–Requirement definition

- The testing organization needs specified requirements in the form of proper documents to proceed with the function test.
- These requirements need to be itemized under an appropriate functional partition.

–Test case design

- A tester designs and implements a test case to validate that the product performs in accordance with the requirements.
- These requirements need to be itemized under an appropriate functional partition and mapped to the requirements being tested.

–Traceability matrix formation

- Test cases need to be mapped back to the appropriate requirement.
- A function coverage matrix is prepared.
- This matrix is a table, listing specific functions to be tested, the priority for testing each function, and test cases required to test each function.
- Once all the aspects of the function have been tested by one or more test cases, then the test design activity for that function can be considered complete.

Function coverage matrix

Functions/Features	Priority	Test Cases
F1	3	T2, T4, T6
F2	1	T1, T3, T5

–Test case execution

- In all phases of testing, an appropriate set of test cases need to be executed and the result of those test cases recorded.
- So, the test case to be executed should be defined in the test plan.
- If the current application does not support the testing, then it should be deferred.

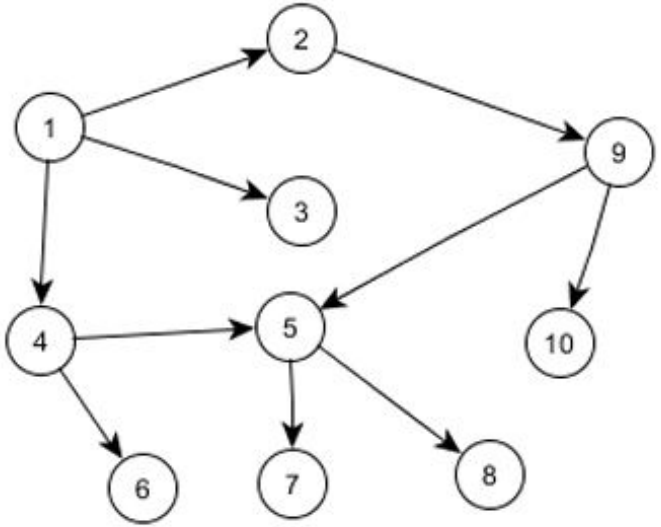
Summary

- Discussed call graph-based integration testing in detail.
  - Pair-wise Integration
  - Neighborhood Integration
- Explained path-based integration testing.
- Explained Function Testing in detail.
- Described the primary processes/ deliverables for requirement based function test.

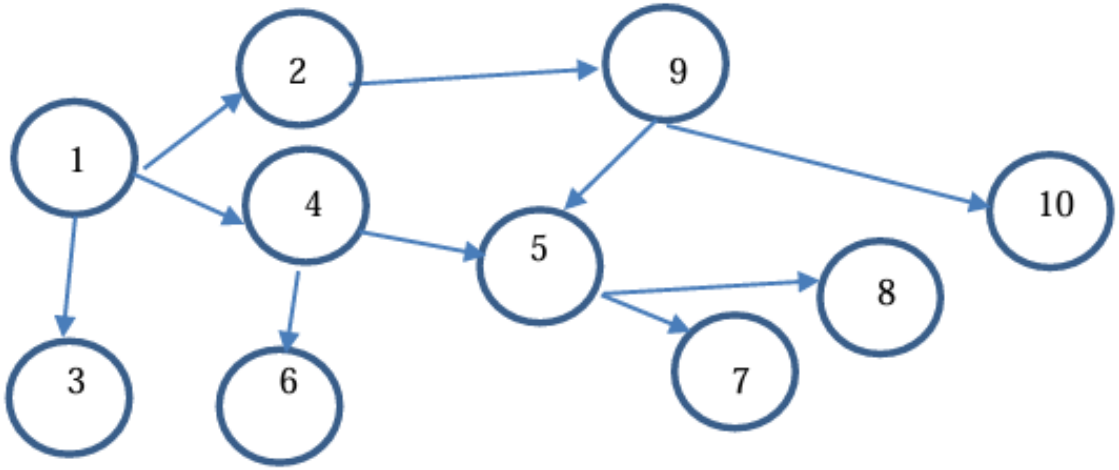
An application passes all unit tests but fails when two modules exchange data. Explain why this can happen and how Integration Testing can identify the problem.

?

Perform neighborhood integration testing for the call graph shown below and calculate the effort required for this testing.



Perform pair-wise integration testing using the following call graph. Show all the intermediate steps. Calculate the testing effort required for pair-wise integration testing for the following call graph.



References

1. Rajib Mall, Fundamentals of Software Engineering, (Chapter – 10), Fifth Edition, PHI Learning Pvt. Ltd., 2018.
2. Naresh Chauhan, Software Testing: Principles and Practices, (Chapter – 7), Second Edition, Oxford University Press, 2016.

Thank You